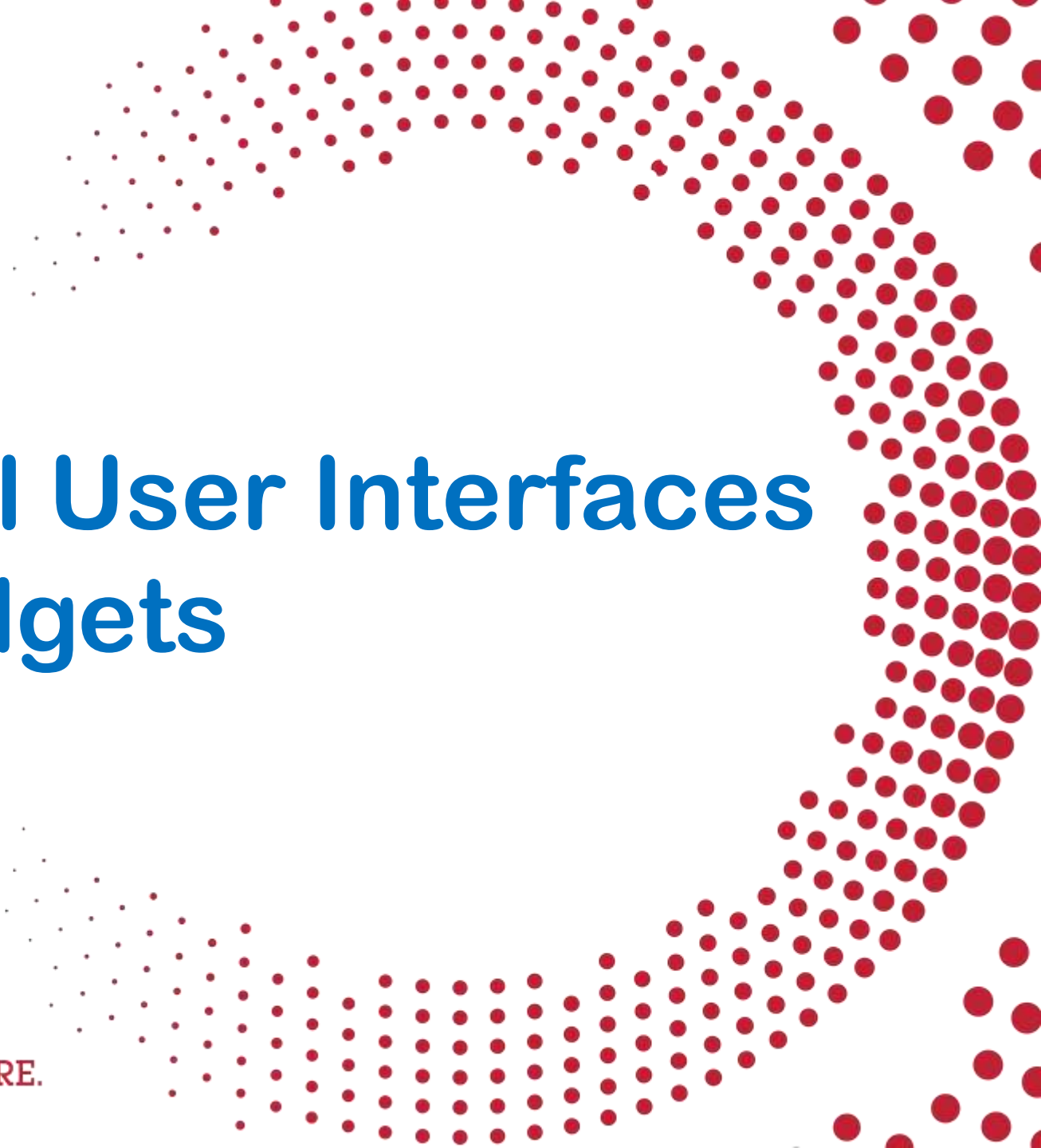




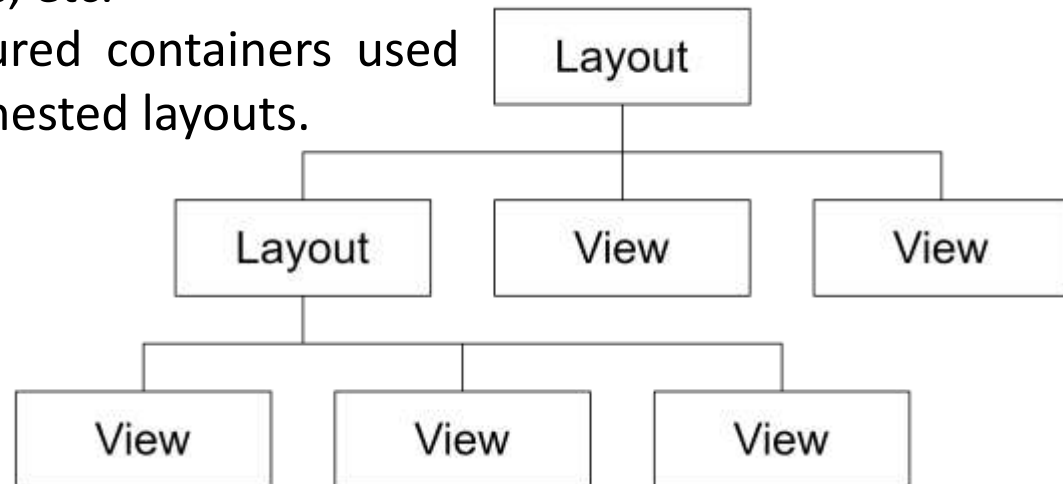
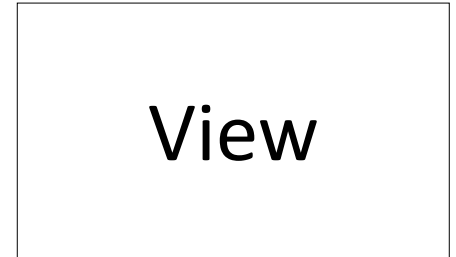
Graphical User Interfaces

Basic widgets

ONE LOVE. ONE FUTURE.

The View class

- The **View class** is the Android's most basic component from which user interfaces can be created. It acts as a container of displayable elements.
- A **View** occupies a rectangular area on the screen and is responsible for *drawing* and *event handling*.
- **Widgets** are subclasses of View. They are used to create interactive UI components such as buttons, checkboxes, labels, text fields, etc.
- **Layouts** are invisible structured containers used for holding other Views and nested layouts.



Using XML to represent UIs



Actual UI displayed by the app

Text version: [activity_main.xml](#) file →

```
<RelativeLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:paddingBottom="@dimen/activity_vertical_margin"
    android:paddingLeft="@dimen/activity_horizontal_margin"
    android:paddingRight="@dimen/activity_horizontal_margin"
    android:paddingTop="@dimen/activity_vertical_margin">

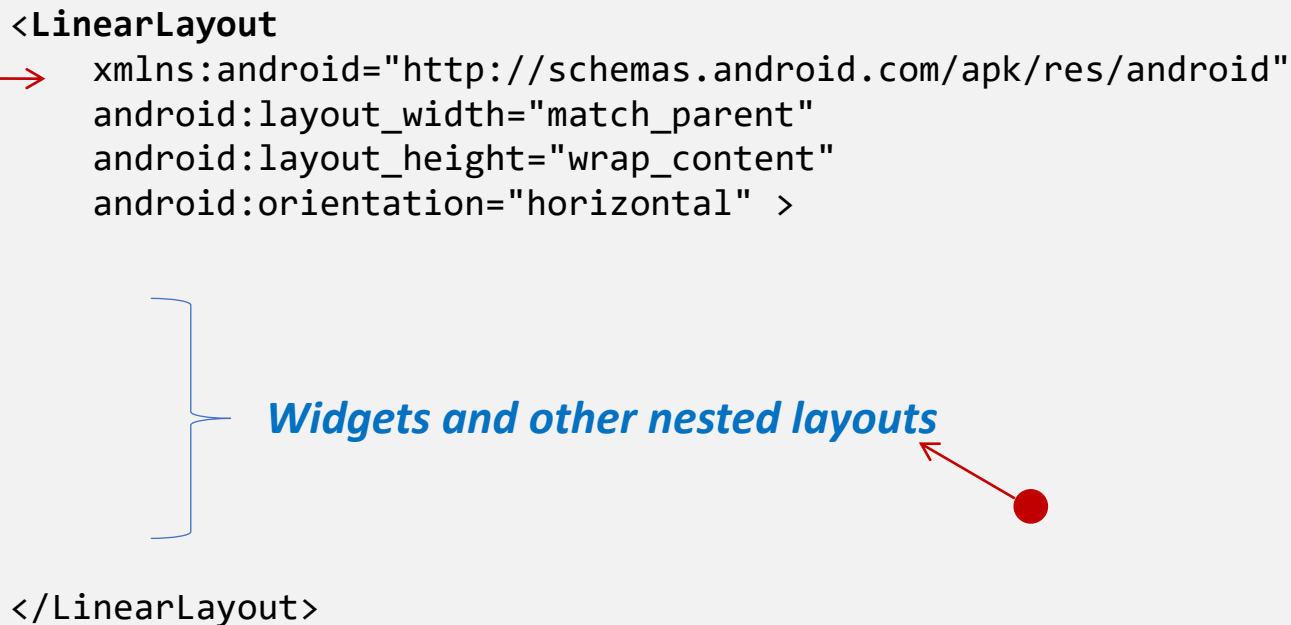
    <EditText
        android:id="@+id/editText1"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_alignParentTop="true"
        android:layout_centerHorizontal="true"
        android:layout_marginTop="36dp"
        android:text="@string/edit_user_name"
        android:ems="12" >
        <requestFocus />
    </EditText>

    <Button
        android:id="@+id/button1"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_below="@+id/editText1"
        android:layout_centerHorizontal="true"
        android:layout_marginTop="48dp"
        android:text="@string/btn_go" />

</RelativeLayout>
```

Nesting XML Layouts

- An Android's **XML view** file consists of a **layout** design holding a hierarchical arrangement of its contained elements.
- The inner elements could be basic widgets or user-defined nested layouts holding their own viewgroups.
- An Activity uses the `setContentView(R.layout.xmlfilename)` method to render a view on the device's screen.

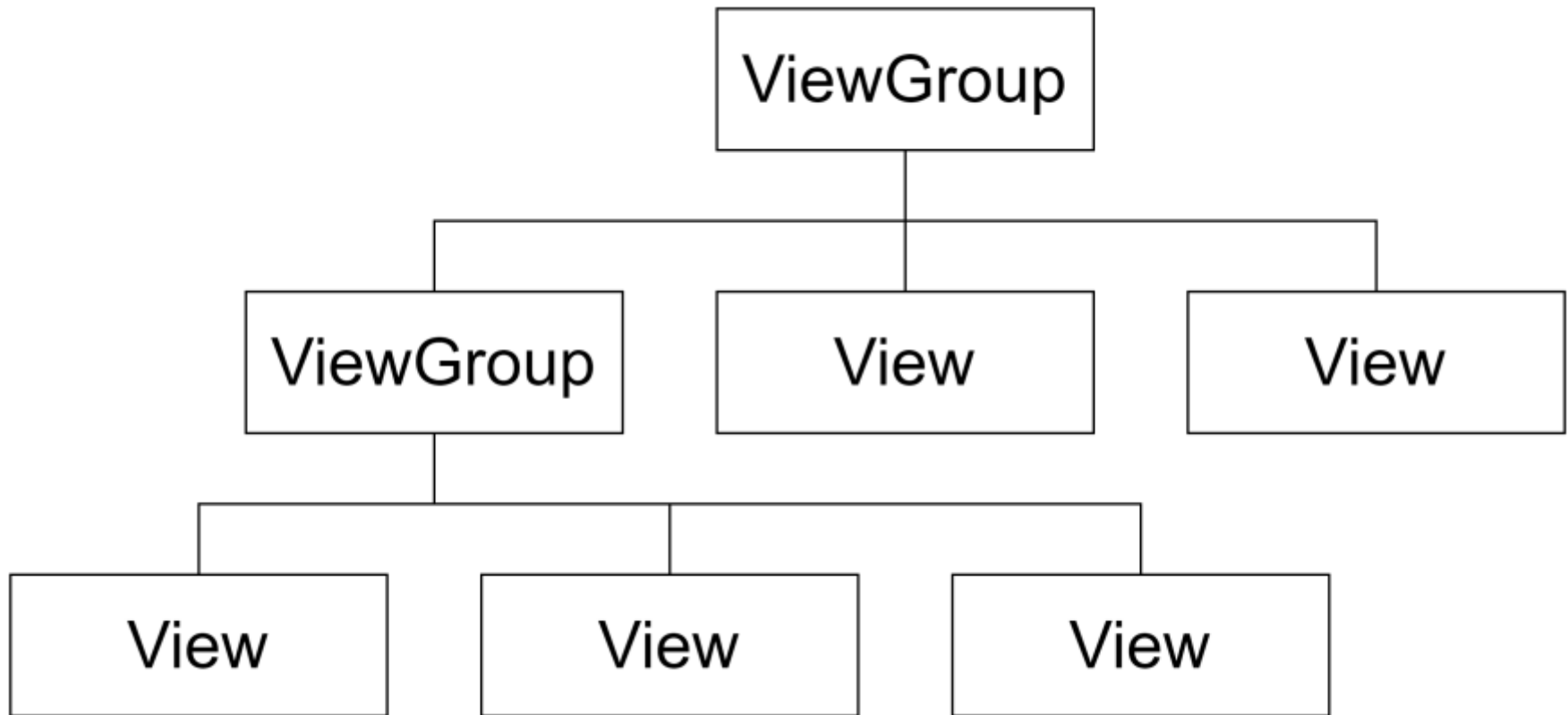


```
<LinearLayout
  xmlns:android="http://schemas.android.com/apk/res/android"
  android:layout_width="match_parent"
  android:layout_height="wrap_content"
  android:orientation="horizontal" >

  Widgets and other nested layouts

</LinearLayout>
```

Nesting XML Layouts



Setting Views to Work

Dealing with widgets & layouts typically involves the following operations:

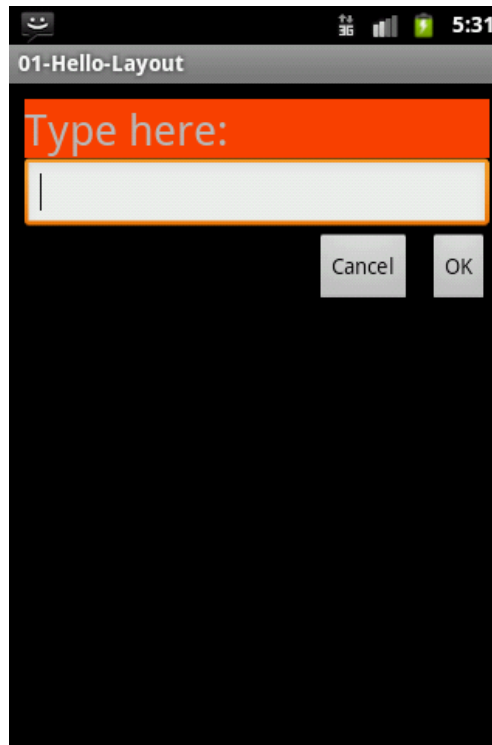
1. **Set properties:** For instance, when working with a *TextView* you set the background color, text, font, alignment, size, padding, margin, etc.
2. **Set up listeners:** For example, an image could be programmed to respond to various events such as: click, long-tap, mouse-over, etc.

A Sample of Common Android LAYOUTS



Linear Layout

A LinearLayout places its inner views either in horizontal or vertical disposition.



Relative Layout

A RelativeLayout is a ViewGroup that allows you to position elements relative to each other.



Table Layout

A TableLayout is a ViewGroup that places elements using a row & column disposition.

A Sample of Common Android WIDGETS



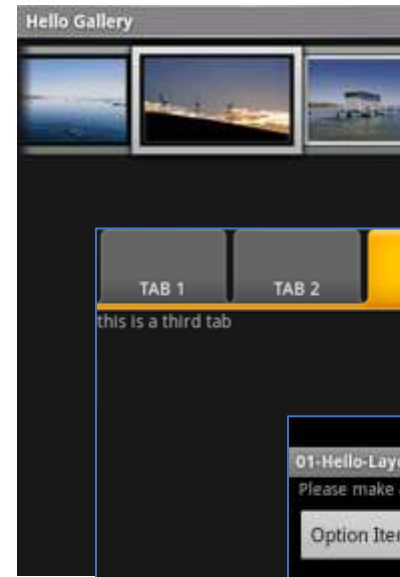
TimePicker
AnalogClock
DatePicker

A *DatePicker* is a widget that allows the user to select a month, day and year.



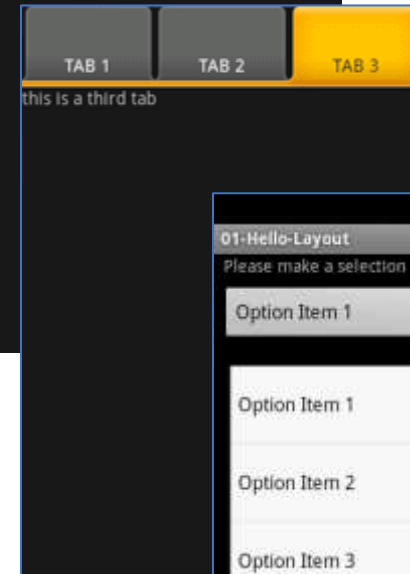
Form Controls

Includes a variety of typical form widgets, like:
buttons, image buttons, text fields, checkboxes and radio buttons.



GalleryView

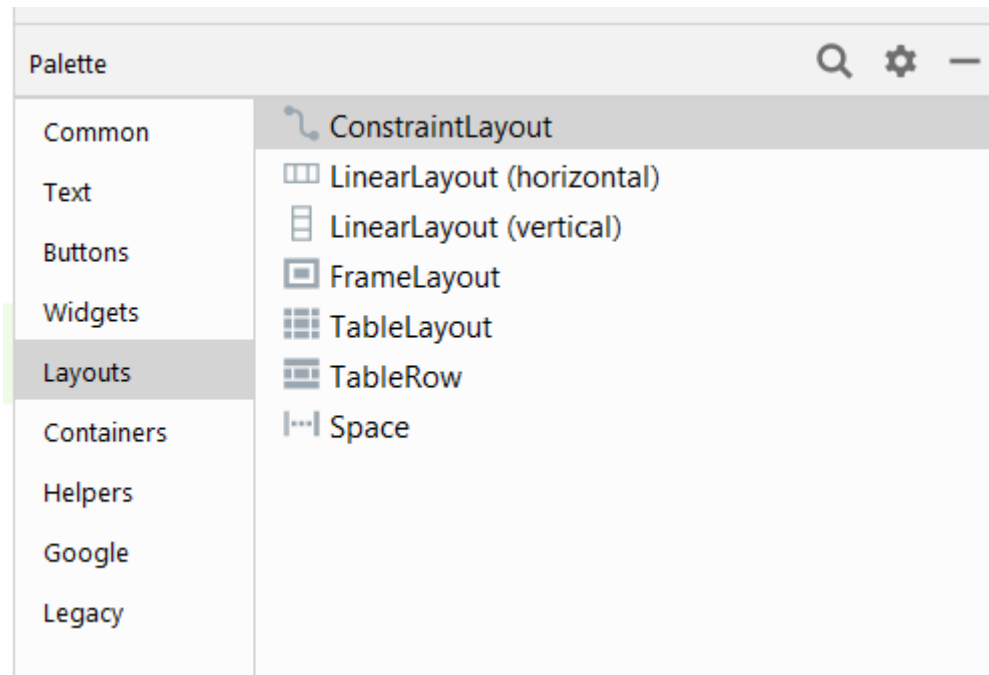
TabWidget



Spinner

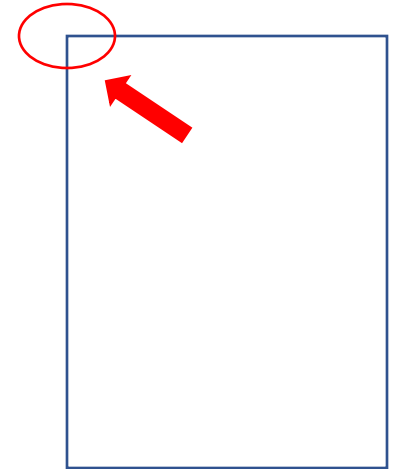
GUI Elements: The LAYOUT

- Android GUI *Layouts* are containers having a predefined structure and placement policy such as relative, linear horizontal, grid-like, etc.
- **Layouts can be nested**, therefore a cell, row, or column of a given layout could be another layout.



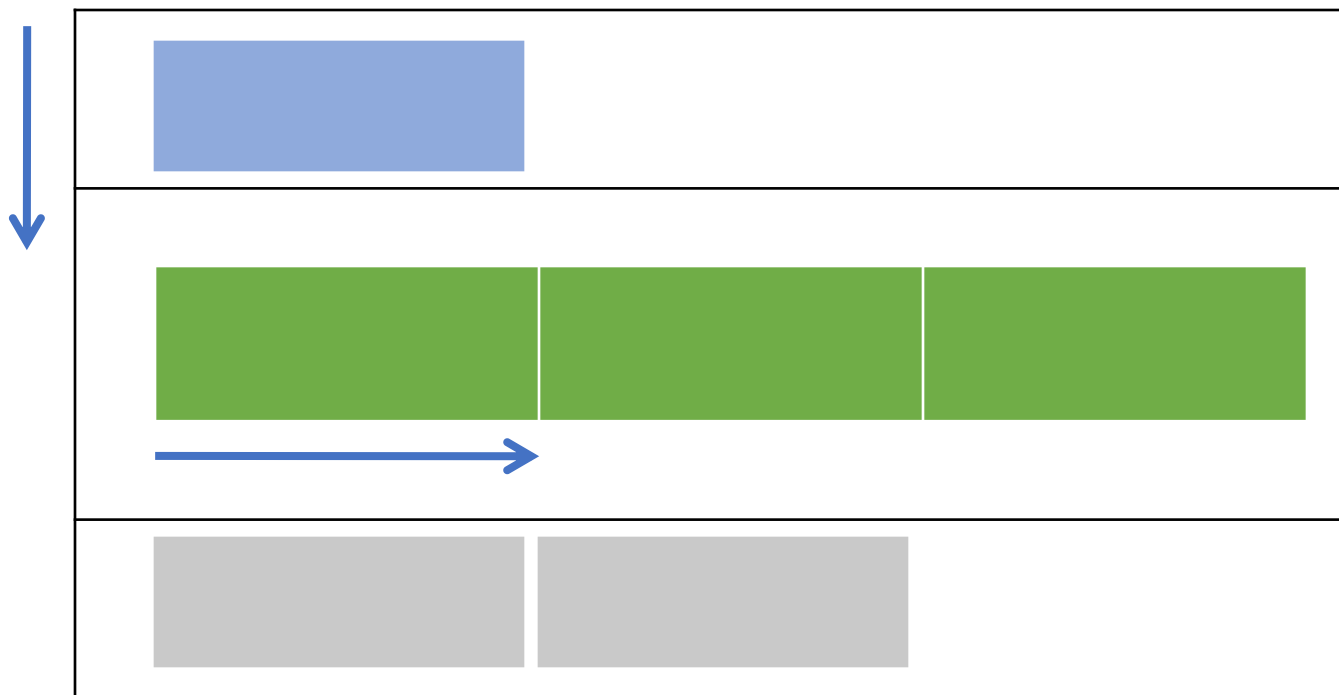
FrameLayout

- The **FrameLayout** is the simplest type of GUI container.
- It is useful as an *outermost* container holding a window.
- Allows you to define how much of the screen (high, width) is to be used.
- All its children elements are *aligned to the top left corner of the screen*.



LinearLayout

- The **LinearLayout** supports a filling strategy in which new elements are stacked either in a **horizontal** or **vertical** fashion.
- If the layout has a vertical orientation new *rows* are placed one on top of the other.
- A horizontal layout uses a side-by-side *column* placement policy.



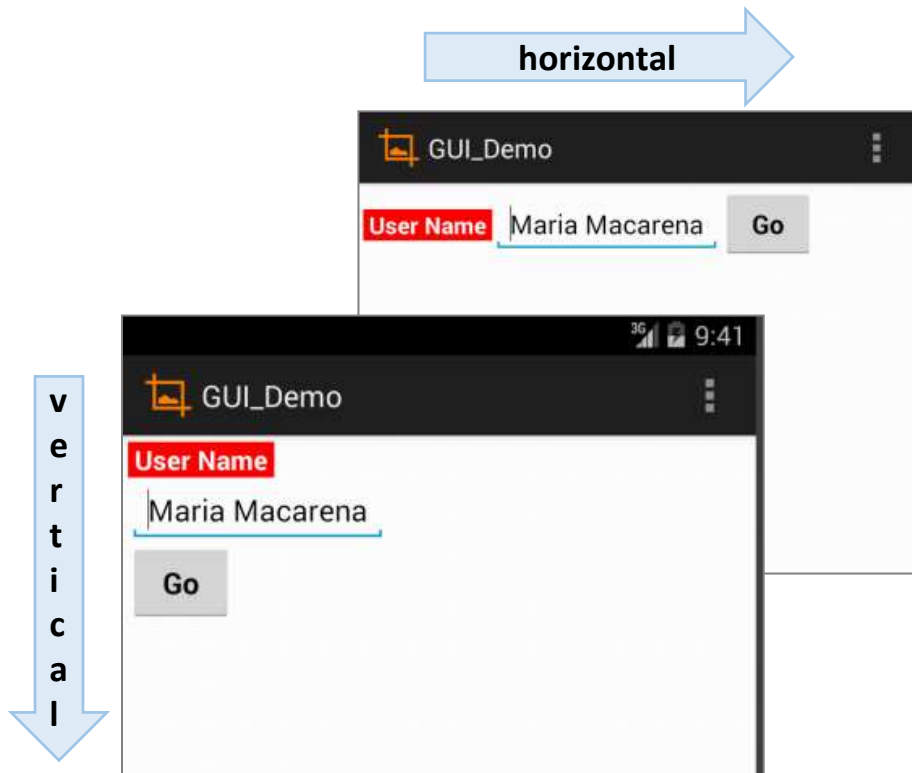
Setting Attributes

Configuring a **LinearLayout** usually requires you to set the following attributes:

- **orientation** (*vertical, horizontal*)
- **fill model** (*match_parent, wrap_contents*)
- **weight** (*0, 1, 2, ...n*)
- **gravity** (*top, bottom, center,...*)
- **padding** (*dp – dev. independent pixels*)
- **margin** (*dp – dev. independent pixels*)

LinearLayout : Orientation

The **android:orientation** property can be set to: **horizontal** for columns, or **vertical** for rows. Use *setOrientation()* for runtime changes.



```
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/and
roid"
    android:id="@+id/myLinearLayout"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="horizontal" ←
    android:padding="4dp" >

    <TextView
        android:id="@+id/labelUserName"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:background="#ffff0000"
        android:text=" User Name "
        android:textColor="#ffffff"
        android:textSize="16sp"
        android:textStyle="bold" />

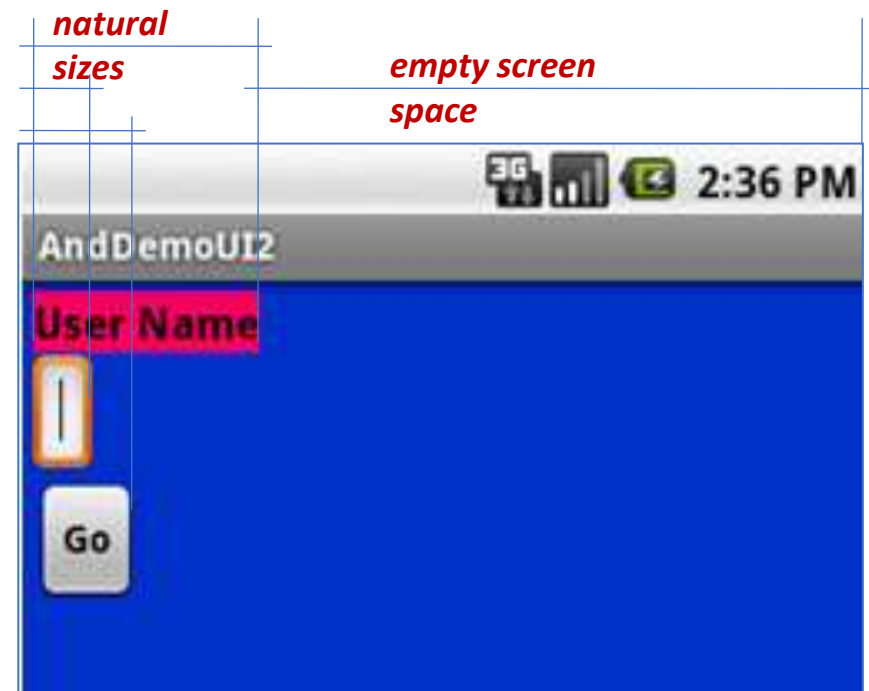
    <EditText
        android:id="@+id/ediName"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Maria Macarena"
        android:textSize="18sp" />

    <Button
        android:id="@+id/btnGo"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Go"
        android:textStyle="bold" />

</LinearLayout>
```

LinearLayout : Fill Model

- Widgets have a "**natural size**" based on their included text (*rubber band* effect).
- On occasions you may want your widget to have a specific space allocation (height, width) even if no text is initially provided (as is the case of the empty text box shown below).



LinearLayout : Fill Model

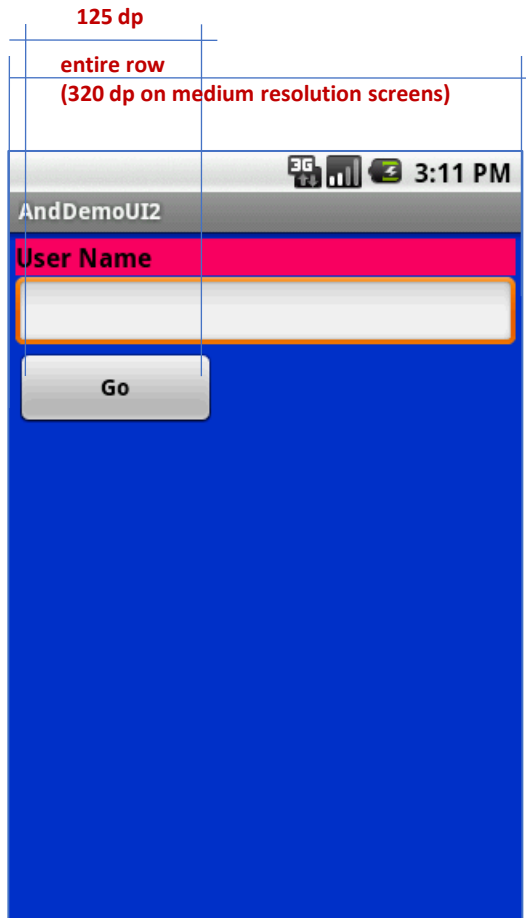
All widgets inside a LinearLayout **must** include 'width' and 'height' attributes.

android:layout_width
android:layout_height

Values used in defining height and width can be:

1. A specific dimension such as **125dp** (device independent pixels **dip**)
2. **wrap_content** indicates the widget should just fill up its natural space.
3. **match_parent** (previously called '**fill_parent**') indicates the widget wants to be as big as the enclosing parent.

LinearLayout : Fill Model



Medium resolution is: 320 x 480 dpi.
Shown on a Gingerbread device

```
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
android:id="@+id/myLinearLayout"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:background="#ff0033cc"
android:orientation="vertical"
android:padding="6dp" >
```

Row-wise

```
<TextView
    android:id="@+id/LabelUserName"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:background="#ffff0066"
    android:text="User Name"
    android:textColor="#ff000000"
    android:textSize="16sp"
    android:textStyle="bold" />
```

Use all the row

```
<EditText
    android:id="@+id/ediName"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:textSize="18sp" />
```

```
<Button
    android:id="@+id/btnGo"
    android:layout_width="125dp"
    android:layout_height="wrap_content"
    android:text="Go"
    android:textStyle="bold" />
```

Specific size: 125dp

```
</LinearLayout>
```

LinearLayout : Weight

The extra space left unclaimed in a layout could be assigned to any of its inner components by setting its **Weight** attribute.

Use **0** if the view should not be stretched. The bigger the weight the larger the extra space given to that widget.

Example

The XML specification for this window is similar to the previous example.

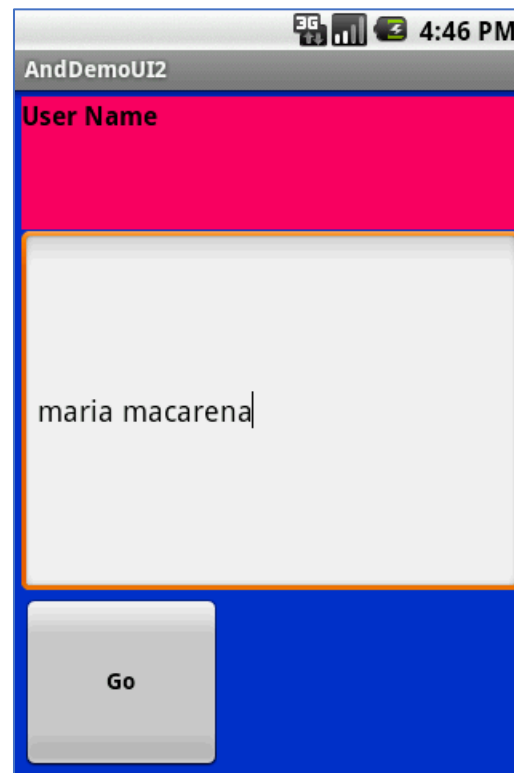
The TextView and Button controls have the additional property

`android:layout_weight="1"`

whereas the EditText control has

`android:layout_weight="2"`

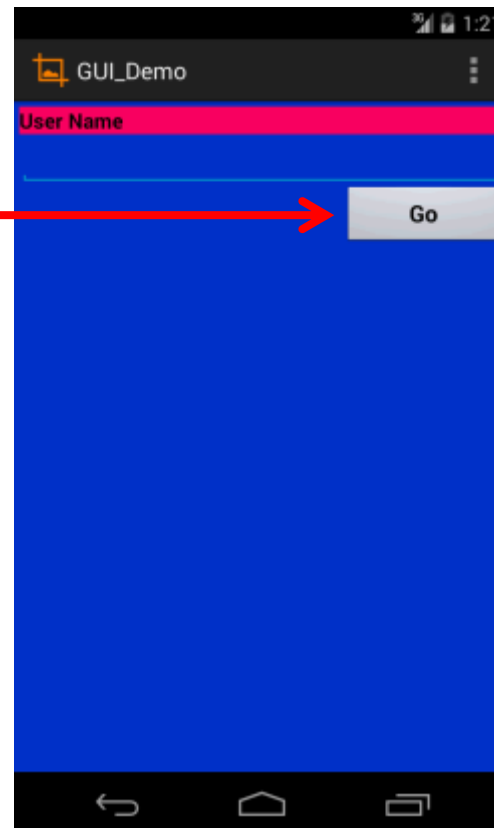
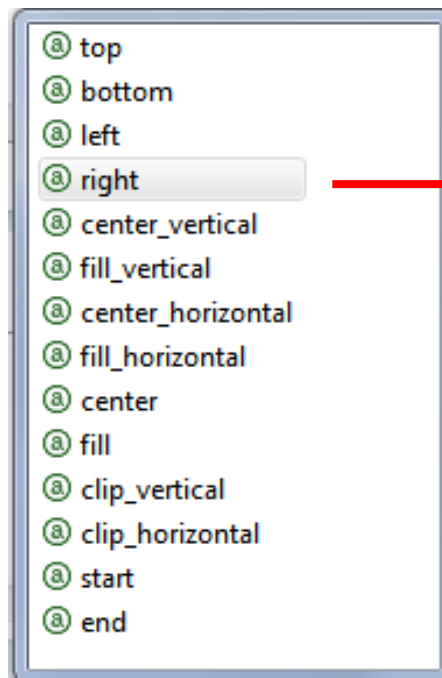
Remember, default value is 0



Takes: 2 / (1+1+2)
of the screen space

LinearLayout : Gravity

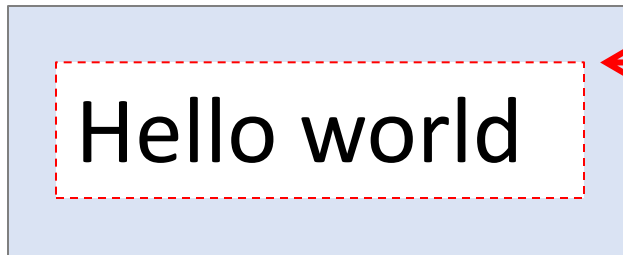
- **Gravity** is used to indicate how a control will align on the screen.
- By default, widgets are *left-* and *top*-aligned.
- You may use the XML property `android:layout_gravity="..."` to set other possible arrangements: *left, center, right, top, bottom*, etc.



Button has
right
layout_gravity

LinearLayout : Padding

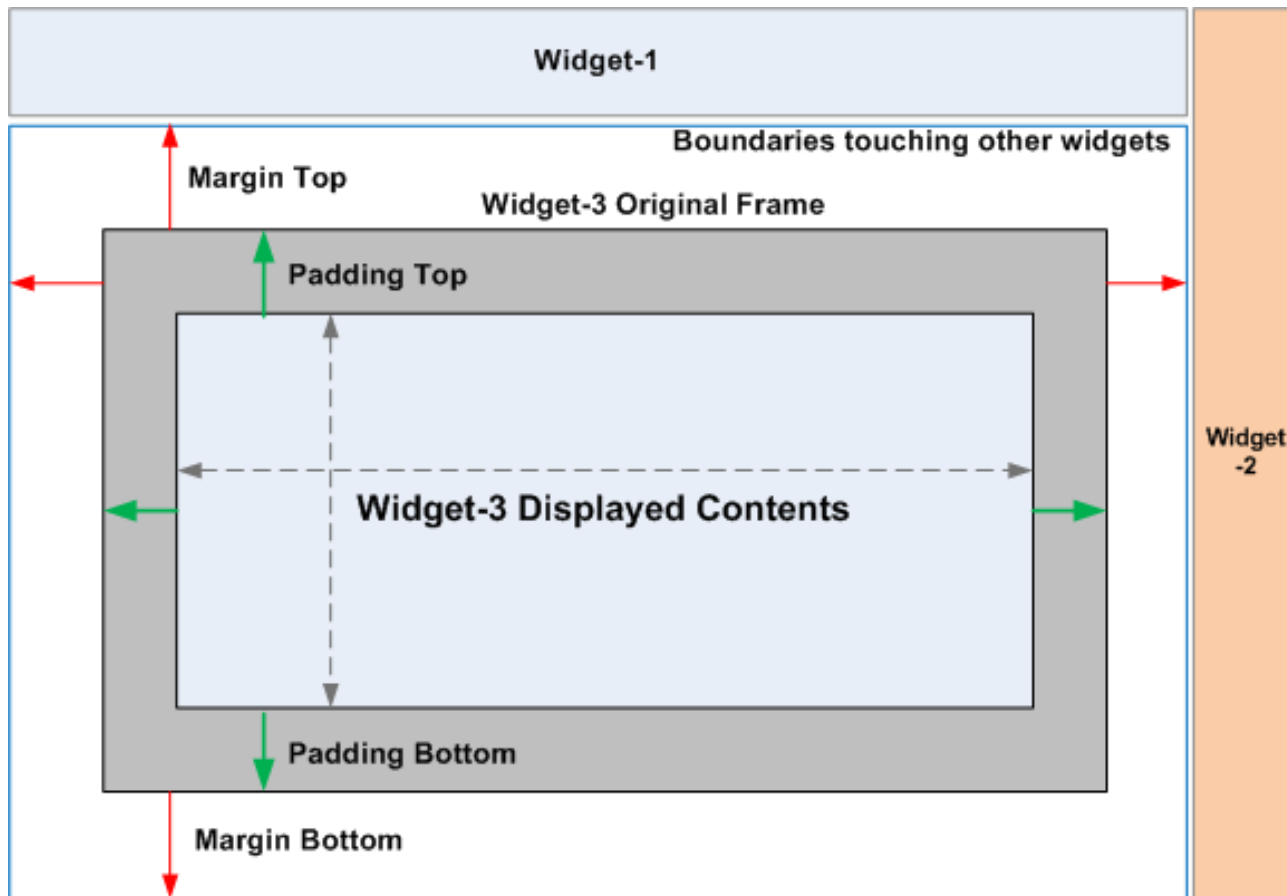
- The **padding** attribute specifies the widget's internal margin (in **dp** units).
- The internal margin is the extra space between the borders of the widget's "cell" and the actual widget contents.



The 'blue' surrounding space around the text represents the inner view's padding

LinearLayout : Padding and Margin

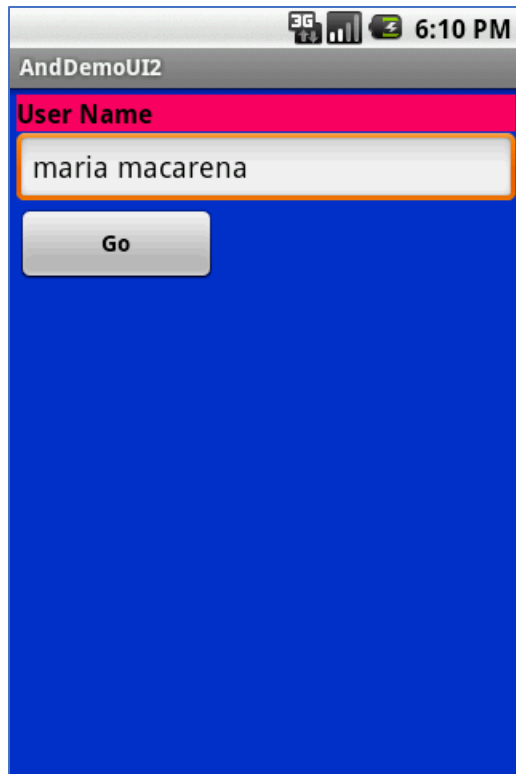
Padding and Margin represent the *internal* and *external* spacing between a widget and its included and surrounding context (respectively).



LinearLayout : Set internal margins using padding

Example:

The EditText box has been changed to include 30dp of padding all around

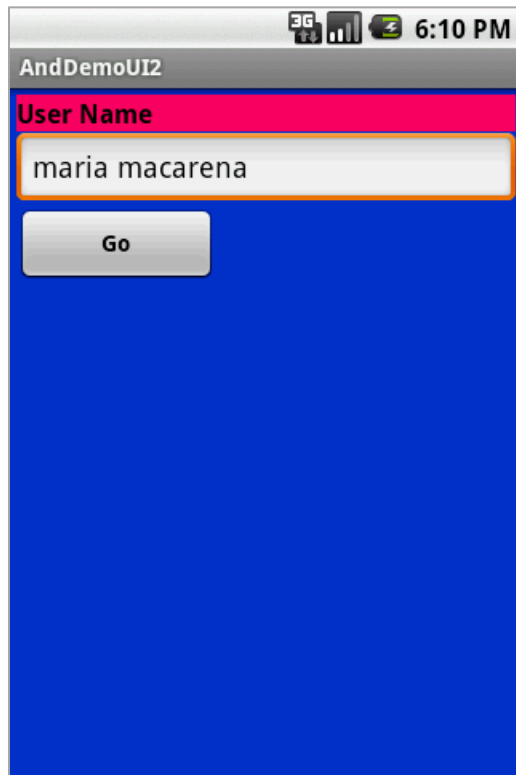


```
<EditText
    android:id="@+id/ediName"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:textSize="18sp"
    android:padding="30dp" />
```

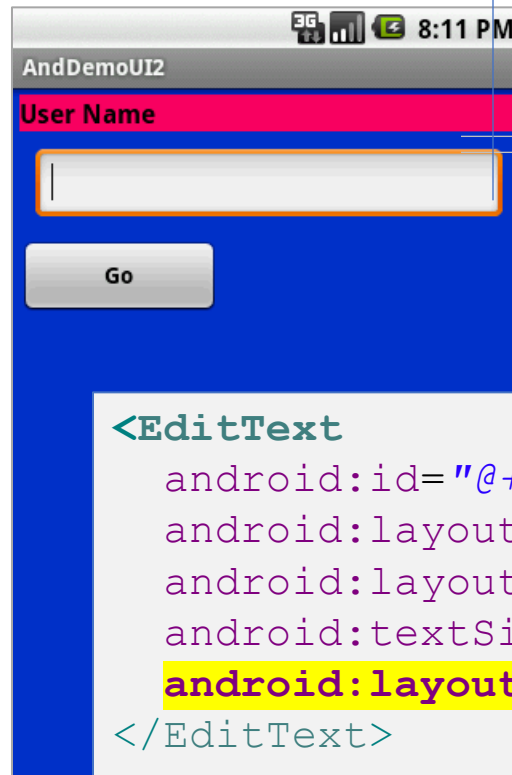
...

LinearLayout : Set External Margins

- Widgets –by default– are closely displayed next to each other.
- To increase space between them use the **android:layout_margin** attribute



Using default spacing between widgets

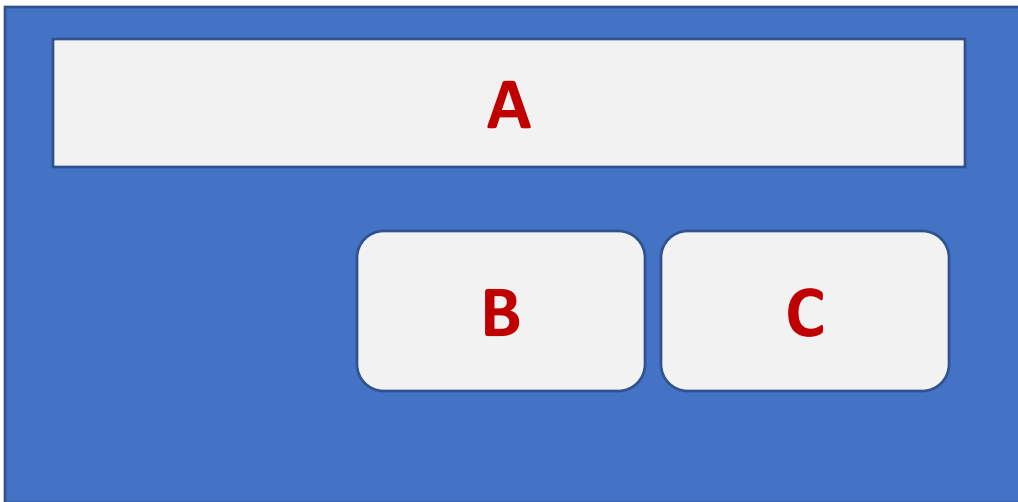


Increased inter-widget space

```
<EditText
    android:id="@+id/ediName"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:textSize="18sp"
    android:layout_margin="6dp">
</EditText>
...
```

Relative Layout

The placement of a widget in a **RelativeLayout** is based on its *positional relationship* to other widgets in the container as well as the parent container.



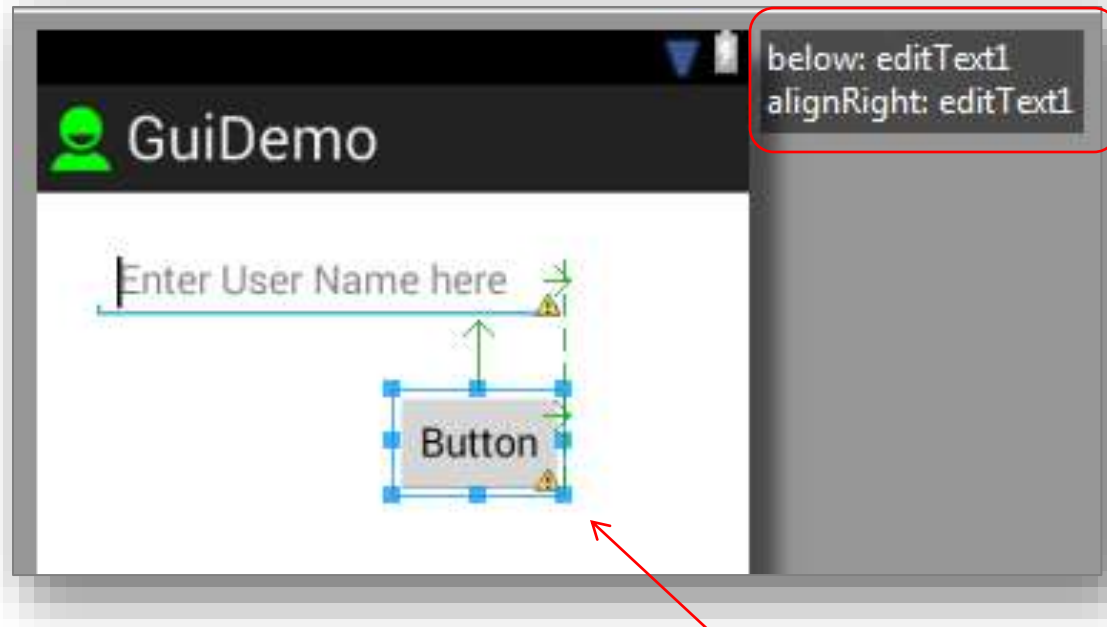
Example:

A is by the parent's top

C is below **A**, to its right

B is below **A**, to the left of **C**

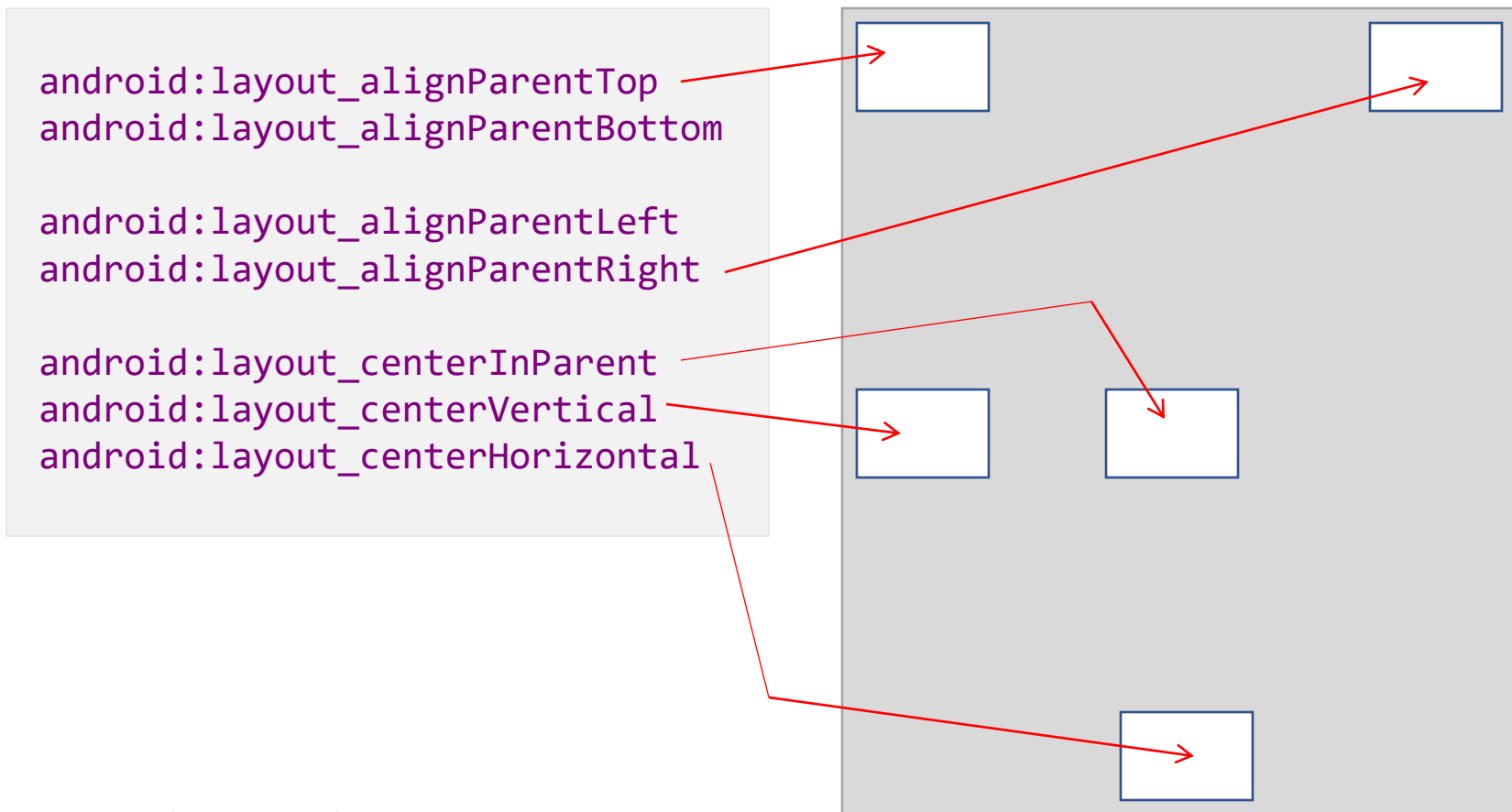
Relative Layout - Example



Location of the button is expressed in reference to its *relative* position with respect to the EditText box.

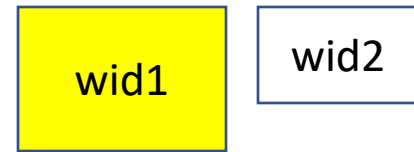
Relative Layout - Referring to the container

Below there is a sample of various positioning *XML boolean properties* (**true/false**) which are useful for collocating a widget based on the location of its **parent** container.

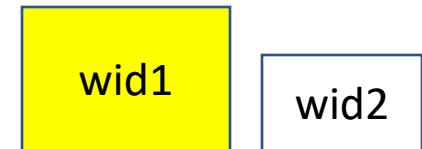


Relative Layout - Referring to Other Widgets

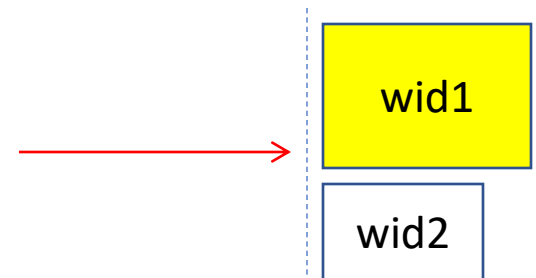
`android:layout_alignTop="@+id/wid1"`



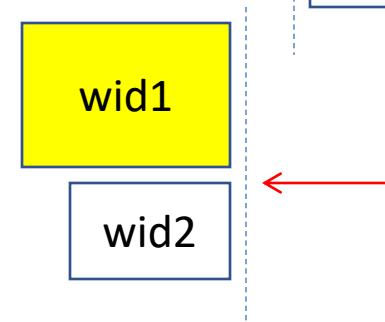
`android:layout_alignBottom="@+id/wid1"`



`android:layout_alignLeft="@+id/wid1"`



`android:layout_alignRight="@+id/wid1"`



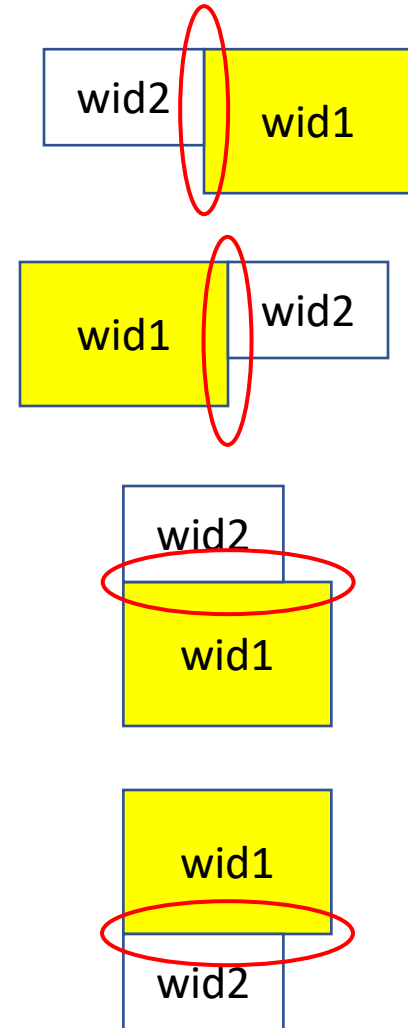
Relative Layout - Referring to Other Widgets

```
android:layout_toLeftOf="@+id/wid1"
```

```
android:layout_toRightOf="@+id/wid1"
```

```
android:layout_above="@+id/wid1"
```

```
android:layout_below="@+id/wid1"
```



Relative Layout - Referring to Other Widgets

When using relative positioning you need to:

1. Use identifiers (**android:id** attributes) on *all elements* that you will be referring to.
2. XML elements are named using the prefix: **@+id/...** For instance an EditText box could be called: **android:id="@+id/txtUserName"**
3. You must refer only to widgets that have been already defined. For instance a new control to be positioned below the *txtUserName* EditText box could refer to it using: **android:layout_below="@+id/txtUserName"**

Relative Layout - Example

<RelativeLayout

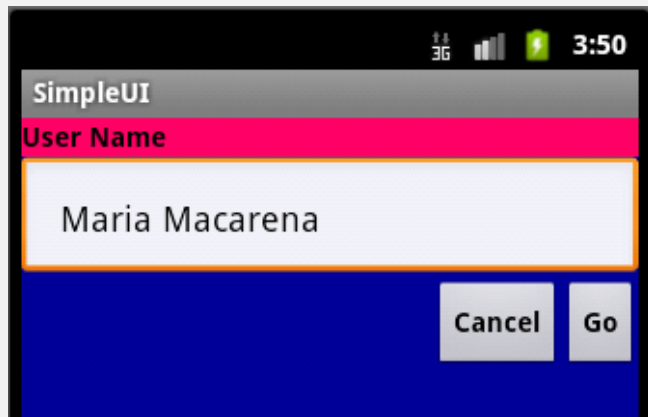
```
xmlns:android="http://schemas.android.com/apk/res/android"
```

```
android:id="@+id/myRelativeLayout"  
android:layout_width="match_parent"  
android:layout_height="match_parent"  
android:background="#ff000099" >
```

<TextView

```
android:id="@+id/lblUserName"  
android:layout_width="match_parent"  
android:layout_height="wrap_content"  
android:layout_alignParentLeft="true"  
android:layout_alignParentTop="true"  
android:background="#ffff0066"  
android:text="User Name"  
android:textColor="#ff000000"  
android:textStyle="bold" >
```

```
</TextView>
```



<EditText

```
android:id="@+id/txtUserName"  
android:layout_width="match_parent"  
android:layout_height="wrap_content"  
android:layout_alignParentLeft="true"  
android:layout_below="@+id/lblUserName"  
android:padding="20dp" >
```

```
</EditText>
```

<Button

```
android:id="@+id/btnGo"  
android:layout_width="wrap_content"  
android:layout_height="wrap_content"  
android:layout_alignRight="@+id/txtUserName"  
android:layout_below="@+id/txtUserName"  
android:text="Go"  
android:textStyle="bold" >
```

```
</Button>
```

<Button

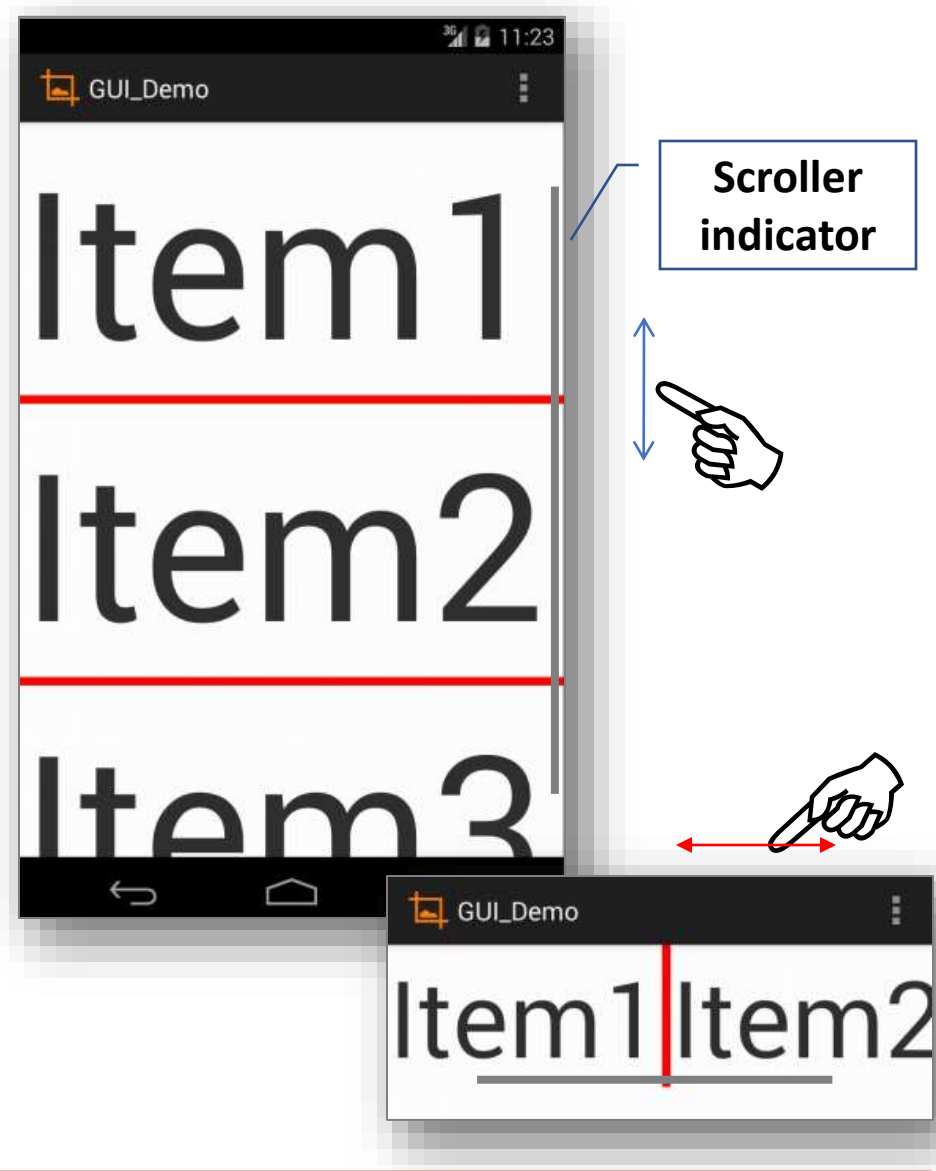
```
android:id="@+id/btnCancel"  
android:layout_width="wrap_content"  
android:layout_height="wrap_content"  
android:layout_below="@+id/txtUserName"  
android:layout_toLeftOf="@+id/btnGo"  
android:text="Cancel"  
android:textStyle="bold" >
```

```
</Button>
```

```
</RelativeLayout>
```

ScrollView Layout (Vertical & Horizontal)

- The **ScrollView** control is useful in situations in which we have *more data to show* than what a single screen could display.
- **ScrollViews** provide a vertical sliding (up/down) access to the data.
- The **HorizontalScrollView** provides a similar left/right sliding mechanism)
- Only a portion of the user's data can be seen at one time, however the rest is available for viewing.



Example: Vertical ScrollView Layout

```
<ScrollView xmlns:android="
http://schemas.android.com/apk/res/android"
    android:id="@+id/myVerticalScrollView1"
    android:layout_width="match_parent"
    android:layout_height="match_parent" >
```

<LinearLayout

```
    android:id="@+id/myLinearLayoutVertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical" >
```

<TextView

```
        android:id="@+id/textView1"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Item1"
        android:textSize="150sp" />
```

<View

```
            android:layout_width="match_parent"
            android:layout_height="6dp"
            android:background="#ffff0000" />
```

<TextView

```
        android:id="@+id/textView2"
        android:layout_width="match_parent"
```

```
        android:layout_height="wrap_content"
        android:text="Item2"
        android:textSize="150sp" />
```

<View

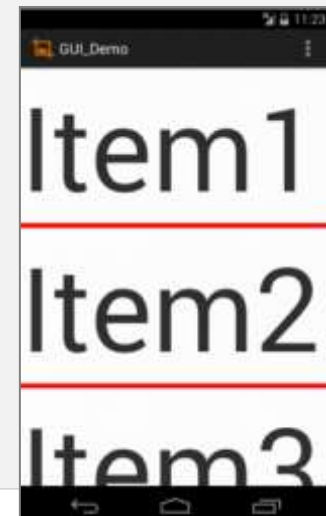
```
        android:layout_width="match_parent"
        android:layout_height="6dp"
        android:background="#ffff0000" />
```

<TextView

```
        android:id="@+id/textView3"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Item3"
        android:textSize="150sp" />
```

</LinearLayout>

</ScrollView>



Example: HorizontalScrollView Layout

<HorizontalScrollView

```
xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/myHorizontalScrollView1"
    android:layout_width="match_parent"
    android:layout_height="wrap_content" >
```

<LinearLayout

```
    android:id="@+id/myLinearLayoutVertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="horizontal" >
```

<TextView

```
        android:id="@+id/textView1"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Item1"
        android:textSize="75sp" />
```

<View

```
        android:layout_width="6dp"
        android:layout_height="match_parent"
        android:background="#ffff0000" />
```

<TextView

```
        android:id="@+id/textView2"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Item2"
        android:textSize="75sp" />
```

<View

```
        android:layout_width="6dp"
        android:layout_height="match_parent"
        android:background="#ffff0000" />
```

<TextView

```
        android:id="@+id/textView3"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Item3"
        android:textSize="75sp" />
```

</LinearLayout>

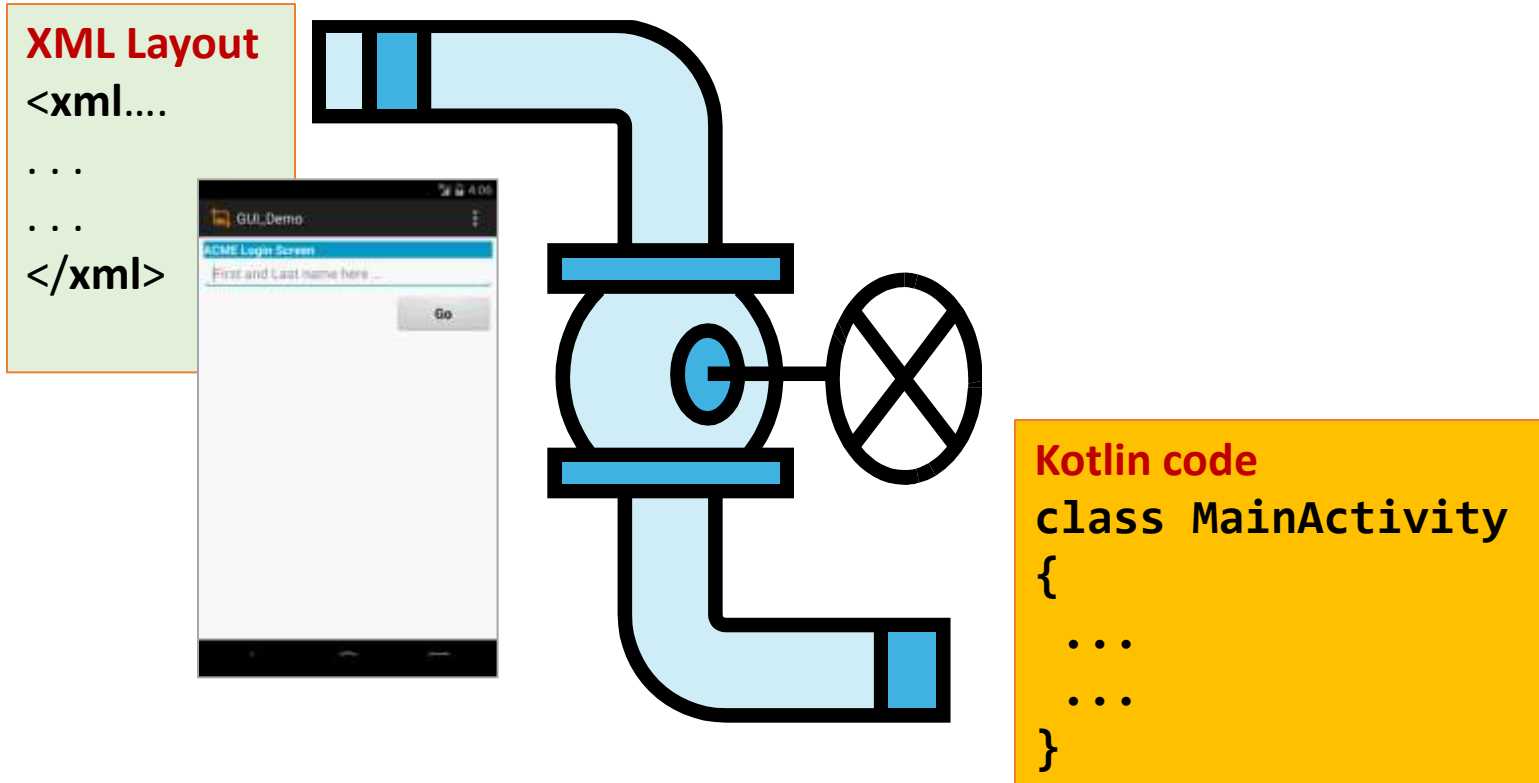
</HorizontalScrollView>



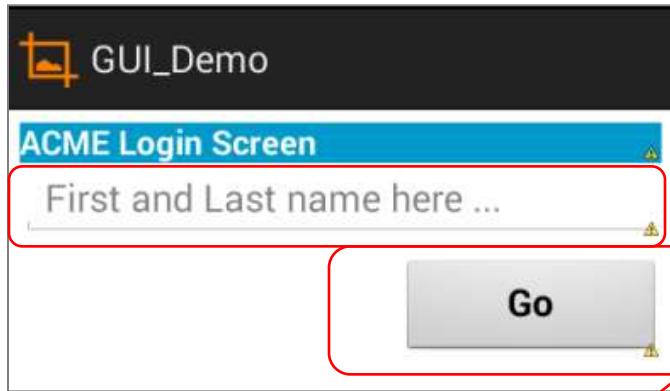
Connecting layouts to Kotlin code

PLUMBING.

You must 'connect' functional XML elements –such as buttons, text boxes, check boxes - with their equivalent Kotlin objects. This is typically done in the **onCreate(...)** method of your main activity. After all the connections are made and programmed, your app should be ready to interact with the user.



Connecting Layouts to Kotlin code



```
<!-- XML LAYOUT -->
<LinearLayout
    android:id="@+id/myLinearLayout"
    ... >

    <TextView
        android:text="ACME Login Screen"
        ... />

    <EditText
        android:id="@+id/edtUserName"
        ... />

    <Button
        android:id="@+id/btnGo"
        ... />
</LinearLayout>
```

Kotlin code

```
class MainActivity : AppCompatActivity() {

    lateinit var edtUserName: EditText
    lateinit var btnGo: Button

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        edtUserName = findViewById(R.id.edtUserName)
        btnGo = findViewById(R.id.btnGo)
    }
}
```

What is the meaning of an Android Context?

On Android, a **Context** defines a logical **workspace** on which an app can load and access resources.

- When a widget is created, it is attached to a particular Context. By means of its affiliation to that environment, it then could access other members of the hierarchy on which it has been collocated.
- For a simple '*one activity app*' - say MainActivity - the property **applicationContext** and the reference **MainActivity.this** return the same result.
- An application could have **several activities**. Therefore, for a *multi-activity* app we have one app context, and a context for each of its activities, each good for accessing what is available in *that context*.

Connecting Layouts to Kotlin code

Assume the UI in *res/layout/activity_main.xml* has been created. This layout could be called by an application using the statement

```
setContentView(R.layout.activity_main)
```

Individual XML defined widgets, such as *btnGo* is later associated to the application using the statement `findViewById(...)` as in

```
val btnGo: Button = findViewById(R.id.btnGo)
```

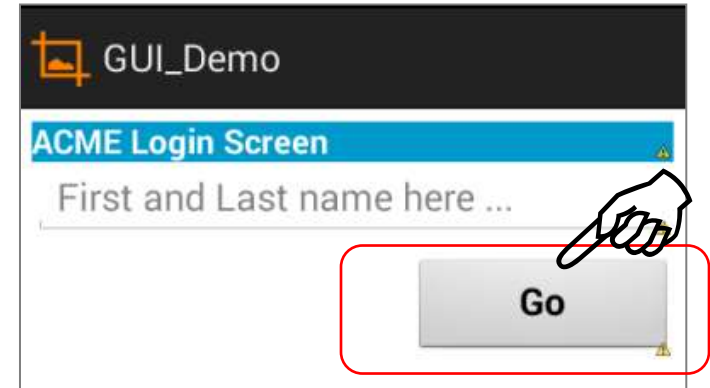
Where **R** is a class automatically generated to keep track of resources available to the application. In particular **R.id...** is the collection of widgets defined in the XML layout (Use Eclipse's Package Explorer, look at your **/gen/package/R.java** contents).

A Suggestion: The widget's identifiers used in the XML layout and Java code could be the same. It is convenient to add a prefix to each identifier indicating its nature. Some options are *txt*, *btn*, *edt*, *rad*, *chk*, etc. Try to be consistent.

Connecting Layouts to Kotlin code

Attaching Listeners to Widgets

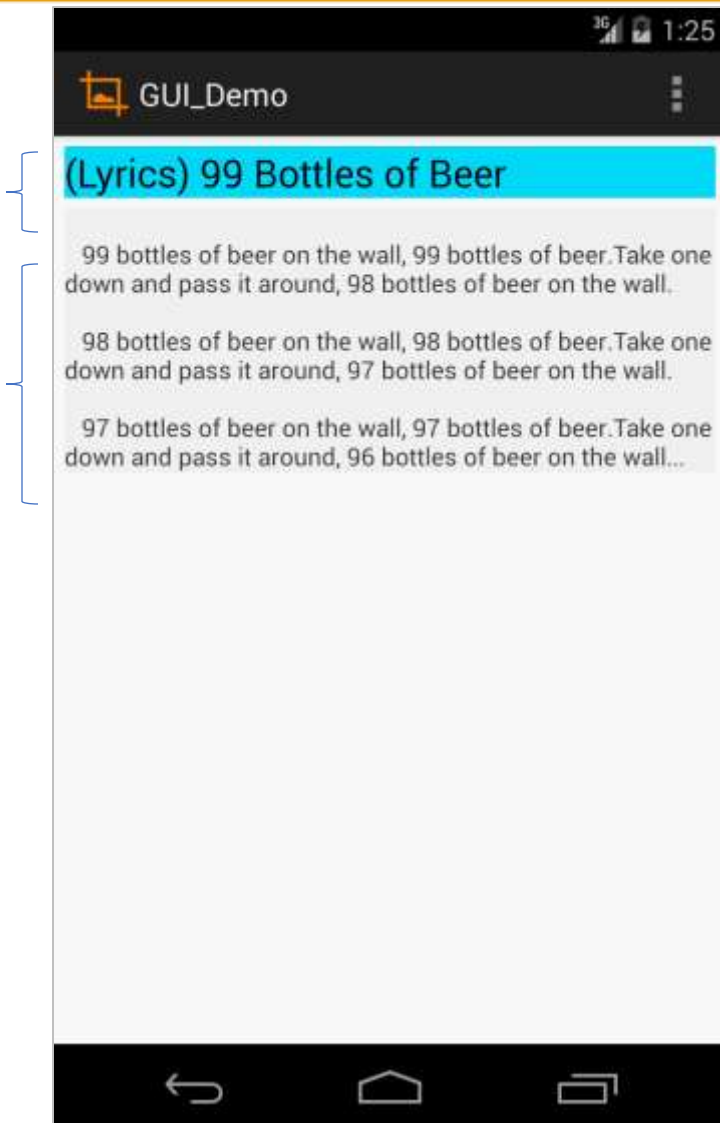
Consider the screen on the right. To make its 'Go' button widget be responsive to the user's pushing of that button, we may add a listener for the **click event**.



```
val btnGo: Button = findViewById(R.id.btnGo)
btnGo.setOnClickListener {
    // put some more logic here
}
```

Note: Other common 'listeners' watch for events such as: `textChanged`, `tap`, `long-press`, `select`, `focus`, etc.

Basic Widgets: TextViews



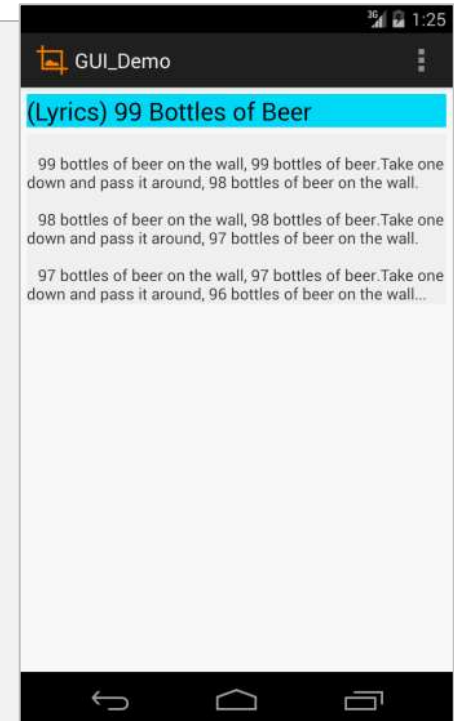
- In Android a **label** or **text-box** is called a **TextView**.
- A TextView is typically used for showing a caption or a text message.
- TextViews are *not* editable, therefore they take no input.
- The text to be shown may include the `\n` formatting character (newLine)
- You may also use HTML formatting by setting the text to:
`Html.fromHtml("your html string")`

Basic Widgets: Example - TextViews

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="6dp" >

    <TextView
        android:id="@+id/textView1"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:background="@color/holo_blue_bright"
        android:text="(Lyrics) 99 Bottles of Beer"
        android:textAppearance="?android:attr/textAppearanceLarge" />

    <TextView
        android:id="@+id/textView2"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="6dp"
        android:background="@color/gray_light"
        android:text="\n\t99 bottles of beer on the wall, 99 bottles of beer.Take one down and
pass it around, 98 bottles of beer on the wall.\n\n\t98 bottles of beer on the wall, 98 bottles
of beer.Take one down and pass it around, 97 bottles of beer on the wall. \n\n\t97 bottles of
beer on the wall, 97 bottles of beer.Take one down and pass it around, 96 bottles of beer on
the wall... "
        android:textSize="14sp" />
</LinearLayout>
```



Basic Widgets: TextViews

- * Set up color (text, background)
 - + In XML: RGB, ARGB, from resource
 - + In Kotlin code: from resource, from Color class
- * Get content from strings.xml resource file
 - + In XML: @string
 - + In Kotlin code: R.string
- * Set custom font from file in the assets folder

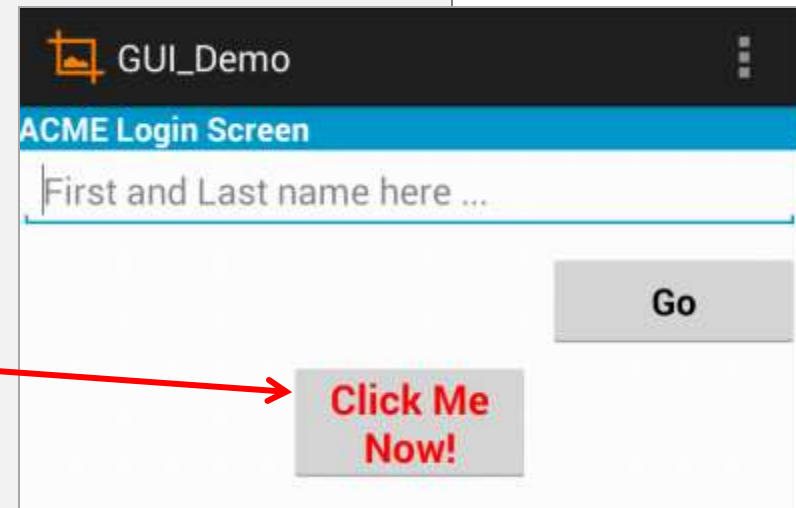
```
val textView: TextView = findViewById(R.id.textView)  
textView.typeface = Typeface.createFromAsset(assets, "fontname.ttf")
```

Basic Widgets: Buttons

- A **Button** widget allows the simulation of a GUI clicking action.
- **Button** is a subclass of **TextView**. Therefore formatting a button's face is similar to the setting of a **TextView**.
- You may alter the default behavior of a button by providing a custom *drawable.xml* specification to be applied as background. In those specs you indicate the shape, color, border, corners, gradient, and behavior based on states (pressed, focused). More on this issue in the appendix.

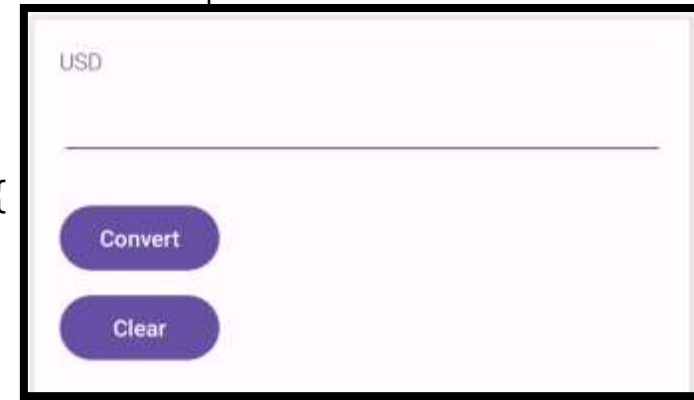
<Button

```
android:id="@+id/btnClickMeNow"  
android:layout_width="120dp"  
android:layout_height="wrap_content"  
android:layout_gravity="center"  
android:layout_marginTop="5dp"  
android:gravity="center"  
android:padding="5dp"  
android:text="Click Me Now!"  
android:textColor="#ffff0000"  
android:textSize="20sp"  
android:textStyle="bold" />
```



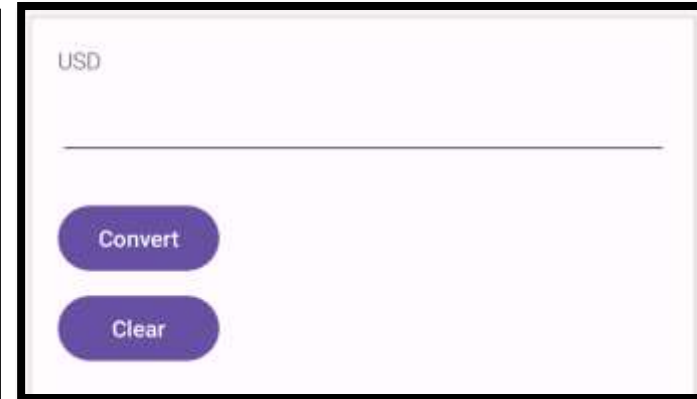
Example: Connecting buttons

```
class MainActivity : AppCompatActivity() {  
  
    lateinit var edtInput: EditText  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
  
        edtInput = findViewById(R.id.edtInput)  
  
        val btnConvert: Button = findViewById(R.id.btnConvert)  
        btnConvert.setOnClickListener {  
            // put some more logic here  
        }  
  
        val btnClear: Button = findViewById(R.id.btnClear)  
        btnClear.setOnClickListener {  
            // put some more logic here  
        }  
    }  
}
```



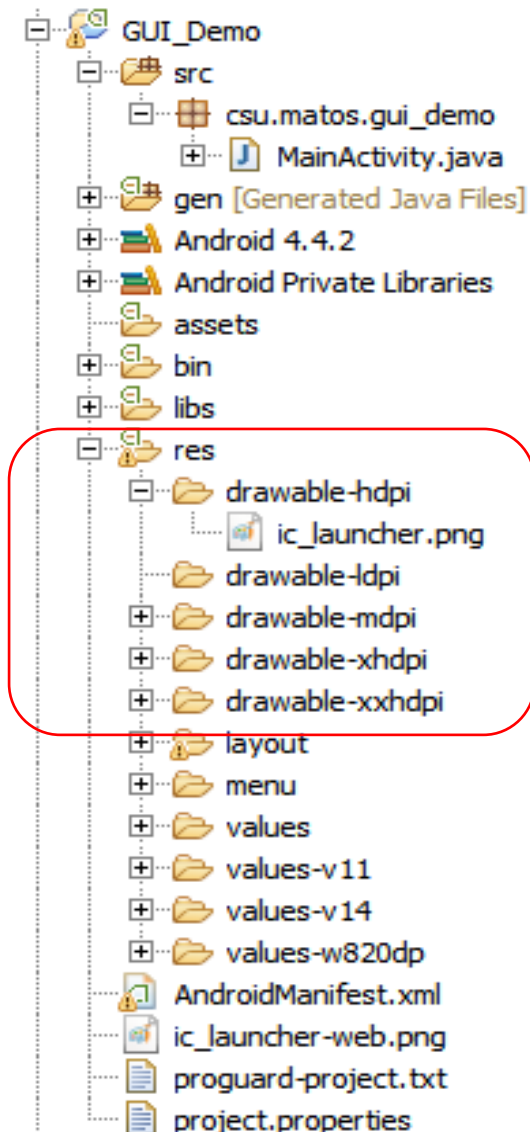
Example: Connecting buttons

```
class MainActivity : AppCompatActivity(), OnClickListener {  
  
    lateinit var edtInput: EditText  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
  
        edtInput = findViewById(R.id.edtInput)  
  
        val btnConvert: Button = findViewById(R.id.btnConvert)  
        btnConvert.setOnClickListener(this)  
  
        val btnClear: Button = findViewById(R.id.btnClear)  
        btnClear.setOnClickListener(this)  
    }  
  
    override fun onClick(p0: View?) {  
        if (p0?.id == R.id.btnConvert) {  
            // put some more logic here  
        } else if (p0?.id == R.id.btnClear) {  
            // put some more logic here  
        }  
    }  
}
```



Basic Widgets: ImageView & ImageButton

- **ImageView** and **ImageButton** allow the embedding of images in your applications (gif, jpg, png, etc).
- Analogue to *TextView* and *Button* controls (respectively).
- Each widget takes an `android:src` or `android:background` attribute (in an XML layout) to specify what picture to use.
- Pictures are stored in the **res/drawable** folder (optionally a *medium*, *high*, *x-high*, *xx-high*, and *xxx-high* respectively definition version of the same image could be stored for later usage with different types of screens). Details available at: <http://developer.android.com/design/style/iconography.html>



Basic Widgets: ImageView & ImageButton

<LinearLayout

```
xmlns:android="http://schemas.android.com/apk/res/android"  
android:layout_width="match_parent"  
android:layout_height="match_parent"  
android:padding="6dp"  
android:orientation="vertical" >
```

<ImageButton

```
android:id="@+id/imgButton1"  
android:layout_width="wrap_content"  
android:layout_height="wrap_content"  
android:src="@drawable/ic_launcher" >
```

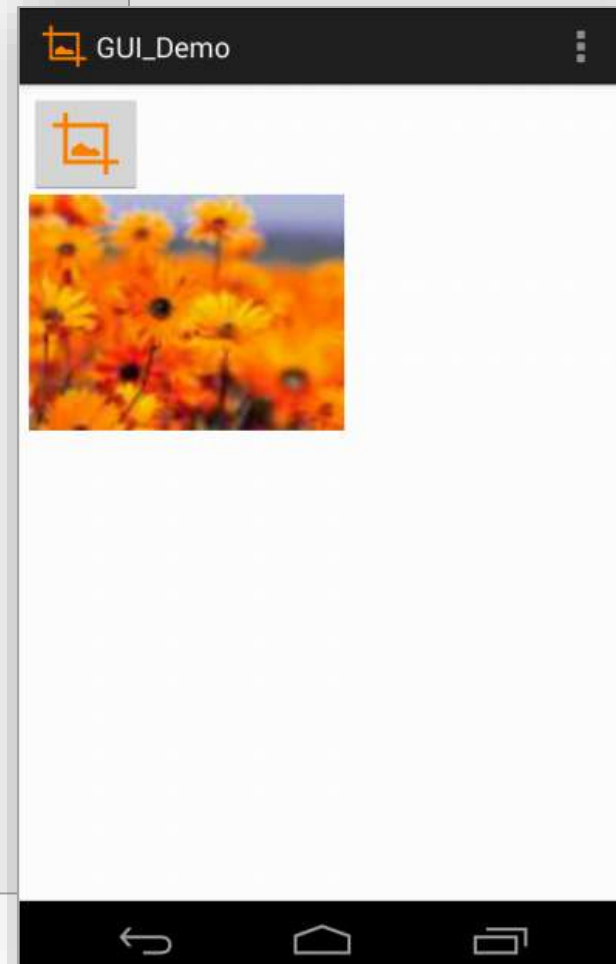
</ImageButton>

<ImageView

```
android:id="@+id/imgView1"  
android:layout_width="200dp"  
android:layout_height="150dp"  
android:scaleType="fitXY"  
android:src="@drawable/flowers1" >
```

</ImageView>

</LinearLayout>



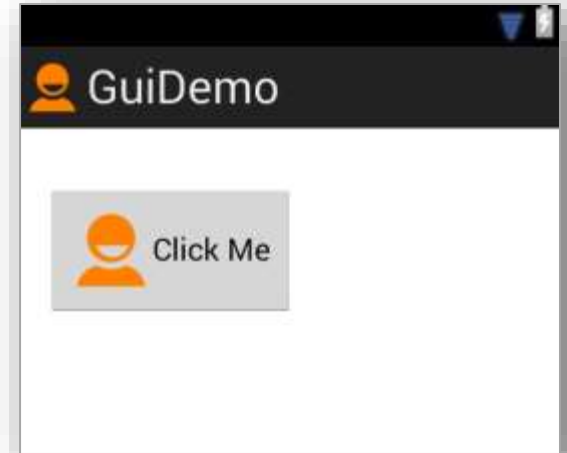
Basic Widgets: Buttons - Combining Images & Text

A common **Button** widget could display text and a simple image as shown below

```
<LinearLayout
    . . .

    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:drawableLeft="@drawable/ic_launcher"
        android:gravity="left/center_vertical"
        android:padding="15dp"
        android:text="Click me" />

</LinearLayout>
```



Basic Widgets: How icons are used in Android?

Icons are small images used to graphically represent your application and/or parts of it. They may appear in different parts of your app including:

- Home screen
- Launcher window.
- Options menu
- Action Bar
- Status bar
- Multi-tab interface.
- Pop-up dialog boxes
- List view

Detailed information on Android's iconography is available at: <http://developer.android.com/design/style/iconography.html>



mdpi (761 bytes)
1x = 48 x 48 pixels
BaseLine



hdpi (1.15KB)
1.5x = 72 x 72 px



x-hdpi (1.52KB)
2x = 96 x 96 px



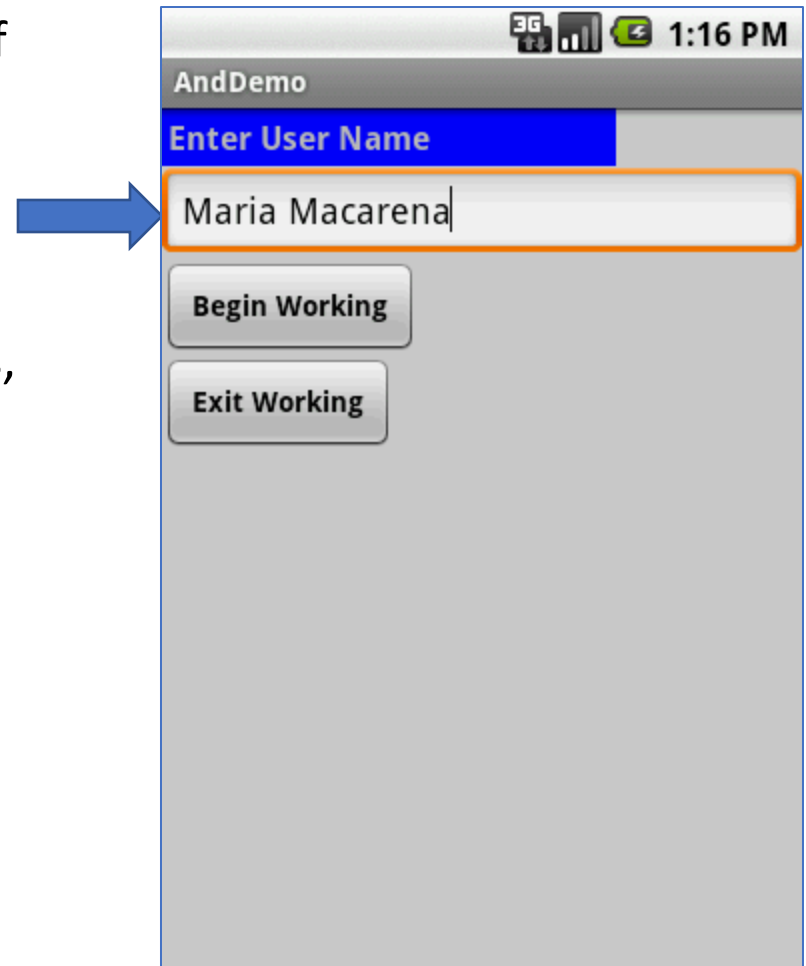
xx-hdpi (2.47KB)
3x = 144 x 144 px

Basic Widgets: EditText Boxes

- The **EditText** widget is an extension of **TextView** that allows user's input.
- In addition to plain text, this widget can display **editable** text formatted with HTML-styles such as bold, italics, underline, etc). This is done with **Html.fromHtml(html_text)**
- Moving data in and out of an EditText box is usually done in Kotlin through the following methods:

```
txtBox.setText("someValue")
```

```
txtBox.text.toString()
```



Basic Widgets: EditText Boxes

Input Type Formats

An EditText box could be set to accept input strings satisfying a particular pattern such as: numbers (with and without decimals or sign), phones, dates, times, uris, etc.

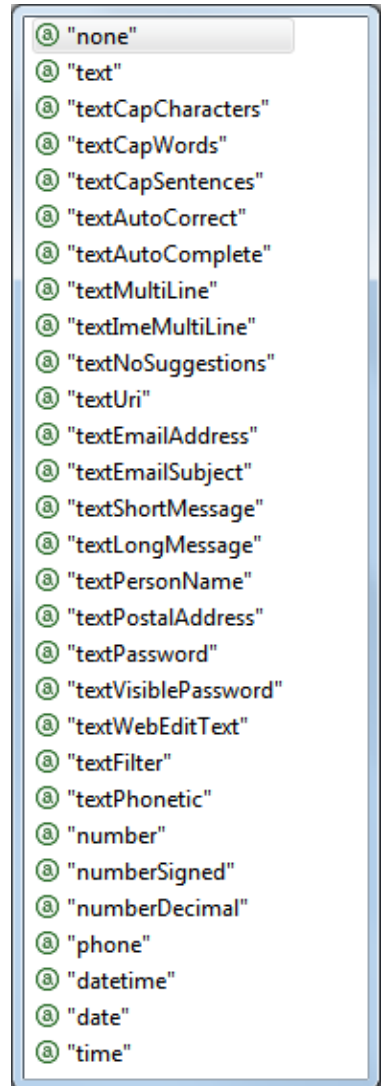
Setting the EditText box to accept a particular choice of data-type, is done through the XML clause

android:inputType="choices"

where **choices** include any of the single values shown in the figure. You may combine types, for instance:

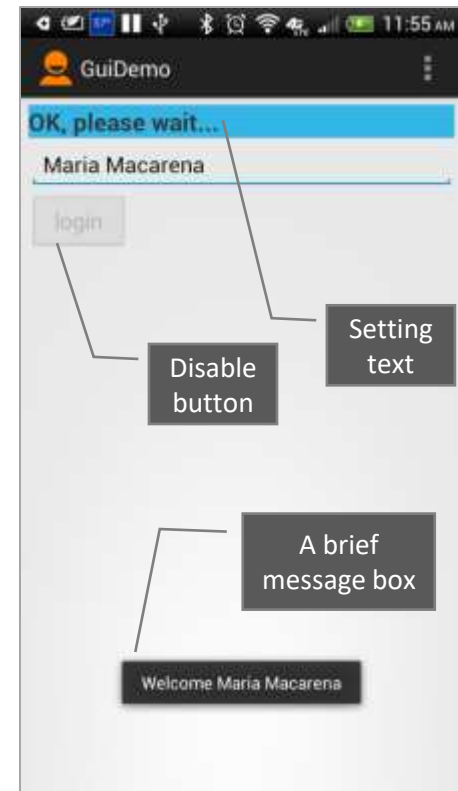
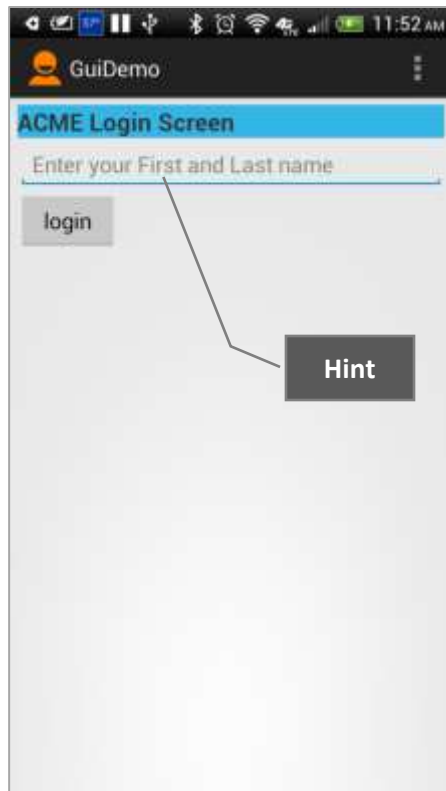
textCapWords | textAutoCorrect

Accepts text that capitalizes every word, incorrect words are automatically changed (for instance 'teh' is converted into 'the', and so on.



Example: Login-Screen

In this example we will create a simple login screen holding a label (**TextView**), a textBox (**EditText**), and a **Button**. When the EditText box gains focus, the system provides a **virtual keyboard** customized to the input-type given to the entry box (capitals & spelling). Clicking the button displays a Toast-message that echoes the supplied user-name.



Example: Login-Screen

LAYOUT 1 of 2

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="6dp" >

    <TextView
        android:id="@+id/txtLogin"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:background="@android:color/holo_blue_light"
        android:text="@string/ACME_Login_Screen"
        android:textSize="20sp"
        android:textStyle="bold" />

    <EditText
        android:id="@+id/edtUserName"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="2dp"
        android:hint="@string/Enter_your_First_and_Last_name"
        android:inputType="textCapWords|textAutoCorrect"
        android:textSize="18sp" >
        <requestFocus />
    </EditText>
```

Example: Login-Screen

LAYOUT 2 of 2

```
<Button
    android:id="@+id/btnLogin"
    android:layout_width="82dp"
    android:layout_height="wrap_content"
    android:layout_marginTop="2dp"
    android:text="@string/Login" />
</LinearLayout>
```

res/values/strings.xml

```
<?xml version="1.0" encoding="utf-8"?>
<!-- this is the res/values/strings.xml file -->
<resources>

    <string name="app_name">GuiDemo</string>
    <string name="action_settings">Settings</string>
    <string name="Login">Login</string>
    <string name="ACME_Login_Screen">ACME Login Screen</string>
    <string name="Enter_your_First_and_Last_name">Enter your First and Last name</string>

</resources>
```

Example: Login-Screen - MainActivity code

```
class MainActivity : AppCompatActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
  
        val context = applicationContext  
        val duration = Toast.LENGTH_SHORT  
  
        val textLogin: TextView = findViewById(R.id.text_login)  
        val editUsername: EditText = findViewById(R.id.edit_username)  
  
        val buttonLogin: Button = findViewById(R.id.button_login)  
        buttonLogin.setOnClickListener {  
            val username = editUsername.text.toString()
```

```
            Log.v("TAG", "duration: $duration")  
            Log.v("TAG", "context: $context")  
            Log.v("TAG", "username: $username")
```

← **Log.v** used for debugging, show in Logcat window

```
            if (username.equals("Maria Macarena")) {  
                textLogin.text = "OK, please wait"  
                Toast.makeText(this, "Welcome $username", duration).show()  
                buttonLogin.isEnabled = false  
            } else {  
                Toast.makeText(this, "$username is not a valid user", duration).show()  
            }  
        }  
    }  
}
```

Basic Widgets: CheckBoxes

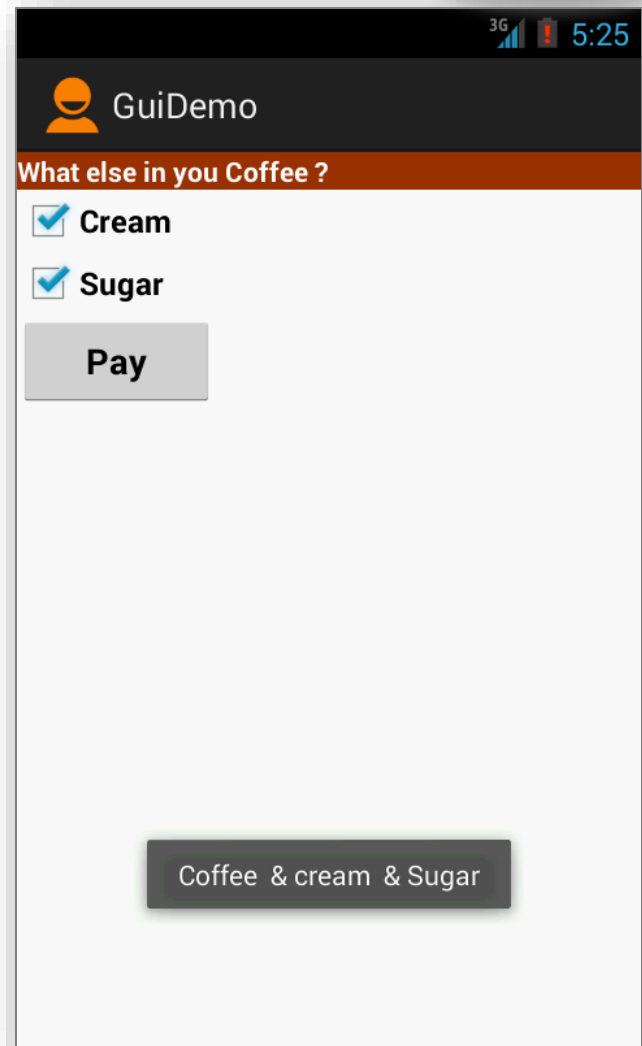


A checkbox is a special **two-states** button which can be either *checked* or *unchecked*.

A screen may include any number of **mutually inclusive** (independent) CheckBoxes. At any time, more than one CheckBox in the GUI could be checked.

In our “CaféApp” example, the screen on the right displays two CheckBox controls, they are used for selecting ‘Cream’ and ‘Sugar’ options. In this image both boxes are ‘checked’.

When the user pushes the ‘Pay’ button a Toast-message is issue echoing the current combination of choices held by the checkboxes.



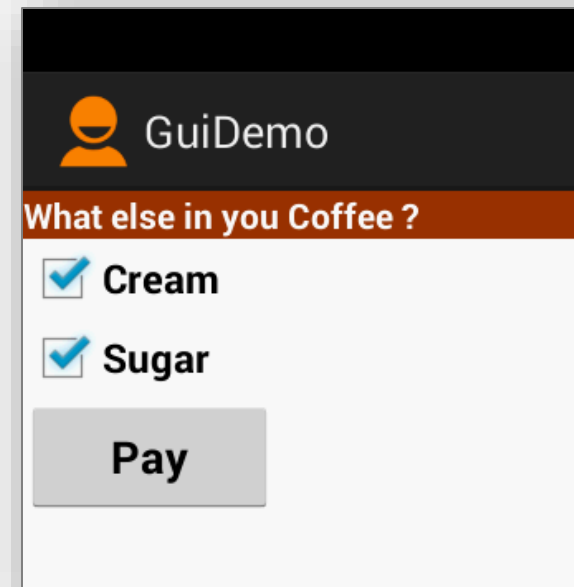
Example: CheckBoxes – CaféApp [Layout 1 of 2]



```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="6dp"
    android:orientation="vertical" >

    <TextView
        android:id="@+id/LabelCoffee"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:background="#ff993300"
        android:text="@string/coffee_addons"
        android:textColor="@android:color/white"
        android:textStyle="bold" />

    <CheckBox
        android:id="@+id/chkCream"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/cream"
        android:textStyle="bold" />
```



Example: CheckBoxes – CaféApp [Layout 2 of 2]



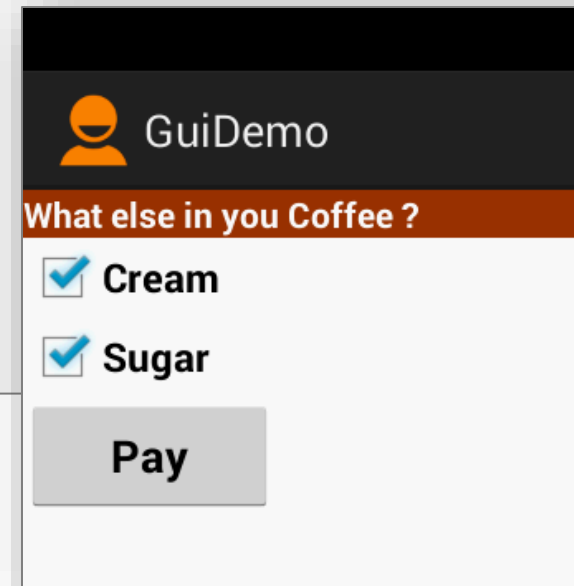
<CheckBox

```
android:id="@+id/chkSugar"  
android:layout_width="wrap_content"  
android:layout_height="wrap_content"  
android:text="@string/sugar"  
android:textStyle="bold" />
```

<Button

```
android:id="@+id/btnPay"  
android:layout_width="153dp"  
android:layout_height="wrap_content"  
android:text="@string/pay"  
android:textStyle="bold" />
```

</LinearLayout>



Example: CheckBoxes – CaféApp [@string/...]

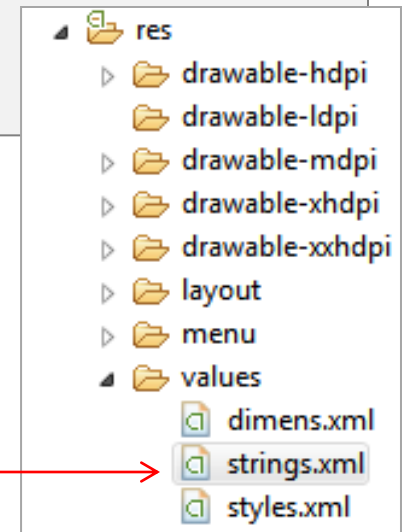


Resources: res/values/strings

```
<?xml version="1.0" encoding="utf-8"?>
<resources>

    <string name="app_name">GuiDemo</string>
    <string name="action_settings">Settings</string>

    <string name="click_me">Click Me</string>
    <string name="sugar">Sugar</string>
    <string name="cream">Cream</string>
    <string name="coffee_addons">What else in your coffee?</string>
    <string name="pay">Pay</string>
</resources>
```



Example: CheckBoxes – CaféApp [Code]



```
class MainActivity : AppCompatActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
  
        val chkCream: CheckBox = findViewById(R.id.chkCream)  
        val chkSugar: CheckBox = findViewById(R.id.chkSugar)  
  
        val buttonLogin: Button = findViewById(R.id.btnPay)  
        buttonLogin.setOnClickListener {  
            var msg = "Coffee "  
            if (chkCream.isChecked)  
                msg += "& cream "  
            if (chkSugar.isChecked)  
                msg += "& sugar "  
            Toast.makeText(applicationContext, msg, Toast.LENGTH_SHORT).show()  
        }  
    }  
}
```

Basic Widgets: Radio Buttons



- A **radio button** (like a CheckBox) is a two-states button that can be either *checked* or *unchecked*.
- Logically related radio buttons are normally put together in a **RadioGroup** container. The container forces the enclosed radio buttons to behave as **mutually exclusive selectors**. That is, the checking of one radio button *unchecks* all the others.
- Properties for *font face, style, color*, etc. are managed in a way similar to setting a TextView.
- You may call the method *isChecked()* to see if a specific RadioButton is selected, or change its state by calling *toggle()*.

Example: CheckBoxes – CaféApp [Layout]



Example

*We extend the previous CaféApp example by adding a **RadioGroup** control that allows the user to pick one type of coffee from three available options.*

RadioGroup

Summary of choices

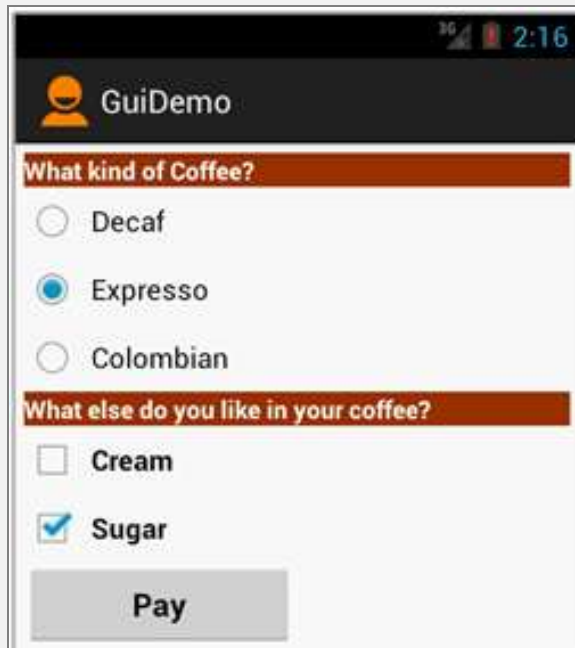
The screenshot shows a mobile application interface for a coffee shop. At the top, there's a status bar with '3G', a battery icon, and the time '2:32'. Below that is a header bar with an orange smiley face icon and the text 'GuiDemo'. The main content area has two sections. The first section is titled 'What kind of Coffee?' in a brown header bar. It contains three radio button options: 'Decaf', 'Espresso' (which is selected with a blue dot), and 'Colombian'. The second section is titled 'What else do you like in your coffee?' in a brown header bar. It contains two checkbox options: 'Cream' (unchecked) and 'Sugar' (checked with a blue checkmark). Below these sections is a grey button labeled 'Pay'. At the bottom of the screen, there is a dark grey button labeled 'Espresso Coffee & Sugar'.

Example: CheckBoxes – CaféApp [Layout]



Only new XML and Kotlin code is shown

```
<TextView
    android:id="@+id/textView1"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:background="#ff993300"
    android:text="@string/kind_of_coffee"
    android:textColor="#ffffff"
    android:textStyle="bold" />
```



```
<RadioGroup
    android:id="@+id/radioGroupCoffeeType"
    android:layout_width="match_parent"
    android:layout_height="wrap_content" >

    <RadioButton
        android:id="@+id/radDecaf"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/decaf" />

    <RadioButton
        android:id="@+id/radEspresso"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/espresso" />

    <RadioButton
        android:id="@+id/radColombian"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:checked="true"
        android:text="@string/colombian" />
</RadioGroup>
```

Example: CheckBoxes – CaféApp [MainActivity]



```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val chkCream: CheckBox = findViewById(R.id.chkCream)
        val chkSugar: CheckBox = findViewById(R.id.chkSugar)

        val radCoffeeType: RadioGroup = findViewById(R.id.radioGroupCoffeeType)
        val radDecaf: RadioButton = findViewById(R.id.radDecaf)
        val radEspresso: RadioButton = findViewById(R.id.radEspresso)
        val radColombian: RadioButton = findViewById(R.id.radColombian)

        val buttonLogin: Button = findViewById(R.id.btnPay)
        buttonLogin.setOnClickListener {
            var msg = "Coffee "
            if (chkCream.isChecked)
                msg = "$msg & cream "
            if (chkSugar.isChecked)
                msg = "$msg & sugar "

            val radioId = radCoffeeType.checkedRadioButtonId

            if (radColombian.id == radioId) msg = "Colombian $msg"
            if (radEspresso.isChecked) msg = "Espresso $msg"
            if (radDecaf.isChecked) msg = "Decaf $msg"
        }
    }
}
```

Example: CheckBoxes – CaféApp [MainActivity]



```
// Alternative
when (radioId) {
    R.id.radColombian -> msg = "Colombian $msg"
    R.id.radEspresso -> msg = "Espresso $msg"
    R.id.radDecaf -> msg = "Decaf $msg"
}

Toast.makeText(applicationContext, msg, Toast.LENGTH_SHORT).show()
}
}
```

Alternative you may also manage a **RadioGroup** as follows (this is simpler because you don't need to define the individual RadioButtons)

Appendix. Using the @string resource



A good programming practice in Android is **NOT** to directly enter literal strings as immediate values for attribute inside xml files.

For example, if you are defining a **TextView** to show a company headquarter's location, a clause such as `android:text="Cleveland"` should not be used (observe it produces a **Warning** `[I18N] Hardcoded string "Cleveland", should use @string resource`)

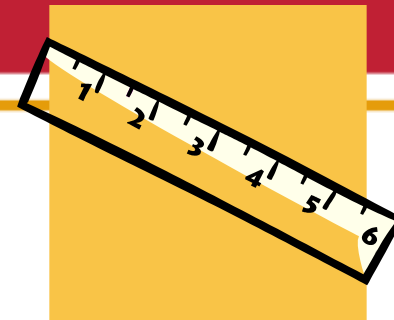
Instead you should apply a two steps procedure in which

1. You write the literal string –say *headquarter* – in **res/values/string.xml**. Enter `<string name="headquarter">Cleveland</string>`
2. Whenever the string is needed provide a reference to the string using the notation *@string/headquarter*. For instance in our example you should enter `android:text="@string/headquarter"`

WHY?

If the string is used in many places and its actual value changes we just update the resource file entry once. It also provides some support for internationalization -easy to change a resource string from one language to another.

Appendix. Measuring Graphic Elements



Q. What is **dpi** (also know as **dp** and **ppi**) ?

Stands for *dots per inch*. It suggests a measure of screen quality. You can compute it using the following formula:

$$dpi = \sqrt{widthPixels^2 + heightPixels^2} / diagonalInches$$

G1 (base device 320x480)	155.92 dpi	(3.7 in diagonally)
Nexus (480x800)	252.15 dpi	
HTC One (1080x1920)	468 dpi	(4.7 in)
Samsung S4 (1080x1920)	441 dpi	(5.5 in)

Q. What is the difference between **dp**, **dip** and **sp** units in Android?

dp *Density-independent Pixels* – is an abstract unit based on the physical density of the screen. These units are relative to a 160 dpi screen, so one dp is one pixel on a 160 dpi screen. Use it for measuring anything but fonts.

sp

Scale-independent Pixels – similar to the relative density dp unit, but used for **font** size preference.

Appendix. Measuring Graphic Elements

How Android deals with screen resolutions?

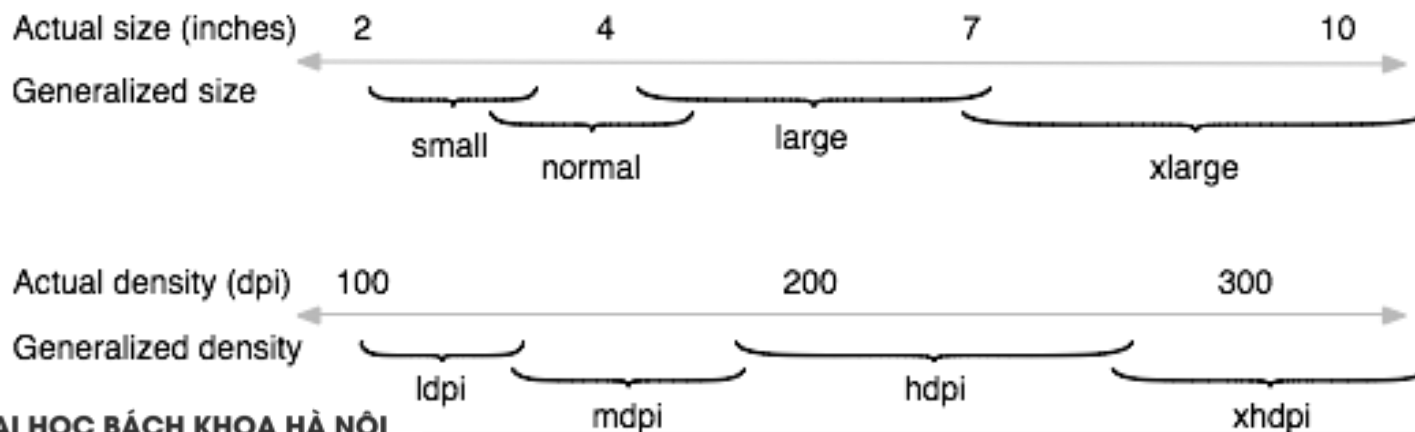
Illustration of how the Android platform maps actual screen densities and sizes to generalized density and size configurations.

A set of four generalized **screen sizes**

<i>xlarge</i>	screens are at least 960dp x 720dp
<i>large</i>	screens are at least 640dp x 480dp
<i>normal</i>	screens are at least 470dp x 320dp
<i>small</i>	screens are at least 426dp x 320dp

A set of six generalized **densities**:

<i>ldpi</i>	~120dpi (low)
<i>mdpi</i>	~160dpi (medium)
<i>hdpi</i>	~240dpi (high)
<i>xhdpi</i>	~320dpi (extra-high)
<i>xxhdpi</i>	~480dpi (extra-extra-high)
<i>xxxhdpi</i>	~640dpi (extra-extra-extra-high)



Appendix. Measuring Graphic Elements

Q. Give me an example on how to use dp units.

Assume you design your interface for a G1 phone having 320x480 pixels (Abstracted density is **160** – See your AVD entry, the actual pixeling is defined as: $[2 \times 160] \times [3 \times 160]$)

Assume you want a 120dp button to be placed in the middle of the screen.

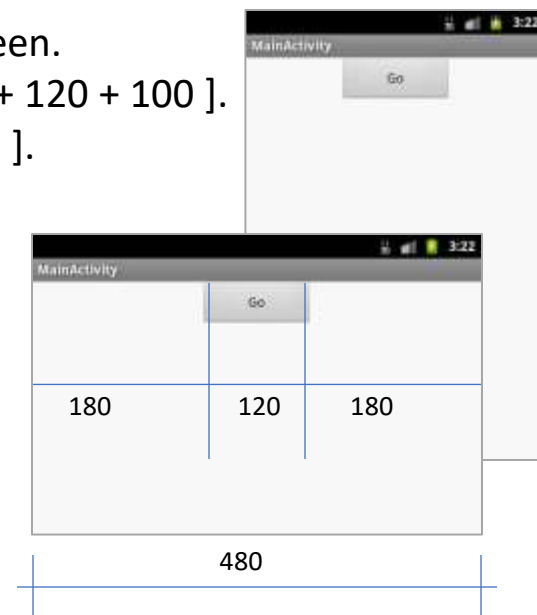
On portrait mode you could allocate the 320 horizontal pixels as $[100 + 120 + 100]$.

On Landscape mode you could allocate 480 pixels as $[180 + 120 + 180]$.

The XML would be

<Button

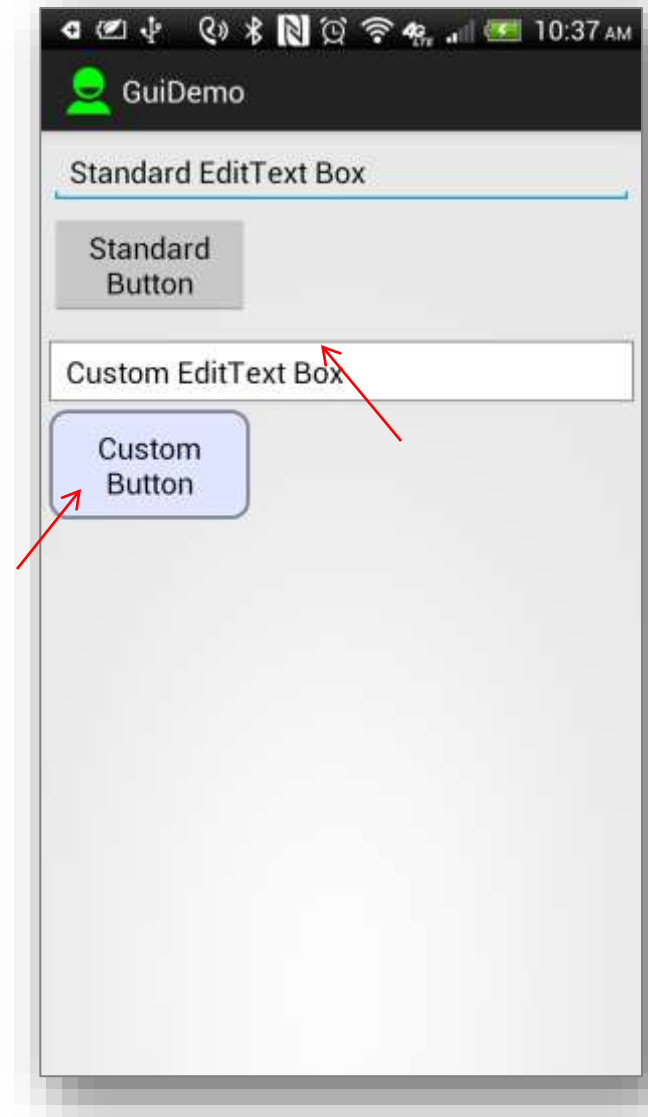
```
android:id="@+id/button1"
android:layout_height="wrap_content"
android:layout_width="120dp"
android:layout_gravity="center"
android:text="@+id/go_caption" />
```



If the application is deployed on devices having a higher resolution the button is still mapped to the middle of the screen.

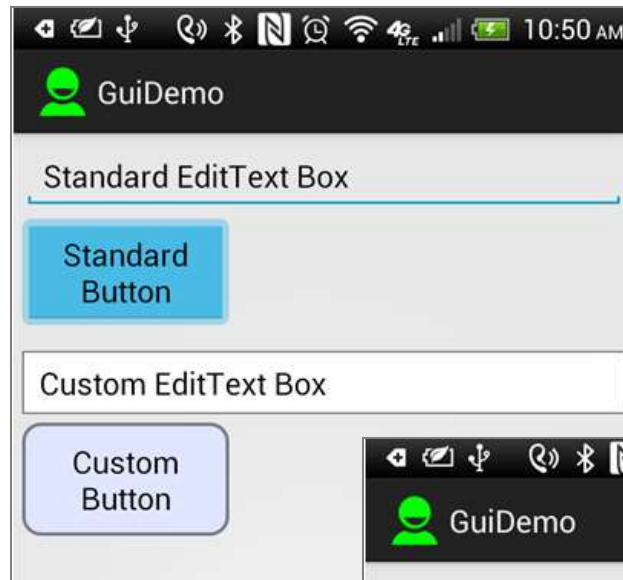
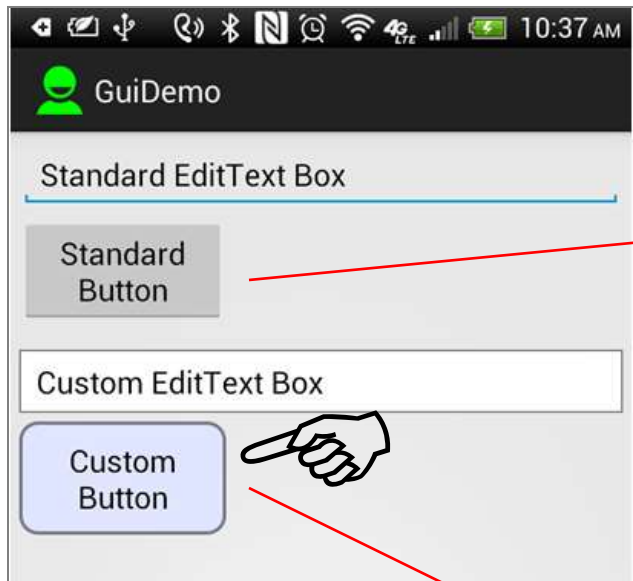
Appendix. Customizing Widgets

1. The appearance of a widget can be adjusted by the user. For example a button widget could be modified by changing its shape, border, color, margins, etc.
2. Basic shapes include: rectangle, oval, line, and ring.
3. In addition to visual changes, the widget's reaction to user interaction could be adjusted for events such as: Focused, Clicked, etc.
4. The figure shows and EditText and Button widgets as *normally* displayed by a device running SDK4.3 (Ice Cream). The bottom two widgets (a TextView and a Button) are custom made versions of those two controls respectively.



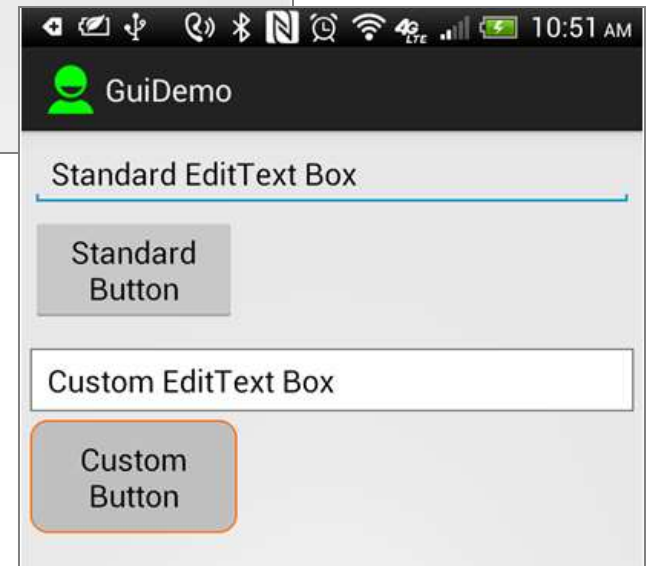
Appendix. Customizing Widgets

The image shows visual feedback provided to the user during the clicking of a standard and a *custom* Button widget.



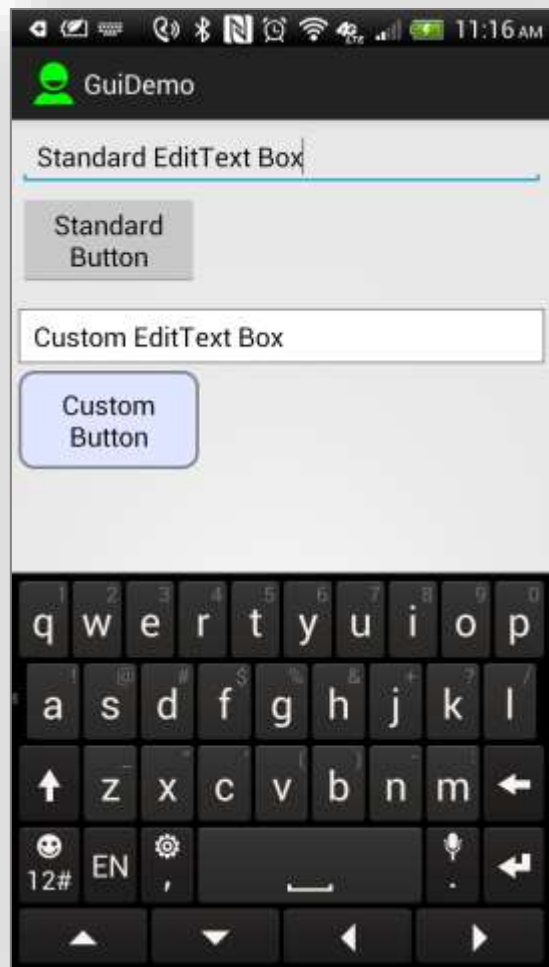
Standard behavior – buttons turns **blue** when it is pressed.

Custom behavior – buttons turns dark grey with an orange border when it is pressed.



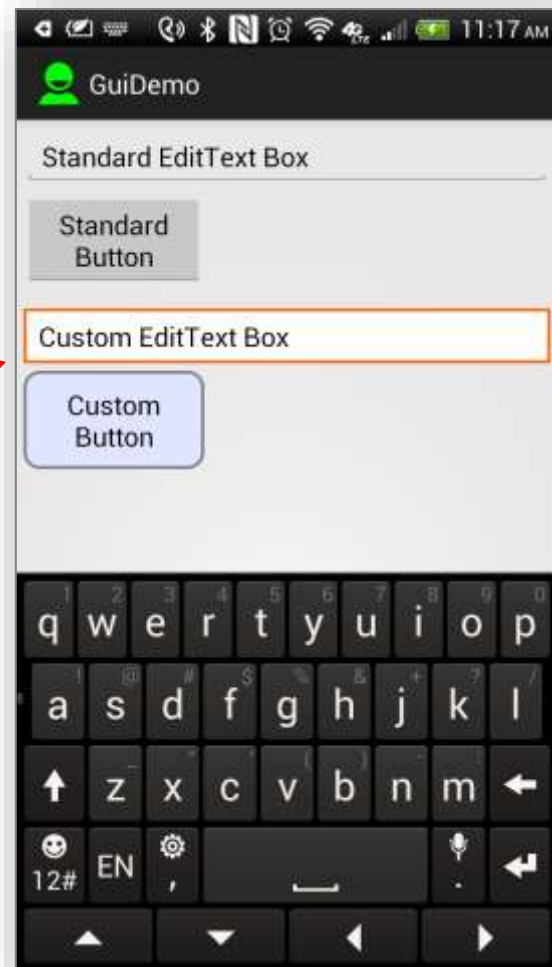
Appendix. Customizing Widgets

Observe the transient response of the standard and custom made EditText boxes when the user touches the widgets provoking the 'Focused' event.



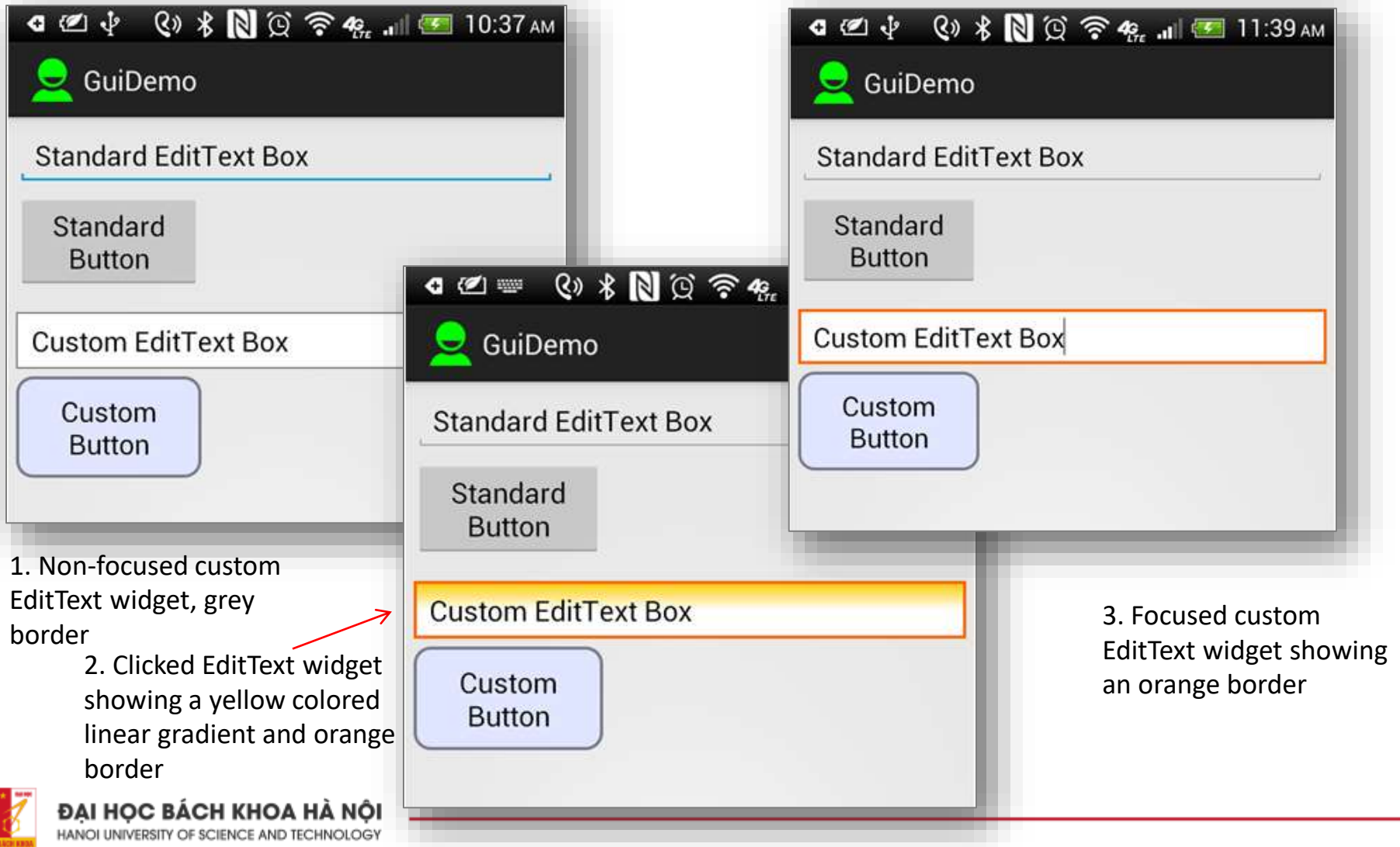
When focused
the standard box
shows a blue
bottom line

A focused
custom box
shows an orange
all-around frame



Appendix. Customizing Widgets

When the user taps on the custom made EditText box a gradient is applied to the box to flash a visual feedback reassuring the user of her selection.

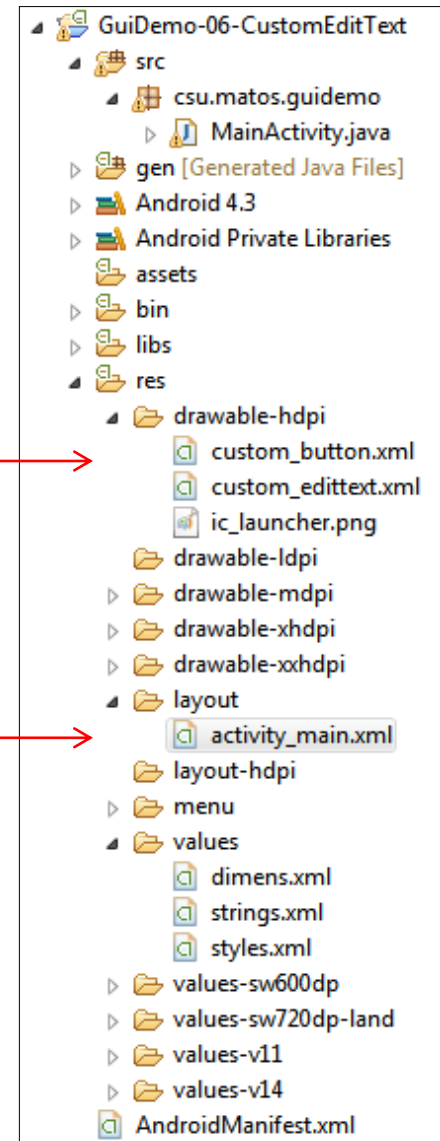


Appendix. Customizing Widgets

Organizing the application

Definition of the custom templates for
Button and EditText widgets

Layout referencing standard and custom
made widgets



Appendix. Customizing Widgets

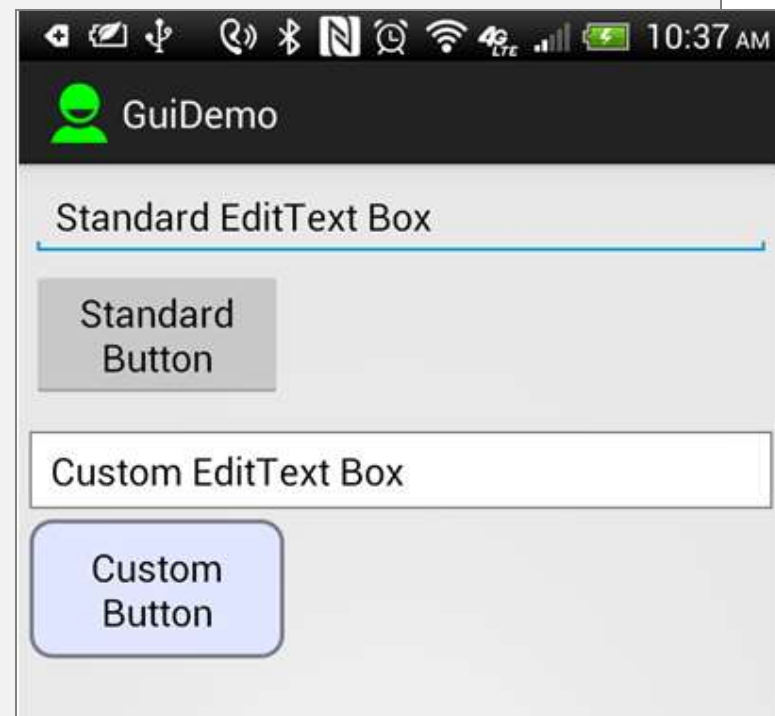
Activity Layout 1 of 2

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="5dp" >

    <EditText
        android:id="@+id/editText1"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginBottom="5dp"
        android:ems="10"
        android:inputType="text"
        android:text="@string/standard_edittext" >

        <requestFocus />
    </EditText>

    <Button
        android:id="@+id/button1"
        android:layout_width="120dp"
        android:layout_height="wrap_content"
        android:layout_marginBottom="15dp"
        android:text="@string/standard_button" />
```



Appendix. Customizing Widgets

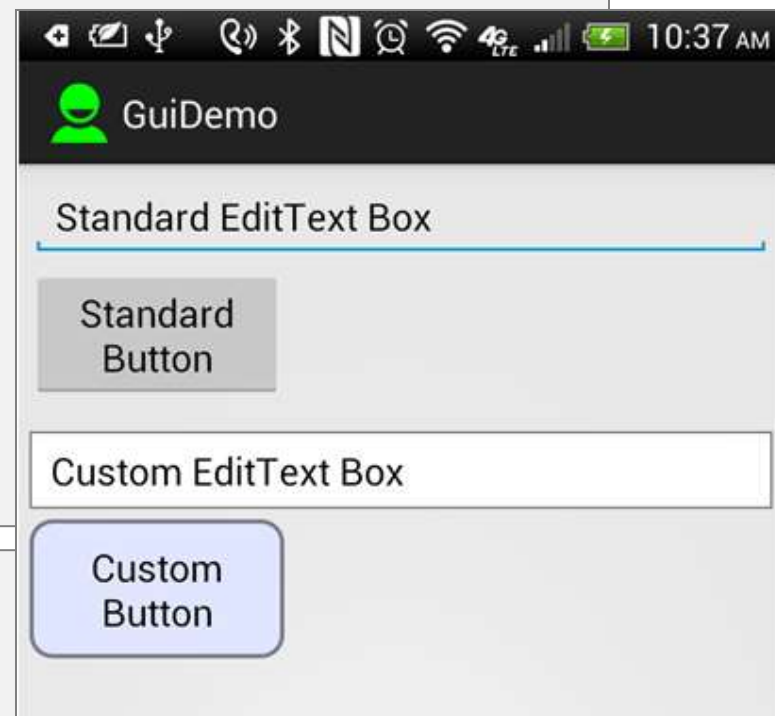
Activity Layout (2 of 2) and Resource: res/values/strings

```
<EditText
    android:id="@+id/editText2"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginBottom="5dp"
    android:background="@drawable/custom_edittext"
    android:ems="10"
    android:inputType="text"
    android:text="@string/custom_edittext" />
```

```
<Button
    android:id="@+id/button2"
    android:layout_width="120dp"
    android:layout_height="wrap_content"
    android:background="@drawable/custom_button"
    android:text="@string/custom_button" />
```

```
</LinearLayout>
```

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string name="app_name">GuiDemo</string>
    <string name="action_settings">Settings</string>
    <string name="standard_button">Standard Button</string>
    <string name="standard_edittext">Standard EditText Box</string>
    <string name="custom_button">Custom Button</string>
    <string name="custom_edittext">Custom EditText Box</string>
</resources>
```

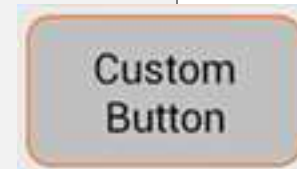


Appendix. Customizing Widgets

Resource: res/drawable/custom_button.xml

The custom Button widget has two faces based on the event **state_pressed** (true, false). The Shape attribute specifies its solid color, padding, border (stroke) and corners (rounded corners have radius > 0)

```
<?xml version="1.0" encoding="utf-8"?>
<selector xmlns:android="http://schemas.android.com/apk/res/android" >
  <item android:state_pressed="true">
    <shape android:shape="rectangle">
      <corners    android:radius="10dp"/>
      <solid      android:color="#ffc0c0c0" />
      <padding    android:left="10dp"
                android:top="10dp"
                android:right="10dp"
                android:bottom="10dp"/>
      <stroke    android:width="1dp" android:color="#ffFF6600"/>
    </shape>
  </item>
  <item android:state_pressed="false">
    <shape android:shape="rectangle">
      <corners    android:radius="10dp"/>
      <solid      android:color="#ffe0e6ff"/>
      <padding    android:left="10dp"
                android:top="10dp"
                android:right="10dp"
                android:bottom="10dp"/>
      <stroke    android:width="2dp" android:color="#ff777b88"/>
    </shape>
  </item>
</selector>
```



Appendix. Customizing Widgets

Resource: res/drawable/custom_edittext.xml

The rendition of the custom made EditText widget is based on three states: normal, state_focused, state_pressed.

```
<?xml version="1.0" encoding="utf-8"?>
<selector xmlns:android="http://schemas.android.com/apk/res/android">

  <item android:state_pressed="true">
    <shape android:shape="rectangle">
      <gradient
        android:angle="90"
        android:centerColor="#FFffffff"
        android:endColor="#FFffcc00"
        android:startColor="#FFffffff"
        android:type="linear" />

      <stroke android:width="2dp"
        android:color="#FFff6600" />
      <corners android:radius="0dp" />
      <padding android:left="10dp"
        android:top="6dp"
        android:right="10dp"
        android:bottom="6dp" />
    </shape>
  </item>
```

Custom EditText Box

Appendix. Customizing Widgets

Resource: res/drawable/custom_edittext.xml

The rendition of the custom made EditText widget is based on three states: normal, state focused, state_pressed.

```
<item android:state_focused="true">
    <shape>
        <solid android:color="#FFFFFF" />
        <stroke android:width="2dp" android:color="#FF6600" />
        <corners android:radius="0dp" />
        <padding android:left="10dp"
            android:top="6dp"
            android:right="10dp"
            android:bottom="6dp" />
    </shape>
</item>

<item>
    <!-- state: "normal" not-pressed & not-focused -->
    <shape>
        <stroke android:width="1dp" android:color="#FF7777" />
        <solid android:color="#FFFFFF" />
        <corners android:radius="0dp" />
        <padding android:left="10dp"
            android:top="6dp"
            android:right="10dp"
            android:bottom="6dp" />
    </shape>
</item>
</selector>
```

A rectangular text box with a white background and a thick orange border. The text "Custom EditText Box" is centered inside the box.A rectangular text box with a white background and a thin gray border. The text "Custom EditText Box" is centered inside the box.

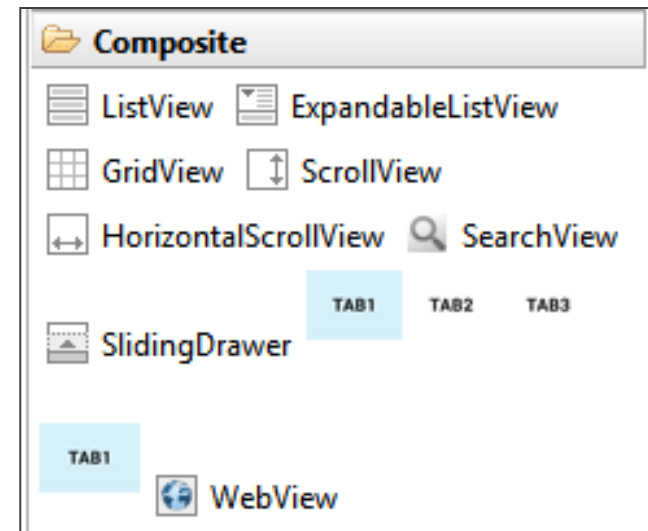
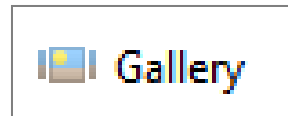


List-Based Widgets: Lists, Grids, and Scroll Views

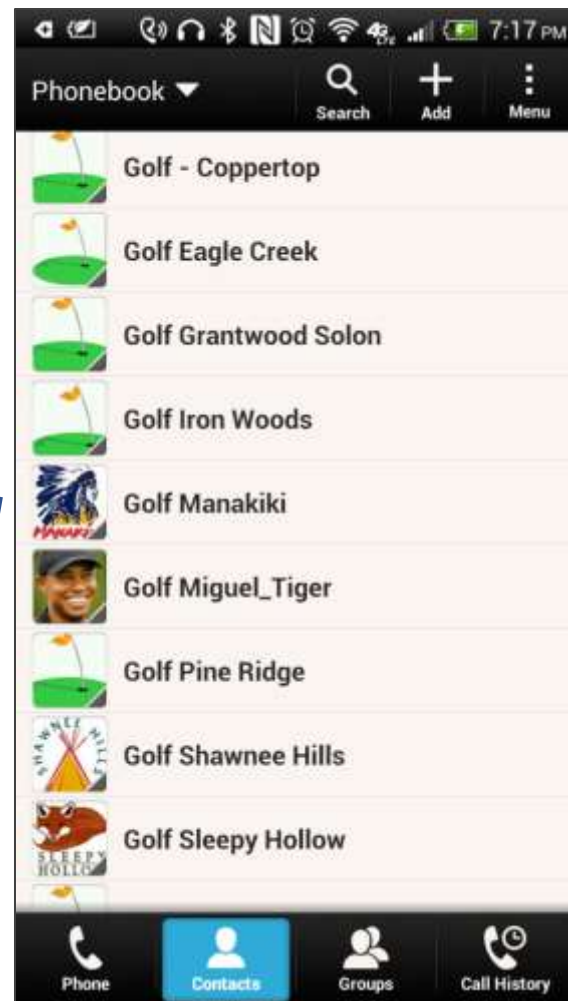
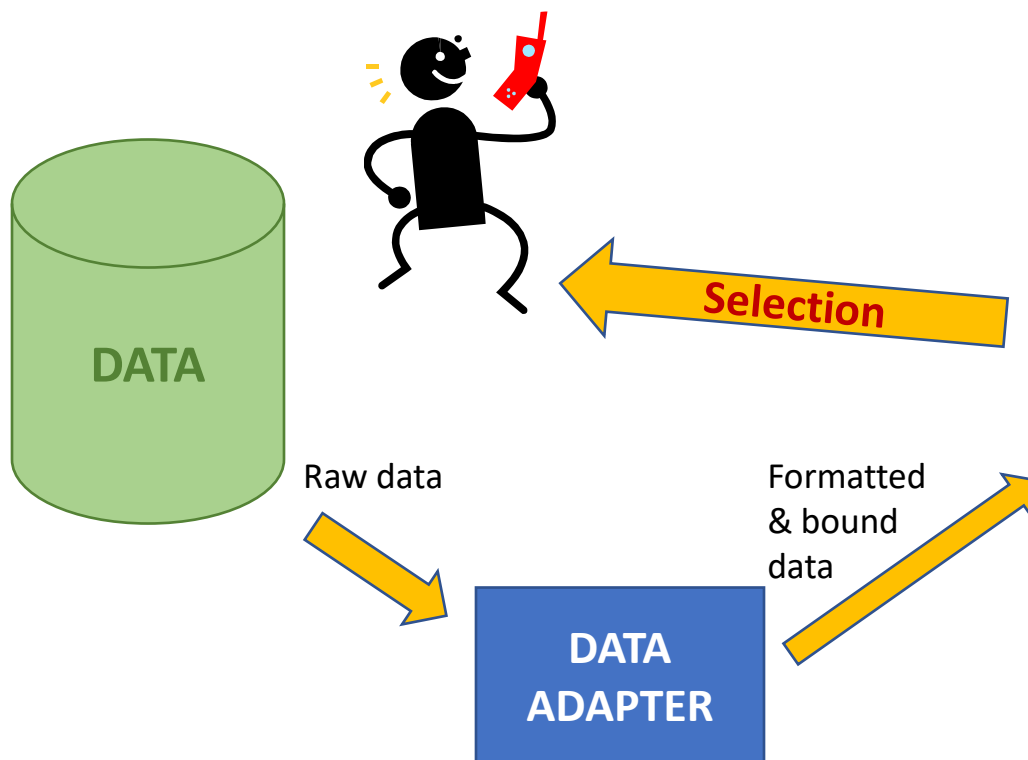
ONE LOVE. ONE FUTURE.

GUI Design for Selection Making

- **RadioButtons** and **CheckButtons** are widgets suitable for selecting options offered by a *small* set of choices. They are intuitive and uncomplicated; however they occupy a permanent space on the GUI (which is not a problem when only a few of them are shown)
- When the set of values to choose from is large, other Android **List-Based Widgets** are more appropriate.
- Example of **List-Based Widgets** include:
 - **ListViews**,
 - **Spinner**,
 - **GridView**
 - **Image Gallery**
 - **ScrollViews**, etc.



Showing a large set of choices on the GUI



Destination layout
Holding a **ListView**

- The Android **DataAdapter** class is used to feed a collection of data items to a *List-Based Widget*.
- The *Adapter* 's raw data may come from a variety of sources, such as small arrays as well as large databases.

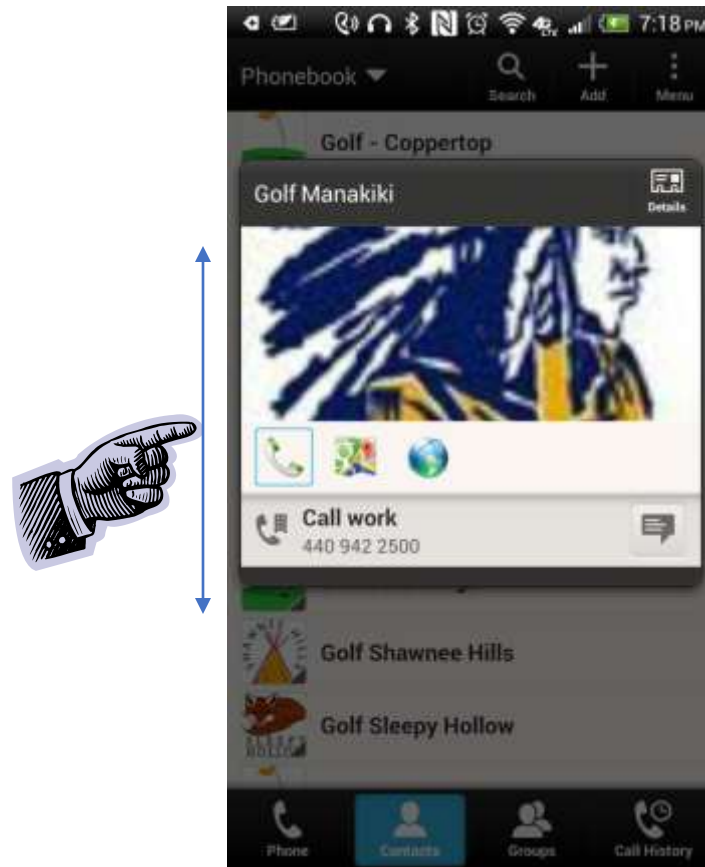
ListViews

The Android **ListView** widget is the most common element used to display data supplied by a **data adapter**.

ListsViews are **scrollable**, each item from the base data set can be shown in an individual row.

Users can **tap** on a row to make a selection.

A row could display one or more lines of text as well as images.

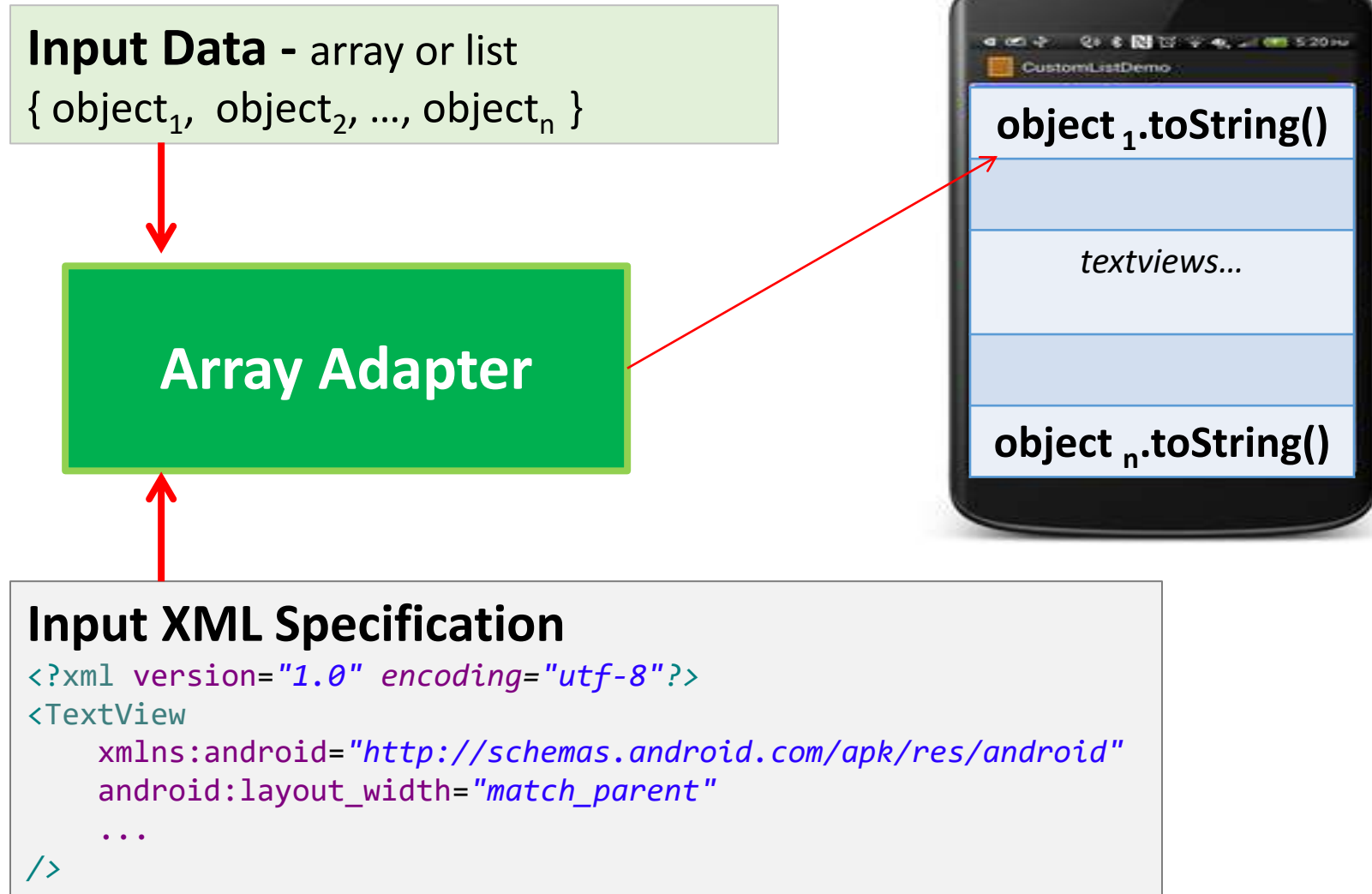


Destination layout
Holding a **ListView**

ArrayAdapter (A Data Beautifier)

- An **ArrayAdapter<T>** accepts for input an **array** (or **ArrayList**) of objects of some arbitrary type T.
- The adapter works on each object by (a) applying its **toString()** method, and (b) moving its formatted output string to a **TextView**.
- The formatting operation is guided by a user supplied XML layout specification which defines the appearance of the receiving TextView.
- For ListViews showing **complex** arrangement of visual elements –such as text plus images- you need to provide a **custom made adapter** in which the **getView(...)** method explains how to manage the placement of each data fragment in the complex layout.

ArrayAdapter (A Data Beautifier)



Using the ArrayAdapter<String> Class

```
val items: Array<String> = arrayOf(
    "Data-0", "Data-1", "Data-2", "Data-3",
    "Data-4", "Data-5", "Data-6", "Data-7")
val adapter: ArrayAdapter<String> = ArrayAdapter(
    this,
    android.R.layout.simple_list_item_1,
    items)
```

Parameters:

1. The current activity's **context** (**this**)
2. The **TextView** layout indicating how an individual row should be written (android.R.id.*simple_list_item_1*).
3. The actual **data source** (array or list containing *items* to be shown).

Example 1: ListView showing a simple list (plain text)

Assume a large collection of input data items is held in a **string** array.
Each row of the ListView must show a line of text taken from the array.
In our example, when the user makes a selection, you must display on a TextView the selected item and its position in the list.



Example 1: Layout

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical" >

    <TextView
        android:id="@+id/txtMsg"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:background="#ffffff00"
        android:text="Using ListViews..."
        android:textSize="16sp" />

    <ListView
        android:id="@+id/my_list"
        android:layout_width="match_parent"
        android:layout_height="match_parent" >
    </ListView>

</LinearLayout>
```

Example 1: MainActivity

```
class MainActivity : AppCompatActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
  
        val txtMsg: TextView = findViewById(R.id.txtMsg)  
  
        val items: Array<String> = arrayOf("Data-0", "Data-1", "Data-2", "Data-3",  
            "Data-4", "Data-5", "Data-6", "Data-7")  
        val adapter: ArrayAdapter<String> = ArrayAdapter(  
            this,  
            android.R.layout.simple_list_item_1,  
            items)  
  
        val listView: ListView = findViewById(R.id.my_list)  
        listView.adapter = adapter  
  
        listView.setOnItemClickListener { adapterView, view, i, l -> txtMsg.text =  
            "Position: $i\nData: ${items[i]}"  
        }  
    }  
}
```

When dataset is changed

Using **notifyDataSetChanged()** method of adapter to refresh the list, when:

- Data is updated
- Items are removed
- Items are added

Example1: Custom ListView

You may want to modify the ListView control to use your **own** GUI design. For instance, you may replace `android.R.layout.simple_list_item_1` with `R.layout.my_custom_text`.

Where `my_custom_text` is the Layout specification listed below (held in the `res/layout` folder). It defines how each row is to be shown.

```
<?xml version="1.0" encoding="utf-8"?>
<TextView
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_margin="2dp"
    android:paddingTop="5dp"
    android:padding="5dp"
    android:textColor="#ffff0000"
    android:background="#22ff0000"
    android:textSize="35sp" />
```



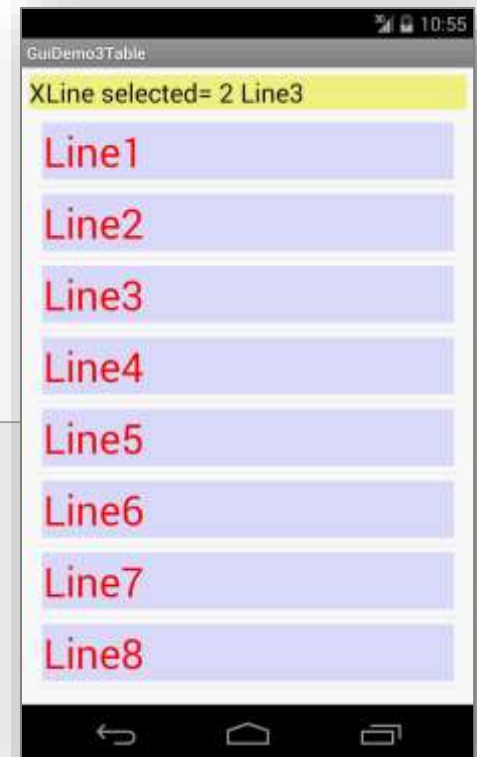
Example1: Custom ListView

You may also create the ArrayAdapter with more parameters. For instance, the following statement:

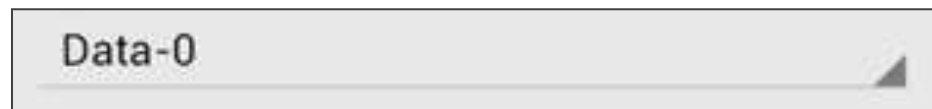
```
val adapter: ArrayAdapter<String> = ArrayAdapter(  
    this,  
    R.layout.item_view,  
    R.id.text_item,  
    items)
```

Defines a custom *list* and *textview* layout to show the contents of the *data* array.

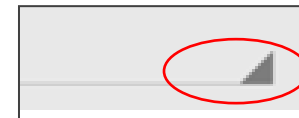
```
<!-- item_view -->  
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"  
    android:layout_width="match_parent"  
    android:layout_height="match_parent"  
    android:orientation="vertical"  
    android:padding="6dp" >  
    <TextView  
        android:id="@+id/text_item"  
        android:layout_width="match_parent"  
        android:layout_height="wrap_content"  
        android:background="#220000ff"  
        android:padding="1dp"  
        android:textColor="#ffffff0000"  
        android:textSize="35sp" />  
</LinearLayout>
```



The Spinner Widget

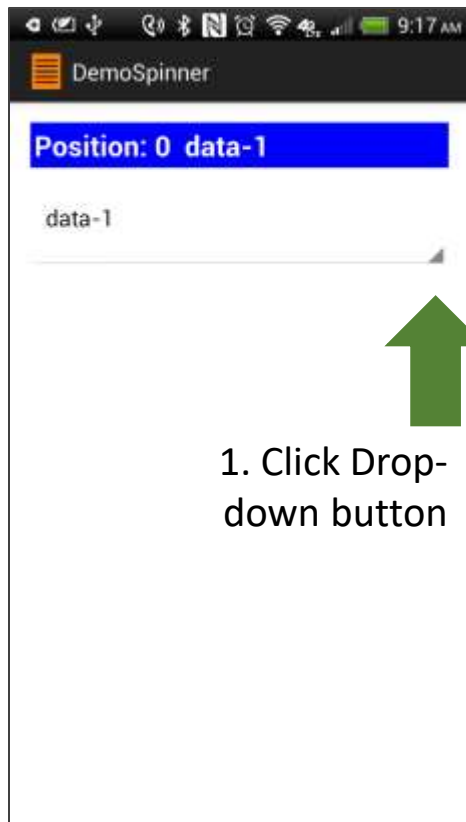


- Android's **Spinner** is equivalent to a *drop-down* selector.
- Spinners have the same functionality of a ListView but take less screen space.
- An Adapter is used to supply its data using *adapter*
- A listener captures selections made from the list with *setOnItemSelectedListener*.



Example2: Using the Spinner Widget

A list of options named 'Data-0', 'Data-1', 'Data-2' and so on, should be displayed when the user taps on the 'down-arrow' portion of the spinner.



1. Click Drop-down button



2. Select this option



3. Selected value

Example 2: Layout

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="3dp"
    tools:context=".MainActivity" >

    <TextView
        android:id="@+id/txtMsg"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:background="#ffffff00"
        android:text="Spinner selection" />

    <Spinner
        android:id="@+id/spinner1"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />

</LinearLayout>
```



Example 2: MainActivity

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val txtMsg: TextView = findViewById(R.id.txtMsg)

        val items: Array<String> = arrayOf("Data-0", "Data-1", "Data-2", "Data-3",
            "Data-4", "Data-5", "Data-6", "Data-7")
        val arrayAdapter: ArrayAdapter<String> = ArrayAdapter(
            this,
            android.R.layout.simple_dropdown_item_1line,
            items)

        findViewById<Spinner>(R.id.spinner1).run {
            adapter = arrayAdapter
            onItemSelectedListener = object : OnItemSelectedListener {
                override fun onItemSelected(p0: AdapterView<*>?, p1: View?, p2: Int, p3: Long) {
                    txtMsg.text = items[p2]
                }
                override fun onNothingSelected(p0: AdapterView<*>?) {
                    TODO("Not yet implemented")
                }
            }
        }
    }
}
```

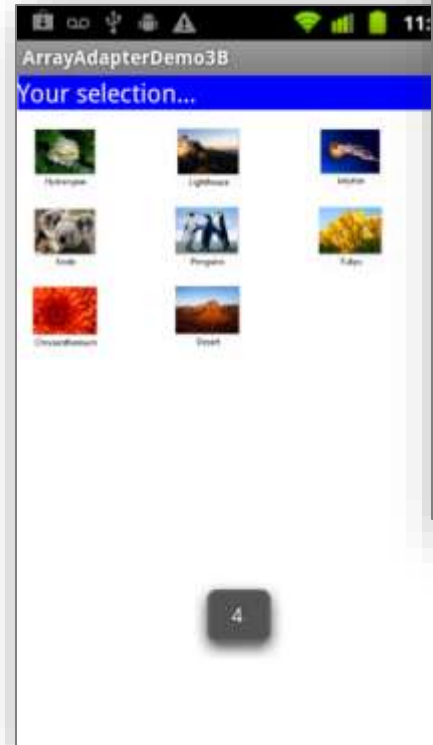
The GridView widget

GridView

GridView is a ViewGroup that displays items in a two-dimensional, scrollable grid.

Data items shown by the grid are supplied by a data adapter.

Grid cells can show text and/or images



GridView: Useful Properties

Some properties used to determine the number of columns and their sizes:

- **android:numColumns**
indicates how many columns to show. When used with option “auto_fit”, Android determines the number of columns based on available space and the properties listed below.
- **android:verticalSpacing** and **android:horizontalSpacing**
indicate how much free space should be set between items in the grid.
- **android:columnWidth**
column width in **dips**.
- **android:stretchMode**
indicates how to modify image size when there is available space not taken up by columns or spacing .

GridView: Fitting the View to the Screen

Suppose the screen is **320** (dip) pixels wide, and we have

android:columnWidth set to **100dip** and
android:horizontalSpacing set to **5dip**.

The user would see three columns taking **310** pixels (three columns of 100 pixels and two separators of 5 pixels).

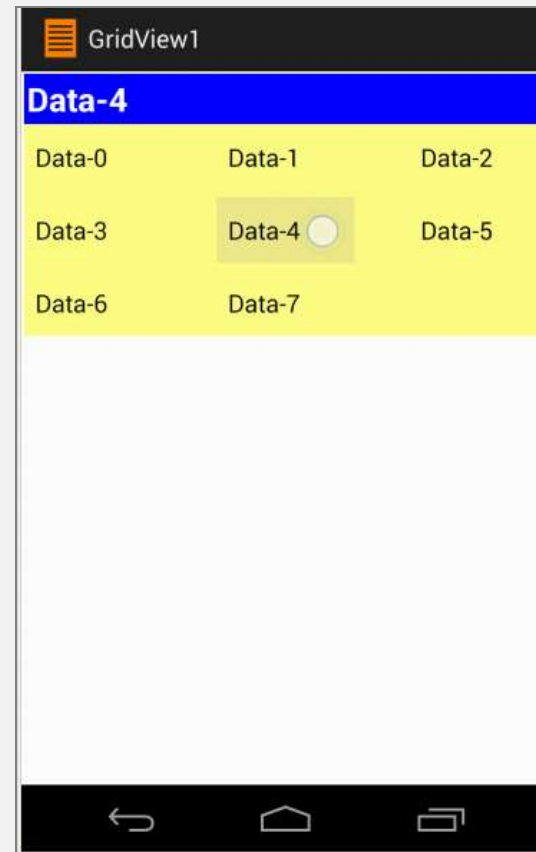
With **android:stretchMode** set to *columnWidth*, the three columns will each expand by 3-4 pixels to use up the remaining 10 pixels.

With **android:stretchMode** set to *spacingWidth*, the two internal whitespaces will each grow by 5 pixels to consume the remaining 10 pixels.

Example 3: GridView Demo - Layout

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/LinearLayout1"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="2dp"
    tools:context=".MainActivity" >
    <TextView
        android:id="@+id/txtMsg"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:background="#ff0000ff"
        android:textSize="24sp"
        android:textStyle="bold"
        android:textColor="#ffffffff"
        android:padding="2dp" />

    <GridView
        android:id="@+id/grid"
        android:background="#77ffff00"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:verticalSpacing="5dip"
        android:horizontalSpacing="5dip"
        android:numColumns="auto_fit"
        android:columnWidth="100dip"
        android:stretchMode="spacingWidth" />
</LinearLayout>
```



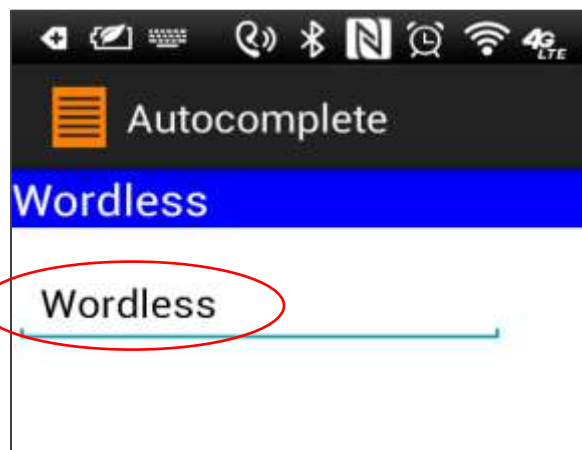
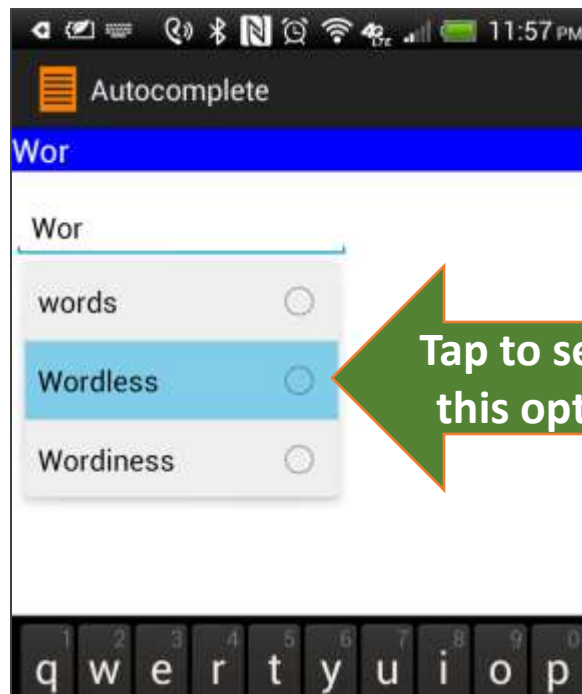
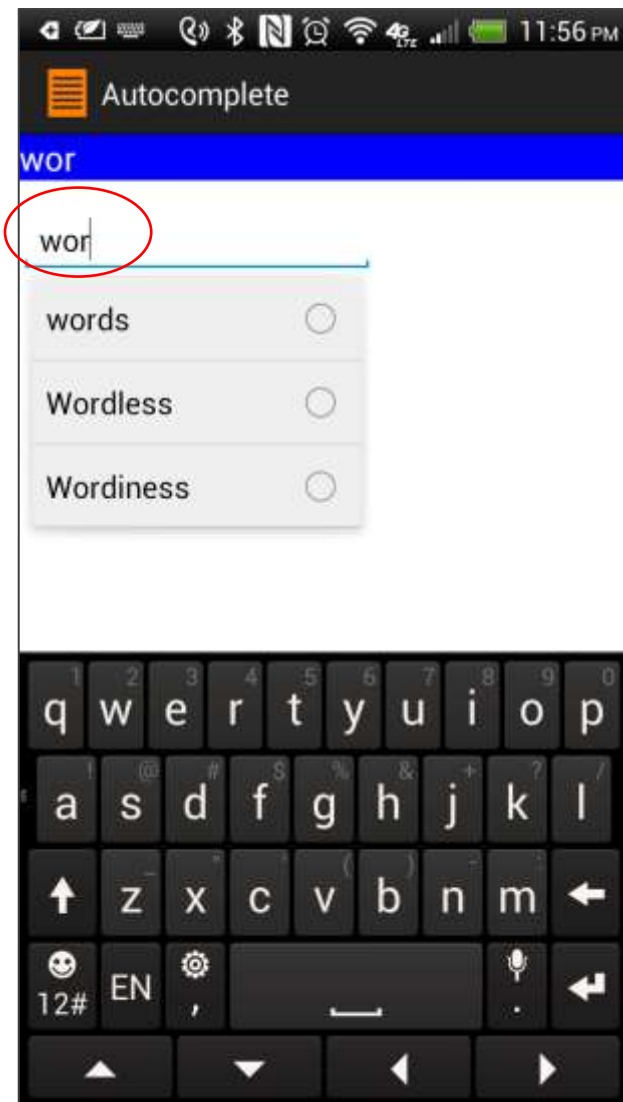
Example 3: GridView Demo - MainActivity

```
class MainActivity : AppCompatActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
  
        val txtMsg: TextView = findViewById(R.id.txtMsg)  
  
        val items: Array<String> = arrayOf("Data-0", "Data-1", "Data-2", "Data-3",  
            "Data-4", "Data-5", "Data-6", "Data-7")  
        val arrayAdapter: ArrayAdapter<String> = ArrayAdapter(  
            this,  
            android.R.layout.simple_list_item_1,  
            items)  
  
        findViewById<GridView>(R.id.grid).run {  
            adapter = arrayAdapter  
            onItemClick = object : OnItemClickListener {  
                override fun onItemClick(p0: AdapterView<*>?, p1: View?, p2: Int, p3: Long) {  
                    txtMsg.text = items[p2]  
                }  
            }  
        }  
    }  
}
```

The AutoCompleteTextView Widget

- An **AutoComplete** box is a more specialized version of the **EditText** view.
- Characters typed so far are compared with the beginning of words held in a user-supplied list of *suggested* values.
- Suggestions matching the typed prefix are shown in a *selection list*.
- The user can choose from the suggestion list or complete typing the word.
- The **android:completionThreshold** property is used to trigger the displaying of the suggestion list. It indicates the number of characters to watch for in order to match prefixes.

The AutoCompleteTextView Widget



Example 4.

A list of selected words beginning with “**wor**” or “**set**” is being watched.

If any of these prefixes (3 letters) are entered the TextWatcher mechanism shows an option list.

Example 4: AutoComplete Demo – Layout

```
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent" >

    <TextView
        android:id="@+id/txtMsg"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:textSize="20sp"
        android:textColor="#ffffffff"
        android:background="#ff0000ff" >
    </TextView>

    <AutoCompleteTextView
        android:id="@+id/autoCompleteTextView1"
        android:hint="type here..."
        android:completionThreshold="3"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_below="@+id/txtMsg"
        android:layout_marginTop="15dp"
        android:ems="10" />

</RelativeLayout>
```



Wait 3 chars to work

Example 4: AutoComplete Demo – Main Activity

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        val txtMsg: TextView = findViewById(R.id.txtMsg)
        val items: Array<String> = arrayOf("words", "starting", "with", "set", "Setback", "Setline",
        "Setoffs", "Setouts", "Setters", "Setting", "Settled", "Settler", "Wordless", "Wordiness", "Adios")
        val arrayAdapter: ArrayAdapter<String> = ArrayAdapter(this,
        android.R.layout.simple_list_item_1, items)
        findViewById<AutoCompleteTextView>(R.id.edit_keyword).run {
            setAdapter(arrayAdapter)
            addTextChangedListener(object : TextWatcher {
                override fun beforeTextChanged(p0: CharSequence?, p1: Int, p2: Int, p3: Int) {
                    TODO("Not yet implemented")
                }

                override fun onTextChanged(p0: CharSequence?, p1: Int, p2: Int, p3: Int) {
                    txtMsg.text = text
                }

                override fun afterTextChanged(p0: Editable?) {
                    TODO("Not yet implemented")
                }
            })
        })
    }
}
```

Example 4: AutoComplete Demo – Main Activity

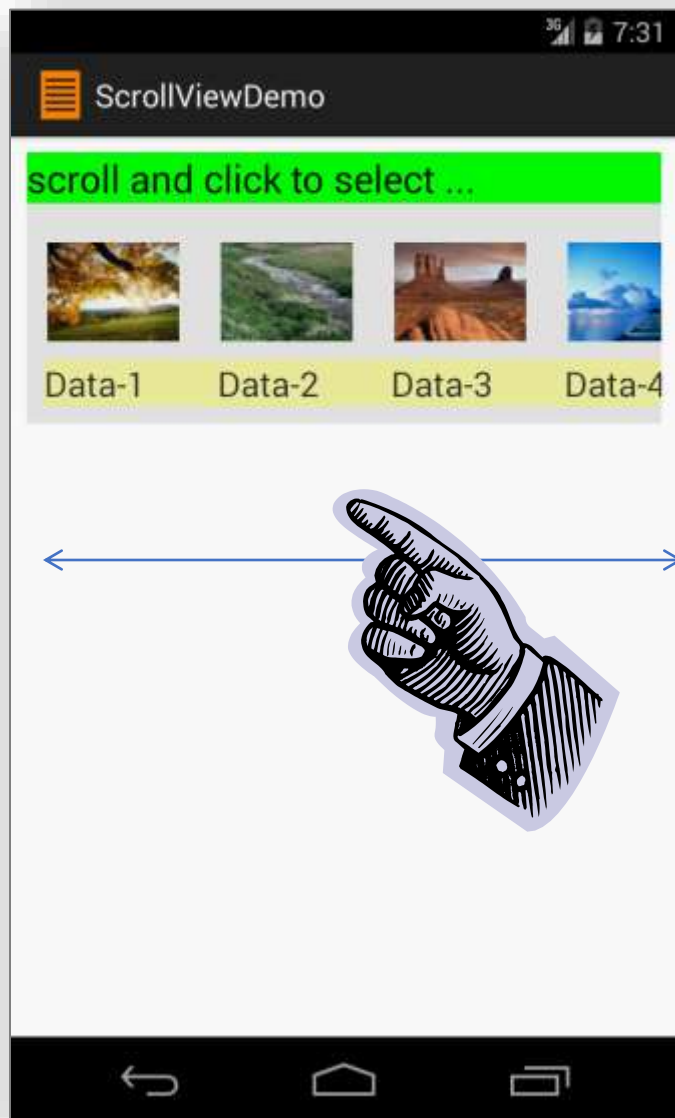
```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val txtMsg: TextView = findViewById(R.id.txtMsg)

        val items: Array<String> = arrayOf("words", "starting", "with", "set",
            "Setback", "Setline", "Setoffs", "Setouts", "Setters", "Setting",
            "Settled", "Settler", "Wordless", "Wordiness", "Adios")
        val arrayAdapter: ArrayAdapter<String> = ArrayAdapter(
            this,
            android.R.layout.simple_list_item_1,
            items)

        findViewById<AutoCompleteTextView>(R.id.edit_keyword).run {
            setAdapter(arrayAdapter)
            addTextChangedListener(onTextChanged = {text, start, count, after ->
                txtMsg.text = text})
        }
    }
}
```

The HorizontalScrollView Widget



HorizontalScrollViews allow the user to graphically select an option from a set of small images called *thumbnails*⁺.

The user interacts with the viewer using two simple actions:

1. Scroll the list (left ↔ right)
2. Click on a thumbnail to pick the option it offers.

In our example, when the user clicks on a thumbnail the app responds by displaying a high-resolution version of the image

⁺. A typical thumbnail size is 100x100 pixels (or less).

Example 5: HorizontalScrollView Demo

- In this example we place a **HorizontalScrollView** at the top of the screen, this view will show a set of thumbnail options.
- The user may scroll through the images and finally tap on a particular selection.
- A better quality version of the selected picture will be displayed in an **ImageView** widget placed below the horizontal scroller.



Example 5: Populating The HorizontalScrollView Widget

1. Our **HorizontalScrollView** will expose a list of frames, each containing an *icon* and a *caption* below the icon.
2. The *frame_icon_caption.xml* layout describes the formatting of icon and its caption. This layout will be **inflated** in order to create run-time GUI objects.
3. After the current *frame* is filled with data, it will be added to the growing set of views hosted by the *scrollViewgroup* container (scrollViewgroup is nested inside the horizontal scroller).
4. Each *frame* will receive an **ID** (its current position in the scrollViewgroup) as well as an individual **onClick** listener.

Example 5: HorizontalScrollView Demo – Layout 1 of 2

```
<?xml version="1.0" encoding="UTF-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:background="#ffffffff"
    android:orientation="vertical"
    android:padding="2dp" >

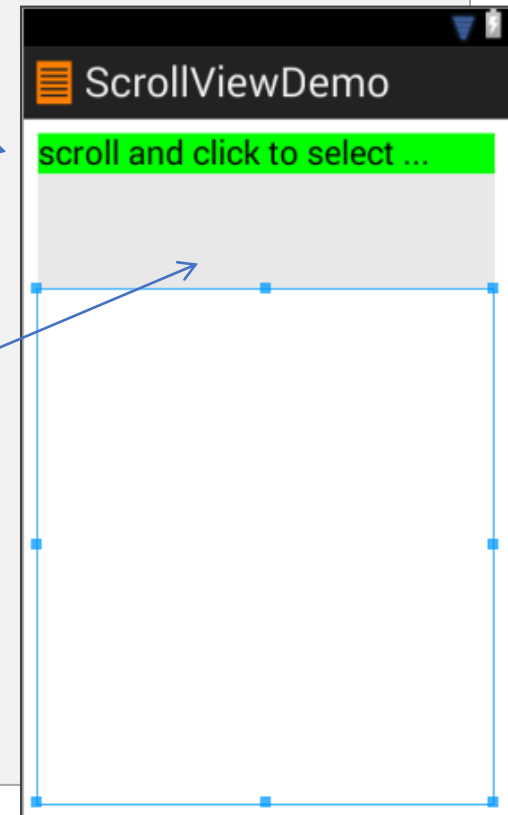
    <TextView
        android:id="@+id/txtMsg"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:background="#ff00ff00"
        android:text="scroll and click to select ..."
        android:textAppearance="?android:attr/textAppearanceLarge" />

    <HorizontalScrollView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:background="#44aaaaaa" >

        <LinearLayout
            android:id="@+id/viewgroup"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:orientation="horizontal"
            android:padding="10dip" >

            </LinearLayout>

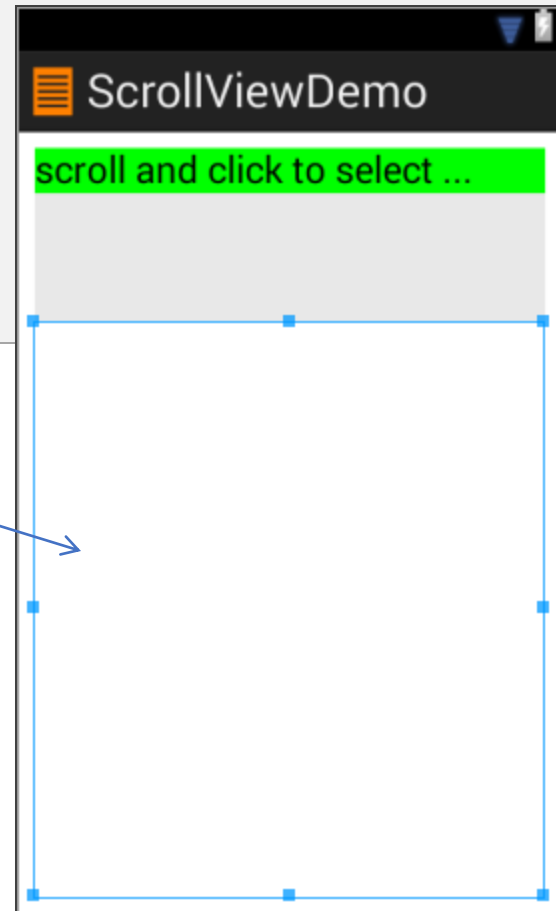
        </HorizontalScrollView>
```



Example 5: HorizontalScrollView Demo – Layout 2 of 2

```
<ImageView  
    android:id="@+id/imageSelected"  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    android:layout_weight="2" />
```

```
</LinearLayout>
```



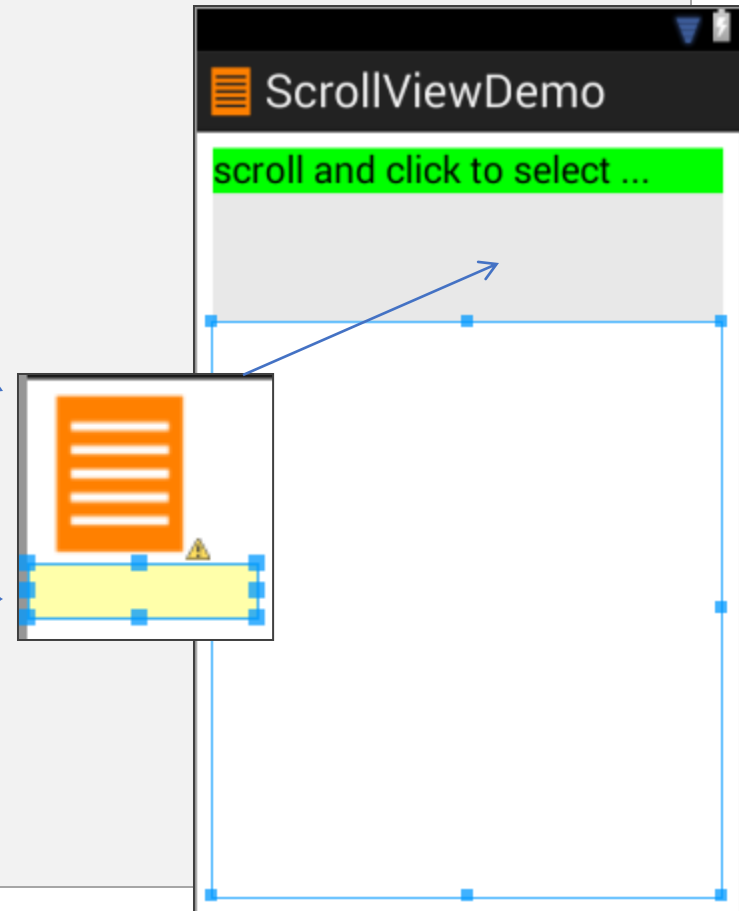
Example 5: Layout: *frame_icon_caption.xml*

```
<?xml version="1.0" encoding="UTF-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical" >

    <ImageView
        android:id="@+id/icon"
        android:layout_width="80dp"
        android:layout_height="80dp"
        android:paddingLeft="2dp"
        android:paddingRight="2dp"
        android:paddingTop="2dp"
        android:src="@drawable/ic_launcher" />

    <TextView
        android:id="@+id/caption"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:background="#55ffff00"
        android:textSize="20sp" />

</LinearLayout>
```



This layout will be used by an **inflater** to dynamically create new views.

These views will be added to the linear layout contained inside the HorizontalScrollerView.

Example 5: HorizontalScrollView Demo – MainActivity 1 of 2

```
class MainActivity : AppCompatActivity() {  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
  
        val txtMsg: TextView = findViewById(R.id.txtMsg)  
        val viewGroup: LinearLayout = findViewById(R.id.viewgroup)  
        val imageSelected: ImageView = findViewById(R.id.imageSelected)  
  
        // Prepare arrays of data  
        val captionList: ArrayList<String> = ArrayList()  
        val thumbList: ArrayList<Int> = ArrayList()  
        val imageList: ArrayList<Int> = ArrayList()  
  
        for (i in 1..27) {  
            captionList.add("Data-$i")  
            thumbList.add(resources.getIdentifier("thumb$i", "drawable", packageName))  
            imageList.add(resources.getIdentifier("wall$i", "drawable", packageName))  
        }  
    }  
}
```



Example 5: HorizontalScrollView Demo – MainActivity 2 of 2

```
// Populate the scroll view
for (i in 0..<captionList.size) {
    val view: View = LayoutInflater.inflate(R.layout.item_view, viewGroup, false)
    view.id = i

    view.findViewById<TextView>(R.id.caption).text = captionList[i]
    view.findViewById<ImageView>(R.id.icon).setImageResource(thumbList[i])

    viewGroup.addView(view)

    // Process click event
    view.setOnClickListener {
        txtMsg.text = "Selected position: ${view.id}"
        imageSelected.setImageResource(imageList[view.id])
    }
}
}
```

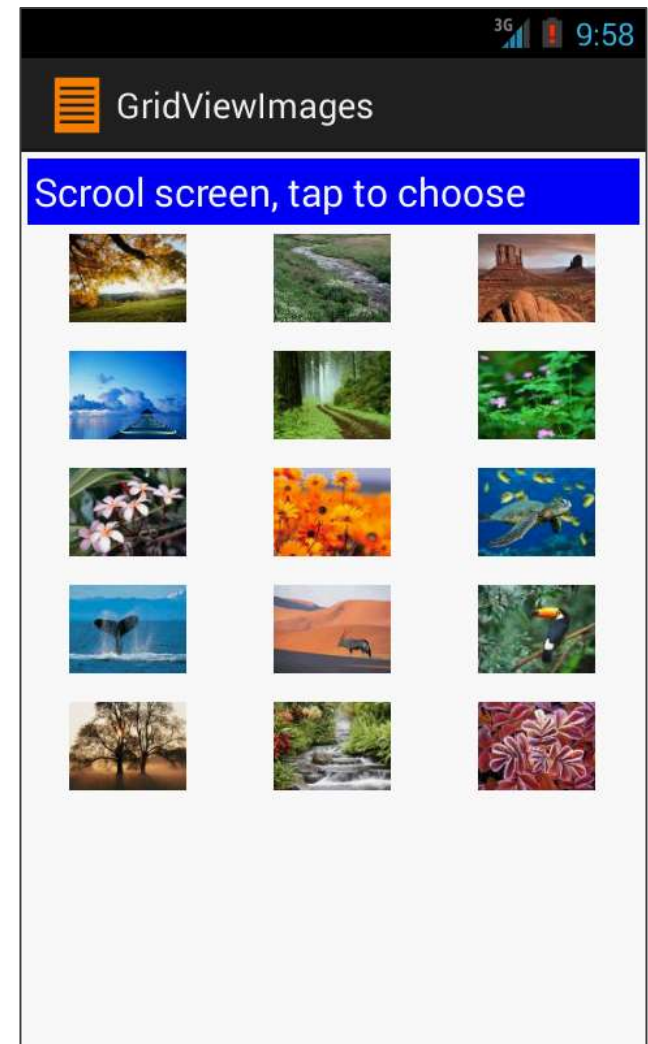


Image-based GridView

Perhaps a more interesting version of the **GridView** control involves the displaying of *images* instead of *text*.

The following example illustrates how to use this control:

1. A screen shows an array of thumbnails.
2. The programmer must provide a custom data adapter to manage the displaying of thumbnails from the data set.



Example 6: GridView Images Demo – Layout (activity_main)

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="3dp" >
```

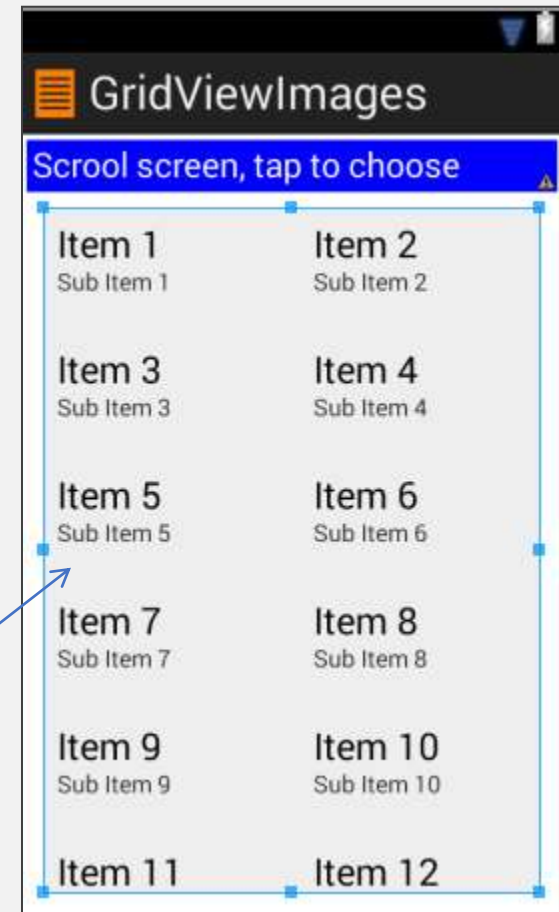
<TextView

```
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:background="#ff0000ff"
    android:padding="3dp"
    android:text="Scrool screen, tap to choose"
    android:textColor="#ffffff"
    android:textSize="20sp" />
```

<GridView

```
    android:id="@+id/gridview"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:layout_margin="1dp"
    android:columnWidth="100dp"
    android:gravity="center"
    android:horizontalSpacing="5dp"
    android:numColumns="auto_fit"
    android:stretchMode="columnWidth"
    android:verticalSpacing="10dp" />
```

```
</LinearLayout>
```



Example 6: MainActivity

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val thumbList = arrayListOf<Int>()
        val imageList = arrayListOf<Int>()

        // Prepare arrays of data
        for (i in 1..27) {
            thumbList.add(resources.getIdentifier("thumb$i", "drawable", packageName))
            imageList.add(resources.getIdentifier("wall$i", "drawable", packageName))
        }

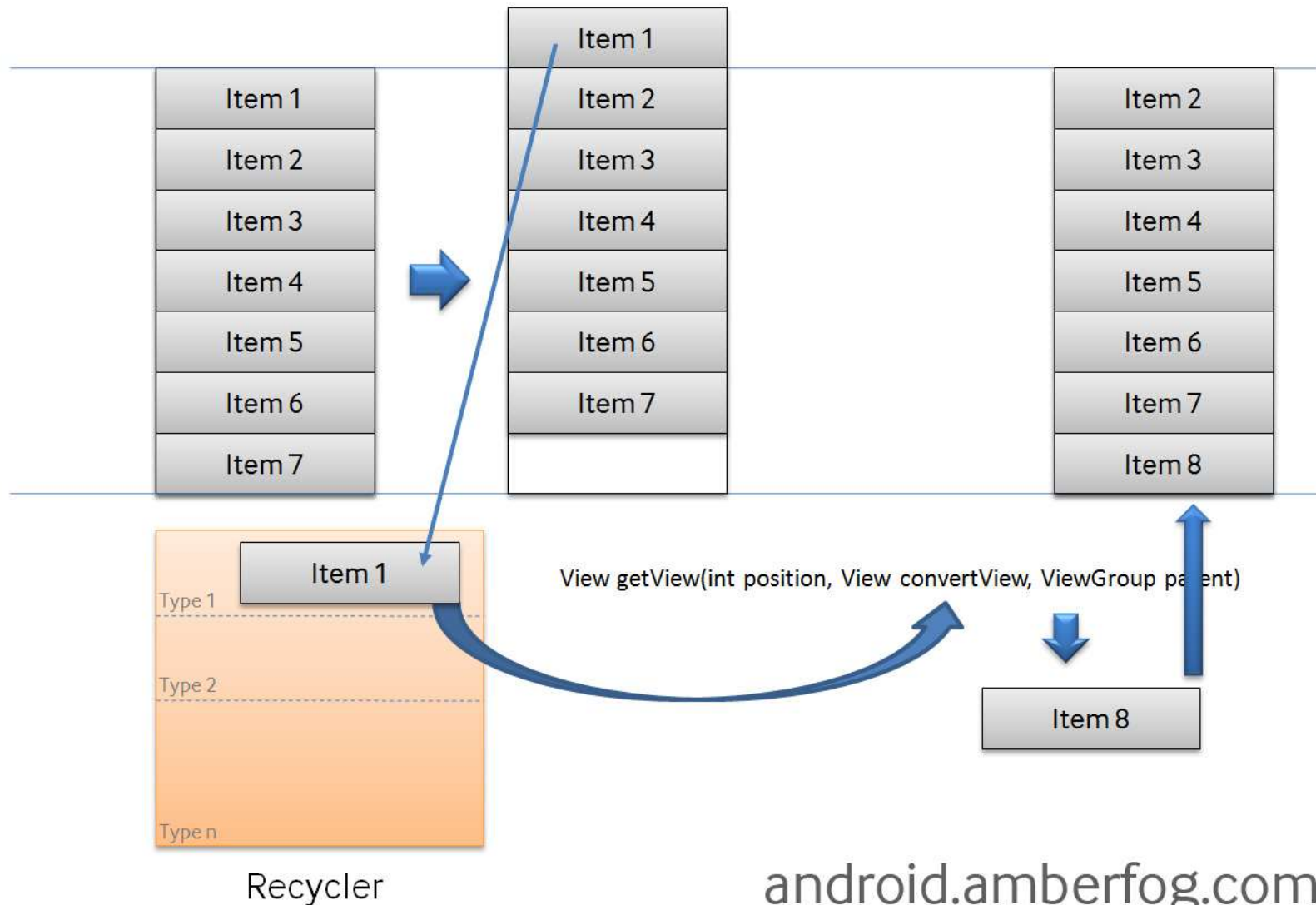
        // Setup grid view
        val gridView = findViewById<GridView>(R.id.gridview)
        gridView.adapter = MyImageAdapter(thumbList)
    }
}
```



Example 6: Custom Adapter - MyImageAdapter

```
class MyImageAdapter(val thumbList: ArrayList<Int>) : BaseAdapter() {  
  
    override fun getCount(): Int = thumbList.size  
  
    override fun getItem(p0: Int): Any = thumbList[p0]  
  
    override fun getItemId(p0: Int): Long = p0.toLong()  
  
    override fun getView(p0: Int, p1: View?, p2: ViewGroup?): View {  
        var imageView: ImageView  
        if (p1 == null) {  
            imageView = ImageView(p2?.context)  
            imageView.layoutParams = LayoutParams(200, 200)  
            imageView.scaleType = ImageView.ScaleType.CENTER_CROP  
            imageView.setPadding(5)  
        } else  
            imageView = p1 as ImageView  
  
        imageView.setImageResource(thumbList[p0])  
  
        return imageView  
    }  
}
```

ListView's Recycler



Example 7: Defining your own ListViews

- Android provides several predefined row layouts for displaying simple lists (such as:
`android.R.layout.simple_list_item_1`,
`android.R.layout.simple_list_item_2`, etc).
- However, there are occasions in which you want a particular disposition and formatting of elements displayed in each list-row.
- In those cases, you should *create your own subclass of a Data Adapter*.
- The next example shows how to do that.

Example 7: Create your own DataAdapter

In order to customize a Data Adapter, you need to:

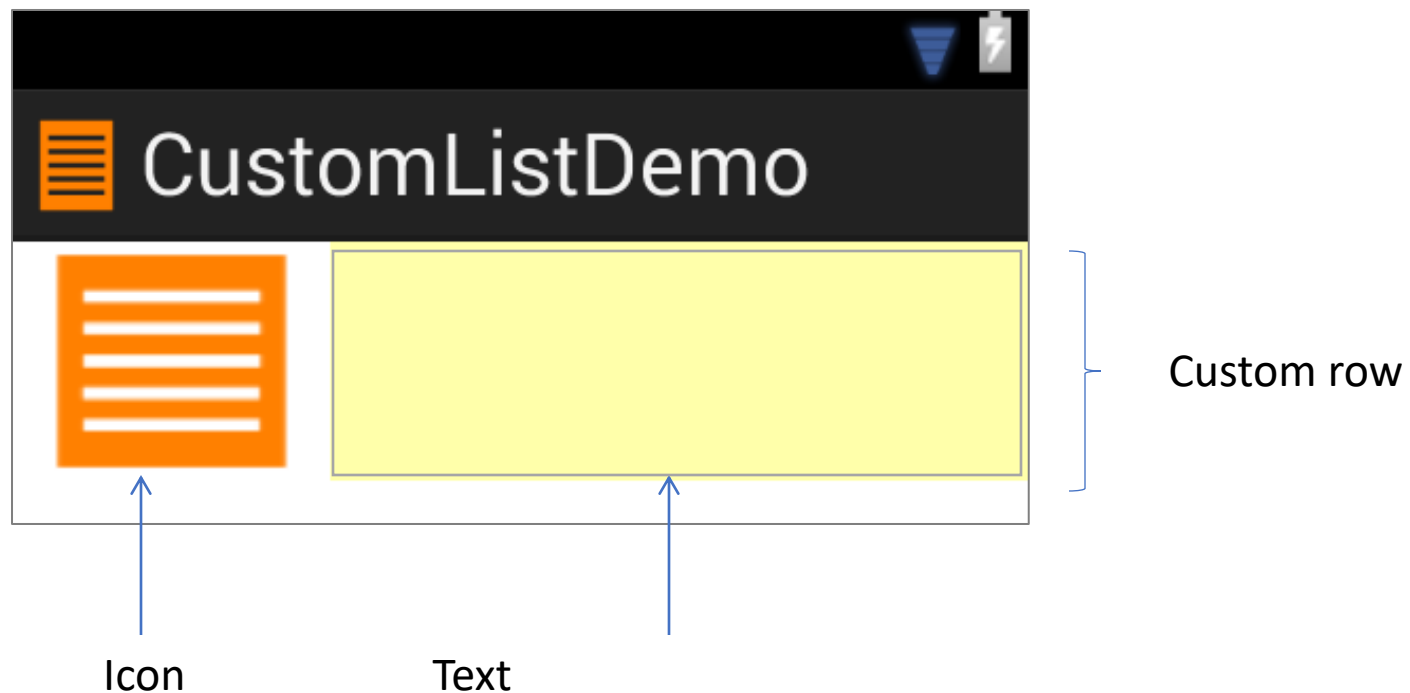
1. Create a class extending the concrete `ArrayAdapter` class
2. Override its `getView()`, and
3. Construct (`inflate`) your rows yourself.

```
class MyCustomAdapter :BaseAdapter() {  
    override fun getCount(): Int {  
        TODO("Not yet implemented")  
    }  
  
    override fun getItem(p0: Int): Any {  
        TODO("Not yet implemented")  
    }  
  
    override fun getItemId(p0: Int): Long {  
        TODO("Not yet implemented")  
    }  
  
    override fun getView(p0: Int, p1: View?, p2: ViewGroup?): View {  
        TODO("Not yet implemented")  
    }  
}
```

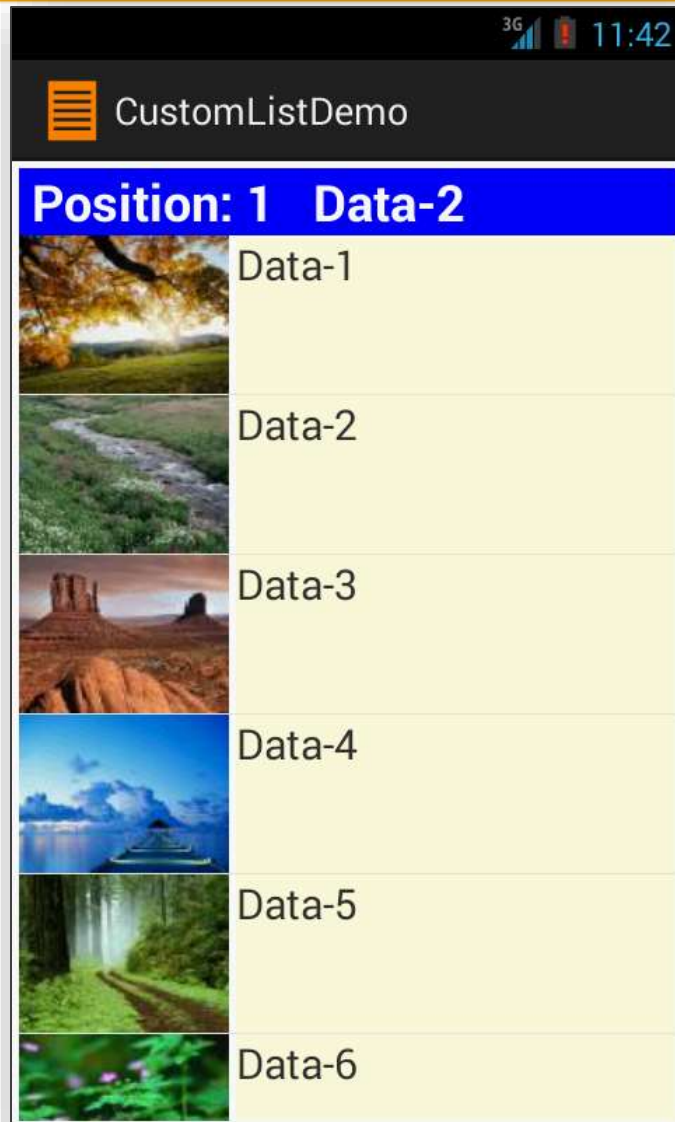
For each data element supplied by the adapter, the method `getView()` returns its 'visible' View.

Example 7: Designing Custom-Rows

In our example each UI row will show an icon (on the left side) and text following the icon to its right side.



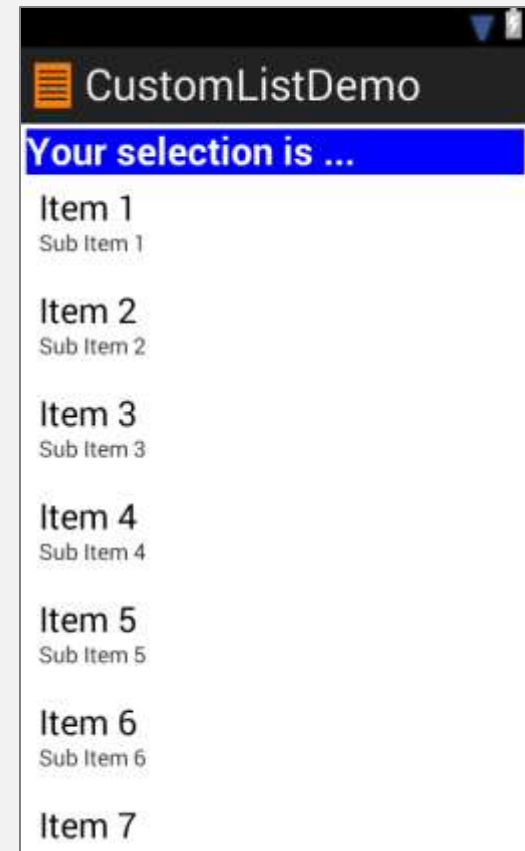
Example 7: Designing Custom-Rows



Custom row consists
of icon & text

Example 7: Layout 1 – activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="3dp"
    android:orientation="vertical" >
    <TextView
        android:id="@+id/txtMsg"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:background="#ff0000ff"
        android:text="Your selection is ..."
        android:textColor="#ffffffff"
        android:textSize="24sp"
        android:textStyle="bold" />
    <ListView
        android:id="@+id/listView"
        android:layout_width="match_parent"
        android:layout_height="match_parent" >
    </ListView>
</LinearLayout>
```



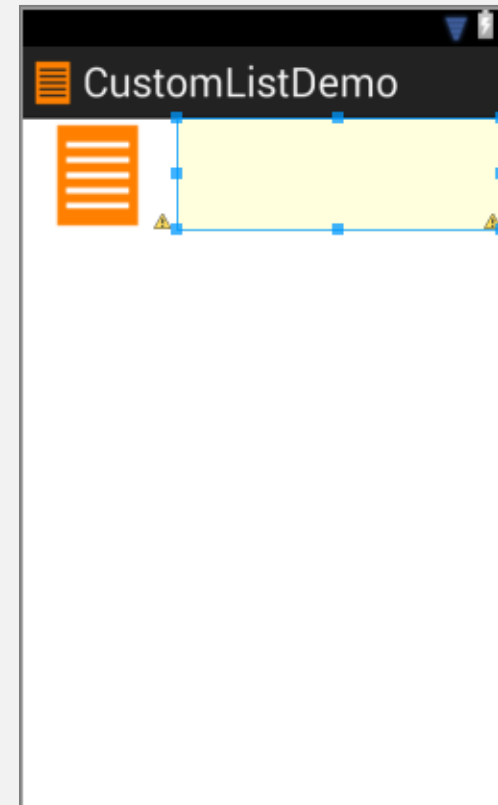
Example 7: Layout 2 – custom_row_icon_label.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="fill_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal" >

    <ImageView
        android:id="@+id/icon"
        android:layout_width="100dp"
        android:layout_height="75dp"
        android:layout_marginRight="3dp"
        android:src="@drawable/ic_launcher" />

    <TextView
        android:id="@+id/label"
        android:layout_width="match_parent"
        android:layout_height="75dp"
        android:background="#22ffff00"
        android:textSize="20sp" />

</LinearLayout>
```



Example 7: MainActivity

```
class MainActivity : AppCompatActivity() {

    lateinit var txtMsg: TextView

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        txtMsg = findViewById<TextView>(R.id.txtMsg)

        // Prepare arrays of data
        val itemList = arrayListOf<ItemModel>()
        for (i in 1..27) {
            itemList.add(ItemModel("Data $i", resources.getIdentifier("thumb$i",
"drawable", packageName)))
        }

        val listView = findViewById<ListView>(R.id.listView)
        listView.adapter = MyCustomAdapter(itemList)
        listView.setOnItemClickListener { adapterView, view, i, l -> txtMsg.text =
"Position $i: ${itemList[i]}"
        }
    }
}
```

Example 7: Custom ListView Demo – Adapter

```
data class ItemModel(val caption: String, val imageResource: Int)
```

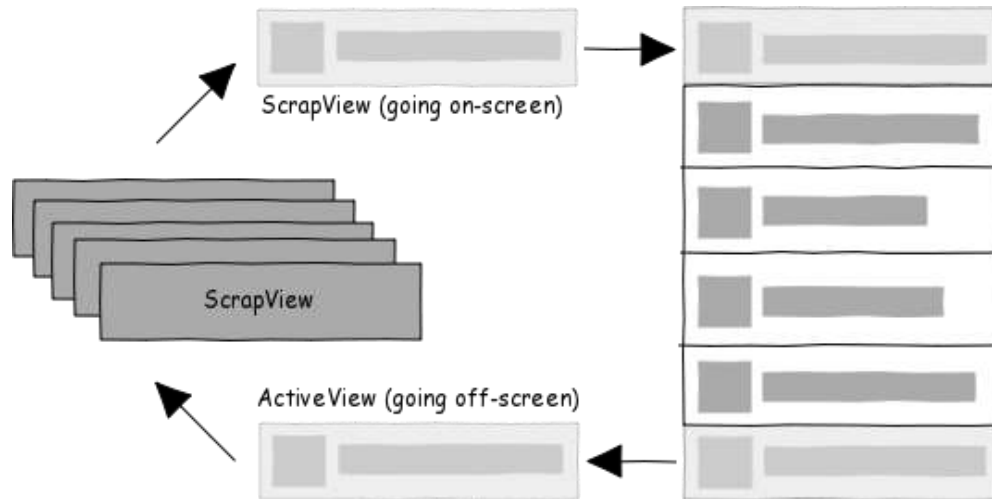
```
class MyCustomAdapter(val items: ArrayList<ItemModel>): BaseAdapter() {  
    override fun getCount(): Int = items.size  
  
    override fun getItem(p0: Int): Any = items[p0]  
  
    override fun getItemId(p0: Int): Long = p0.toLong()  
  
    override fun getView(p0: Int, p1: View?, p2: ViewGroup?): View {  
        val row: View =  
        LayoutInflater.from(p2?.context).inflate(R.layout.custom_row_icon_label, p2,  
        false)  
  
        val textView = row.findViewById<TextView>(R.id.label)  
        val imageView = row.findViewById<ImageView>(R.id.icon)  
  
        textView.text = items[p0].caption  
        imageView.setImageResource(items[p0].imageResource)  
  
        return row  
    }  
}
```

The LayoutInflater Class

- The **LayoutInflater** class converts an XML layout specification into an actual tree of View objects. The objects inflated by code are appended to the selected UI view. It typically works in cooperation with an ArrayAdapter.
- A basic **ArrayAdapter** requires three arguments: current **context**, **layout** on which output rows are shown, source data **items** (data to feed the rows).
 - The overridden **getView()** method inflates the row layout by custom allocating *icons* and *text* taken from data source in the user designed row.
 - Once assembled, the View (row) is returned.
 - This process is repeated for each item supplied by the ArrayAdapter.
 - Next example is a better built custom-adapter using the **ViewHolder** design strategy.

ViewHolder design pattern

The figure below is from “**Performance Tips for Android’s ListView**” by Lucas Rocha <http://lucasr.org/2012/04/05/performance-tips-for-androids-listview/> [Dec, 2014]. It shows a set of rows presented to the user inside a ListView container.



When a row gets out of sight, the memory of its layout is saved in a **scrapview** collection silently kept by the ListView.

If the row comes back to a visible state, you may reuse its scrapview skeleton instead of redoing the row from scratch.

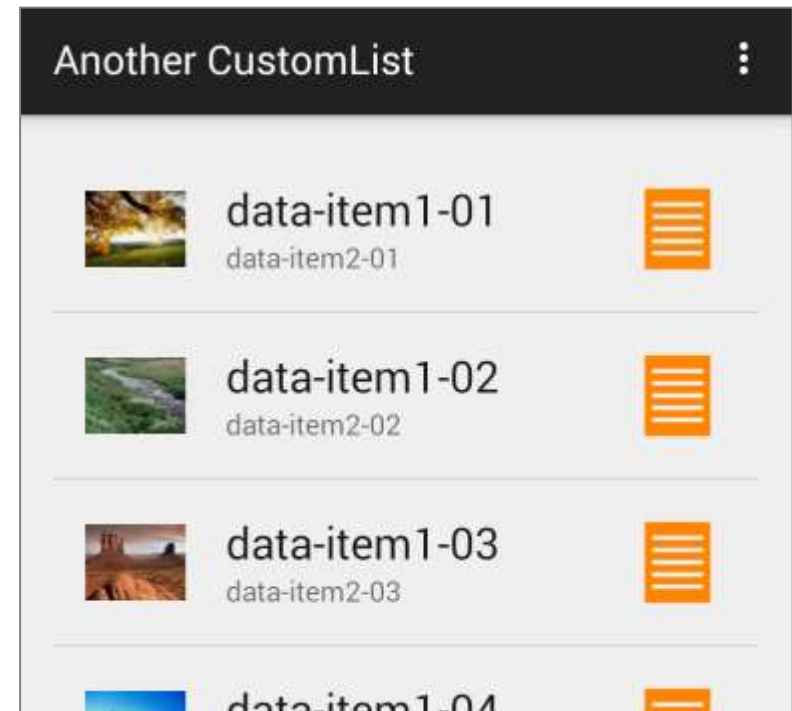
The strategy of reusing these scrapviews is known as the **ViewHolder Design Pattern**. It cuts down on the number of times you have to inflate a row-layout and then get access to its internal widgets by calling the ‘`findViewById()`’ method.

When reusing the scrapviews (made available as ‘`convertView`’) all you need to do is move the appropriate data to the internal widgets and set their `onClick` listeners.

ViewHolder design pattern

In this example a list holding rows showing multiple lines of text and images, is populated with a custom made BaseAdapter that uses the **ViewHolder** strategy for better performance.

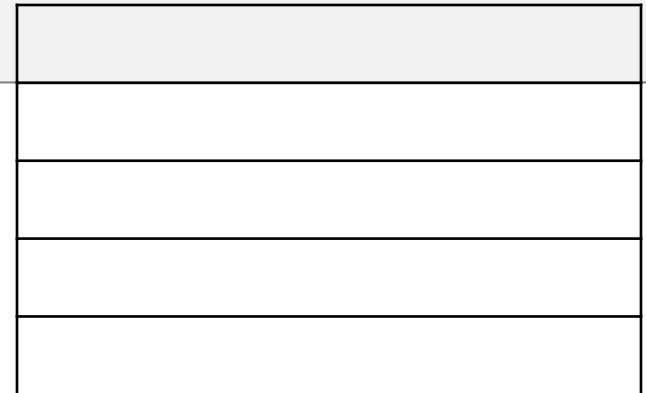
The app consists of two classes: **MainActivity** and **MyCustomAdapter**. It has two layouts: *activity_main* showing the list (see image on the right) and *custom_item_view* describing the structure of individual rows.



Example 8: Custom List – activity_main.xml

Layout *activity_main.xml* shows a `ViewList`.

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="3dp"
    android:orientation="vertical" >
    <ListView
        android:id="@+id/listView"
        android:layout_width="match_parent"
        android:layout_height="match_parent" >
    </ListView>
</LinearLayout>
```



Example 8: Custom List – custom_item_view.xml

Layout *custom_item_view.xml* shows a custom-made row holding one image, one text and one checkbox

```
<?xml version="1.0" encoding="UTF-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="fill_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal" >

    <ImageView
        android:id="@+id/image_icon"
        android:layout_width="75dp"
        android:layout_height="75dp"
        android:src="@drawable/thumb1"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />
```



Example 8: Custom List – custom_item_view.xml

Layout *custom_item_view.xml* shows a custom-made row holding one image, one text and one checkbox

```
<TextView
    android:id="@+id/text_label"
    android:layout_width="0dp"
    android:layout_height="75dp"
    android:background="#22ffff00"
    android:padding="8dp"
    android:text="Data"
    android:textSize="34sp"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toEndOf="@+id/image_icon"
    app:layout_constraintTop_toTopOf="parent" />

<CheckBox
    android:id="@+id/check_select"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_weight="1"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintTop_toTopOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```



Example 8: Custom List – MainActivity

The main activity exposes a ListView. A custom adapter is tied to the ListView. The adapter gets a reference to a test 'database' and the custom row layout.

```
class MainActivity : AppCompatActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
  
        // Prepare arrays of data  
        val itemList = arrayListOf<ItemModel>()  
        for (i in 1..27) {  
            itemList.add(ItemModel("Data $i",  
                                    resources.getIdentifier("thumb$i", "drawable", packageName)))  
        }  
  
        val adapter = MyCustomAdapter(itemList)  
        val listView = findViewById<ListView>(R.id.listView)  
        listView.adapter = adapter  
    }  
}
```

Example 8: Custom List – MyCustomAdapter

The **getView** method in this extended BaseAdapter inflates a supplied row layout, gets access to its internal widgets, fills them with data and set listeners on some of them.

```
class MyCustomAdapter(val items: ArrayList<ItemModel>): BaseAdapter() {  
    override fun getCount(): Int = items.size  
  
    override fun getItem(p0: Int): Any = items[p0]  
  
    override fun getItemId(p0: Int): Long = p0.toLong()  
  
    override fun getView(p0: Int, p1: View?, p2: ViewGroup?): View {  
        var viewHolder: MyViewHolder  
        var itemView: View  
        if (p1 == null) {  
            itemView = LayoutInflater.from(p2?.context)  
                .inflate(R.layout.custom_item_view, p2, false)  
            viewHolder = MyViewHolder()  
            viewHolder.textLabel = itemView.findViewById(R.id.text_label)  
            viewHolder.imageIcon = itemView.findViewById(R.id.image_icon)  
            viewHolder.checkSelect = itemView.findViewById(R.id.check_select)  
            itemView.tag = viewHolder  
        } else {  
            itemView = p1  
            viewHolder = itemView.tag as MyViewHolder  
        }  
    }  
}
```

Example 8: Custom List – MyCustomAdapter

The **getView** method in this extended BaseAdapter inflates a supplied row layout, gets access to its internal widgets, fills them with data and set listeners on some of them.

```
viewHolder.textLabel.text = items[p0].caption
viewHolder.imageIcon.setImageResource(items[p0].imageResource)
viewHolder.checkSelect.isChecked = items[p0].selected

viewHolder.checkSelect.setOnClickListener {
    items[p0].selected = !items[p0].selected
    notifyDataSetChanged()
}

return itemView
}

class MyViewHolder {
    lateinit var textLabel: TextView
    lateinit var imageIcon: ImageView
    lateinit var checkSelect: CheckBox
}
}
```



Dialog, Toast and Snackbar Widgets

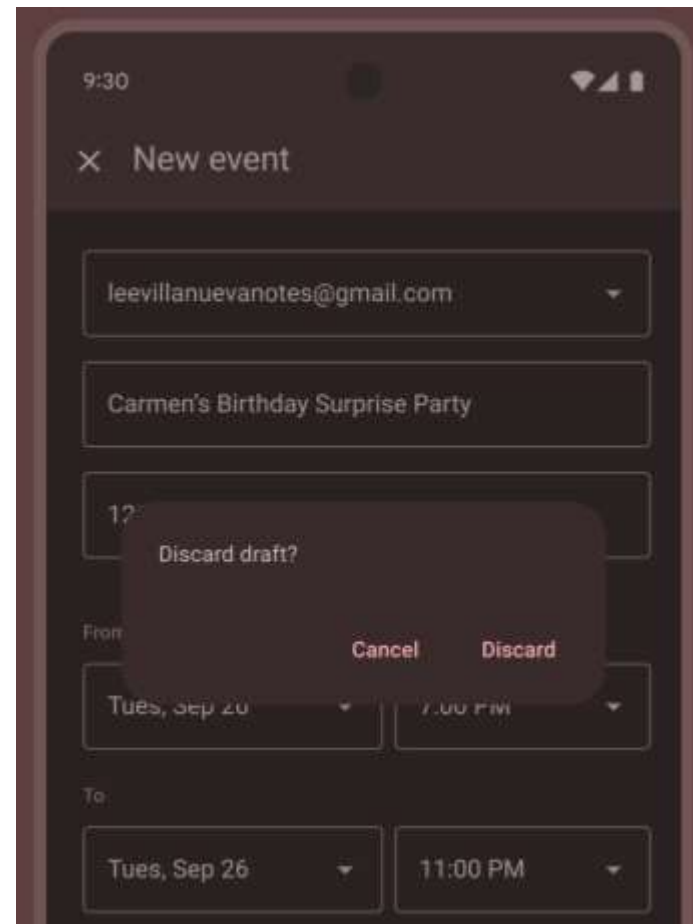
ONE LOVE. ONE FUTURE.

Android Dialogs

A **dialog** is a small window that prompts the user to make a decision or enter additional information. A dialog doesn't fill the screen and is normally used for modal events that require users to take an action before they can proceed.

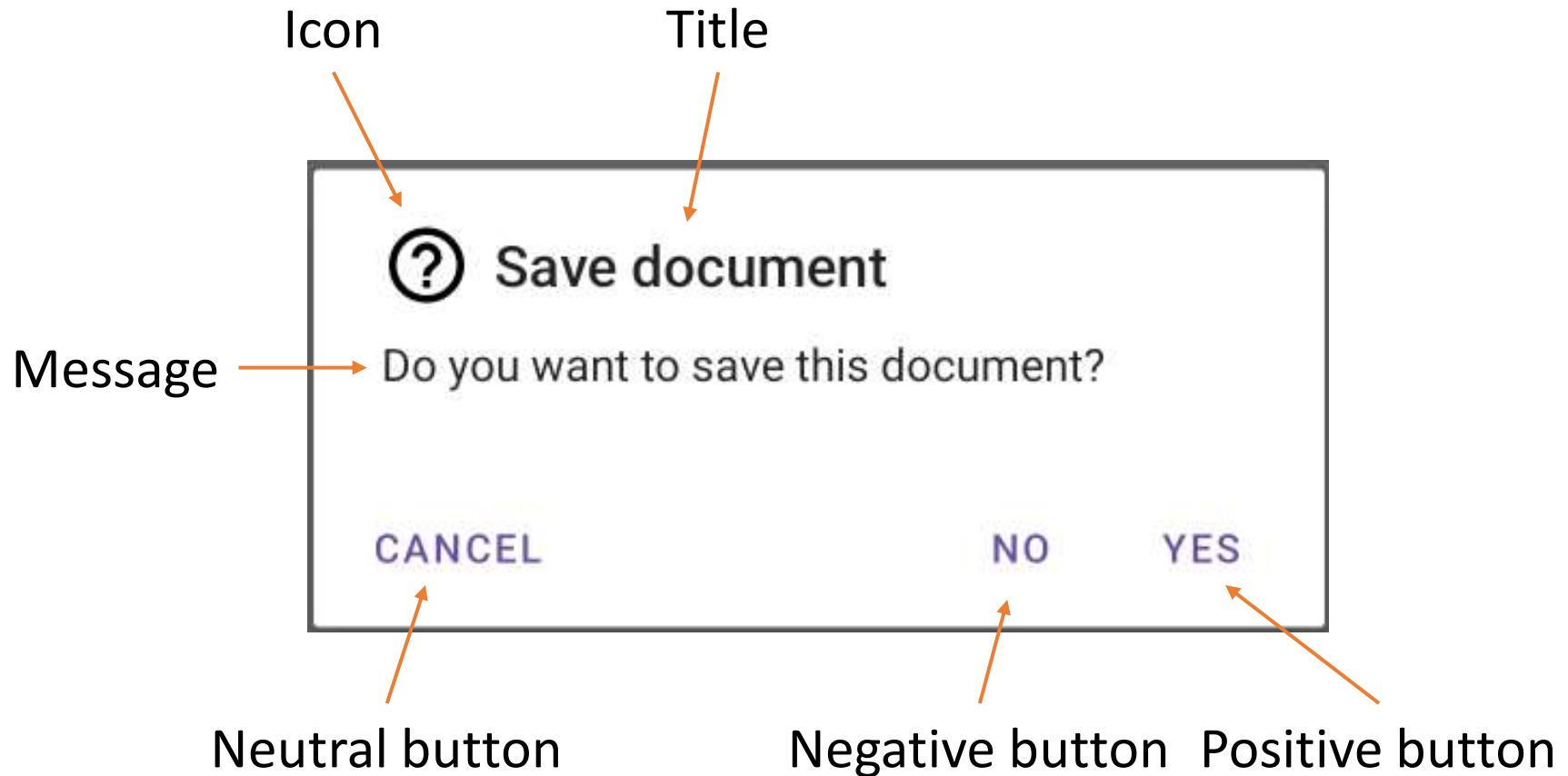
The **Dialog** class is the base class for dialogs, but don't instantiate Dialog directly. Instead, use one of the following subclasses:

- **AlertDialog** A dialog that can show a title, up to three buttons, a list of selectable items, or a custom layout.
- **DatePickerDialog** or **TimePickerDialog** A dialog with a predefined UI that lets the user select a date or time.



The AlertDialog

Dissecting an AlertDialog



Build an alert dialog

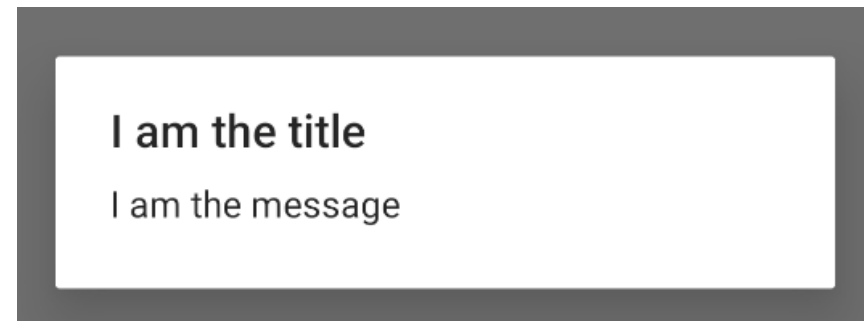
The **AlertDialog** class lets you build a variety of dialog designs and is often the only dialog class you need. As shown in the following figure, there are three regions of an alert dialog:

- Title
- Content area
- Action buttons

The **AlertDialog.Builder** class provides APIs that let you create an AlertDialog with these kinds of content, including a custom layout.

```
val builder: AlertDialog.Builder = AlertDialog.Builder(context)
builder
    .setMessage("I am the message")
    .setTitle("I am the title")
```

```
val dialog: AlertDialog = builder.create()
dialog.show()
```



Add buttons

There are up to three action buttons you can add:

- **Positive:** use this to accept and continue with the action.
- **Negative:** use this to cancel the action.
- **Neutral:** use this when the user might not want to proceed with the action but doesn't necessarily want to cancel. It appears between the positive and negative buttons.

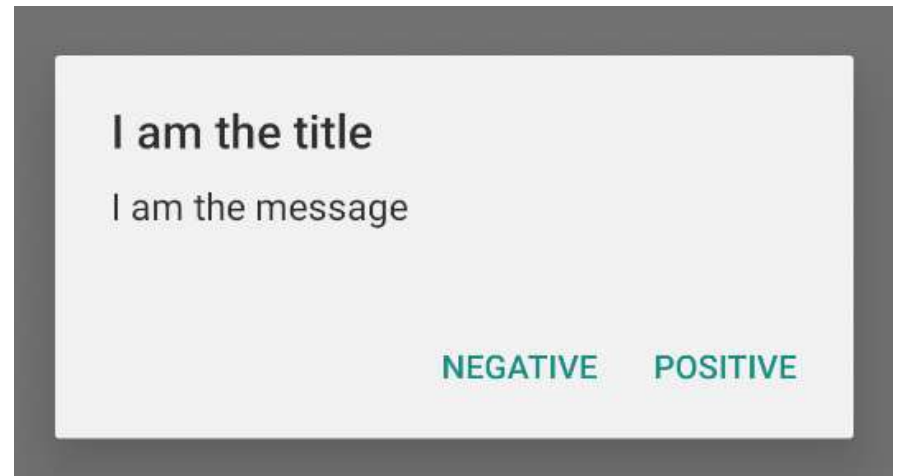
You can add only one of each button type to an **AlertDialog**.

The **set...Button()** methods require a title for the button (supplied by a string resource) and a **DialogInterface.OnClickListener** that defines the action to take when the user taps the button.

Add buttons - Example

```
val builder: AlertDialog.Builder = AlertDialog.Builder(context)
builder
    .setMessage("I am the message")
    .setTitle("I am the title")
    .setPositiveButton("Positive") { dialog, which ->
        // Do something.
    }
    .setNegativeButton("Negative") { dialog, which ->
        // Do something else.
    }

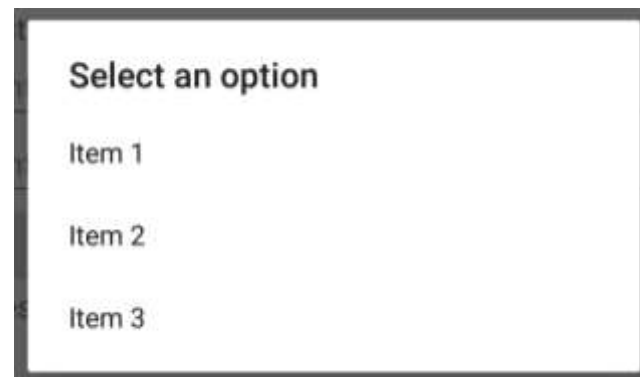
val dialog: AlertDialog = builder.create()
dialog.show()
```



Adding a list

There are three kinds of lists available with the AlertDialog APIs:

- A traditional single-choice list
- A persistent single-choice list (radio buttons)
- A persistent multiple-choice list (checkboxes)



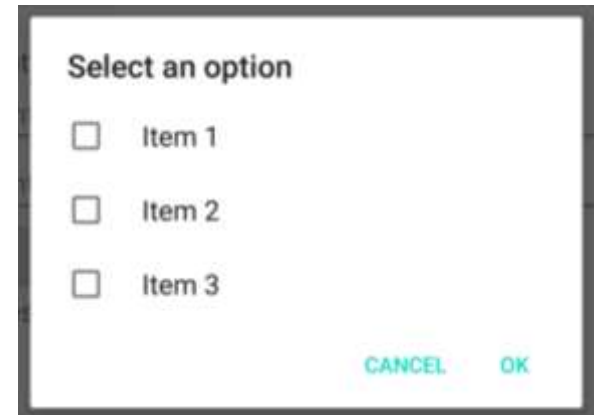
To create a single-choice list like the one in figure 3, use the **setItems()** method:

```
AlertDialog.Builder(this)
    .setTitle("Select an option")
    .setItems(R.array.items) { dialog, which ->
        println("Item selected - $which")
    }
    .create().show()
```

Adding a list

Adding a persistent multiple-choice or single-choice list:

To add a list of multiple-choice items (checkboxes) or single-choice items (radio buttons), use the `setMultiChoiceItems()` or `setSingleChoiceItems()` methods, respectively.



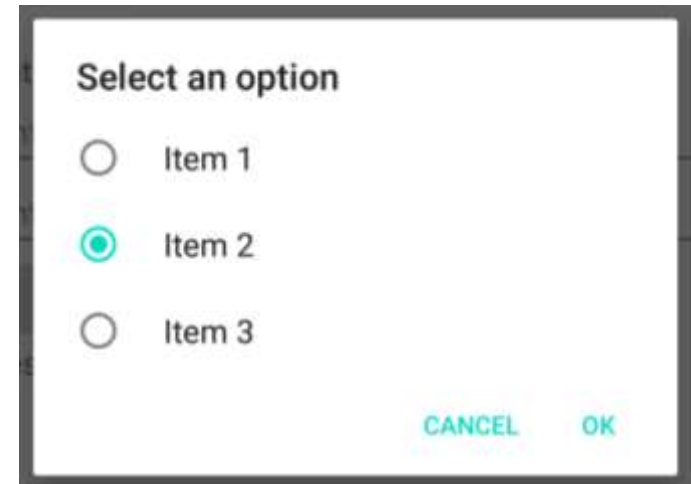
```
val items = resources.getStringArray(R.array.items)
val checked = BooleanArray(items.size)

AlertDialog.Builder(this)
    .setTitle("Select an option")
    .setMultiChoiceItems(items, checked) { dialog, which, isChecked ->
        println("Item selected - $which - $isChecked")
    }
    .setPositiveButton("OK") { dialog, which ->
        println("OK")
    }
    .setNegativeButton("Cancel", null)
    .create().show()
```

Adding a list

Adding a persistent multiple-choice or single-choice list:

To add a list of multiple-choice items (checkboxes) or single-choice items (radio buttons), use the **setMultiChoiceItems()** or **setSingleChoiceItems()** methods, respectively.



```
AlertDialog.Builder(this)
    .setTitle("Select an option")
    .setSingleChoiceItems(items, 0) { dialog, which ->
        println("Item selected - $which")
    }
    .setPositiveButton("OK") { dialog, which ->
        println("OK")
    }
    .setNegativeButton("Cancel", null)
    .create().show()
```

Create a custom layout

If you want a custom layout in a dialog, create a layout and add it to an **AlertDialog** by calling **setView()** on your **AlertDialog.Builder** object, or add the layout to a **Dialog** by using **setContentView()**.

When the user taps an action button created with an **AlertDialog.Builder**, the system dismisses the dialog for you.

The system also dismisses the dialog when the user taps an item in a dialog list, except when the list uses radio buttons or checkboxes. Otherwise, you can manually dismiss your dialog by calling **dismiss()**.

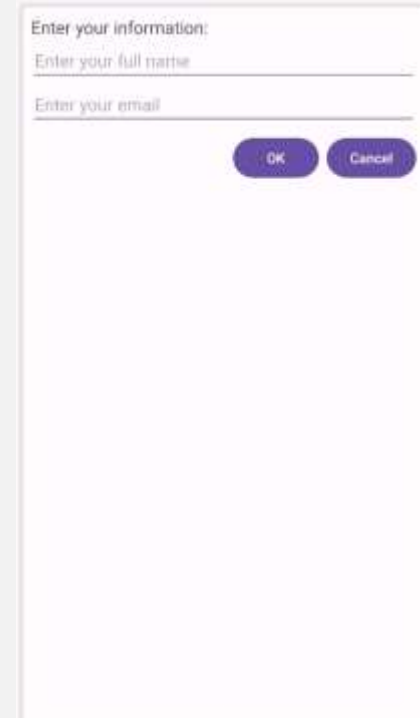
Custom layout example

XML Layout – custom_dialog_layout.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="8dp">

    <TextView
        android:id="@+id/textView"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:text="Enter your information:"
        android:textSize="18sp"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

    <EditText
        android:id="@+id/edit_fullname"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:ems="10"
        android:inputType="text"
        android:hint="Enter your full name"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/textView" />
```



Custom layout example

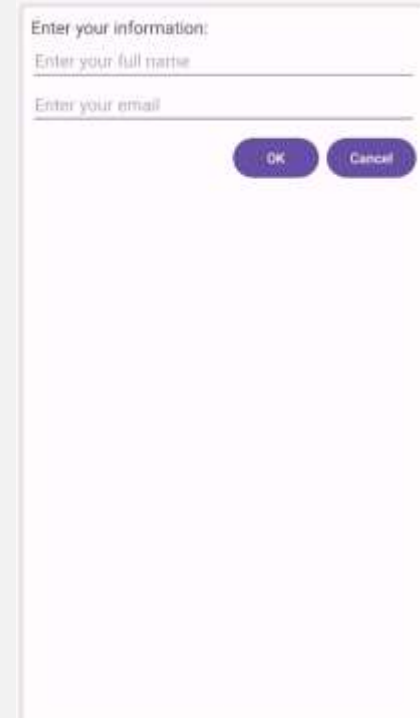
XML Layout – custom_dialog_layout.xml

```
<Button
    android:id="@+id/button_cancel"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="8dp"
    android:text="Cancel"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintTop_toBottomOf="@+id/edit_email" />
```

```
<Button
    android:id="@+id/button_ok"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="8dp"
    android:layout_marginEnd="8dp"
    android:text="OK"
    app:layout_constraintEnd_toStartOf="@+id/button_cancel"
    app:layout_constraintTop_toBottomOf="@+id/edit_email" />
```

```
<EditText
    android:id="@+id/edit_email"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:ems="10"
    android:inputType="textEmailAddress"
    android:hint="Enter your email"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@+id/edit_fullname" />
```

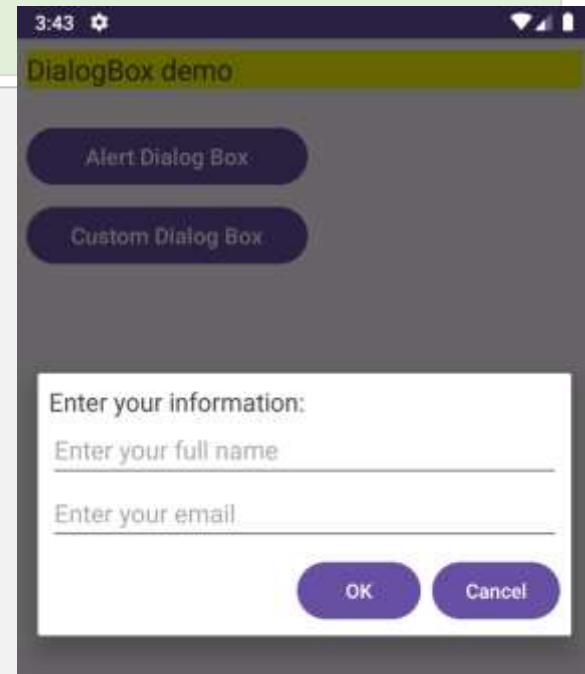
```
</androidx.constraintlayout.widget.ConstraintLayout>
```



Custom layout example

DialogActivity.kt

```
fun showCustomDialog() {  
    val dialog = Dialog(this)  
    dialog setContentView(R.layout.dialog_custom_view)  
    val editFullName = dialog.findViewById<EditText>(R.id.edit_fullname)  
    val editEmail = dialog.findViewById<EditText>(R.id.edit_email)  
    dialog.findViewById<Button>(R.id.button_ok).setOnClickListener {  
        val fullName = editFullName.text.toString()  
        val email = editEmail.text.toString()  
  
        // TODO: Do something with name and email  
  
        dialog.dismiss()  
    }  
    dialog.findViewById<Button>(R.id.button_cancel).setOnClickListener {  
        dialog.dismiss()  
    }  
    dialog.window?.setLayout(ViewGroup.LayoutParams.MATCH_PARENT, ViewGroup.LayoutParams.WRAP_CONTENT)  
    dialog.show()  
}
```



Create a dialog fragment

You also need a **DialogFragment** as a container for your dialog. The **DialogFragment** class provides all the controls you need to create your dialog and manage its appearance, instead of calling methods on the **Dialog** object.

Using **DialogFragment** to manage the dialog makes it correctly handle lifecycle events such as when the user taps the Back button or rotates the screen. The **DialogFragment** class also lets you reuse the dialog's UI as an embeddable component in a larger UI — just like a traditional Fragment — such as when you want the dialog UI to appear differently on large and small screens.

Create a dialog fragment – Example

```
class StartGameDialogFragment : DialogFragment() {
    override fun onCreateDialog(savedInstanceState: Bundle?): Dialog {
        return activity?.let {
            // Use the Builder class for convenient dialog construction.
            val builder = AlertDialog.Builder(it)
            builder.setMessage("Start game")
                .setPositiveButton("Start") { dialog, id ->
                    // START THE GAME!
                }
                .setNegativeButton("Cancel") { dialog, id ->
                    // User cancelled the dialog.
                }
            // Create the AlertDialog object and return it.
            builder.create()
        } ?: throw IllegalStateException("Activity cannot be null")
    }
}

class OldXmlActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_old_xml)

        StartGameDialogFragment().show(supportFragmentManager, "GAME_DIALOG")
    }
}
```

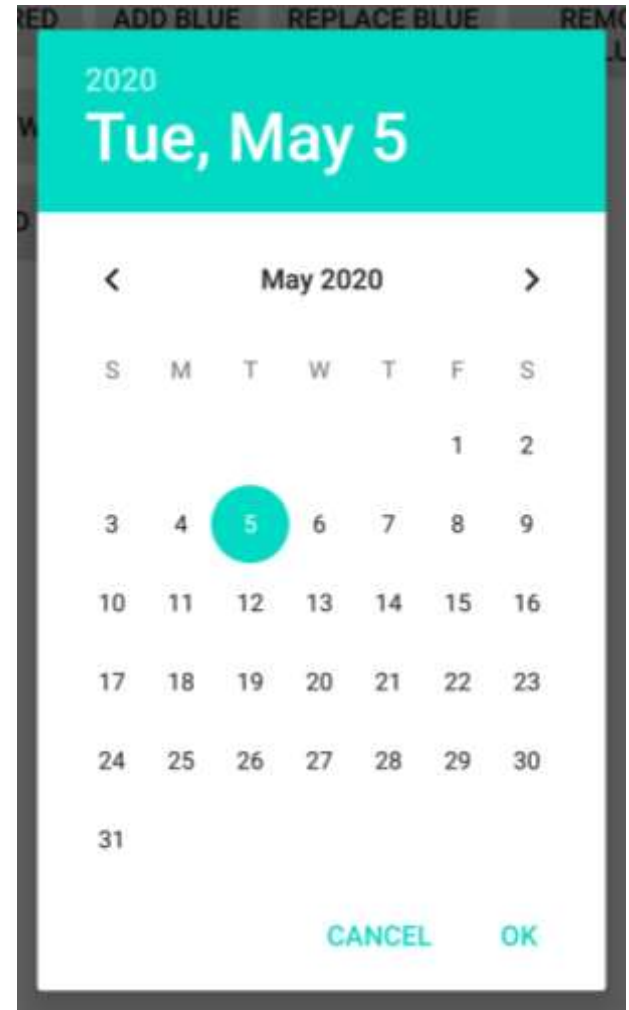
DatePickerDialog

Android provides controls for the user to pick a time or date as ready-to-use dialogs.

These *pickers* provide controls for selecting each part of the time (hour, minute, AM/PM) or date (month, day, year).

```
// Get current date
val c = Calendar.getInstance()
val mYear = c.get(Calendar.YEAR)
val mMonth = c.get(Calendar.MONTH)
val mDay = c.get(Calendar.DAY_OF_MONTH)

// Show dialog
val datePickerDialog = DatePickerDialog(this, {
    view: DatePicker, year: Int, month: Int, day: Int ->
    println("$year-{$month + 1}-$day")
}, mYear, mMonth, mDay)
datePickerDialog.show()
```



TimePickerDialog

```
// Get current time
val c = Calendar.getInstance()
val mHour = c.get(Calendar.HOUR)
val mMinute = c.get(Calendar.MINUTE)

// Show dialog
val timePickerDialog = TimePickerDialog(this, {
    view: TimePicker, hourOfDay: Int, minute: Int ->
        println("$hourOfDay:$minute")
}, mHour, mMinute, false)
timePickerDialog.show()
```

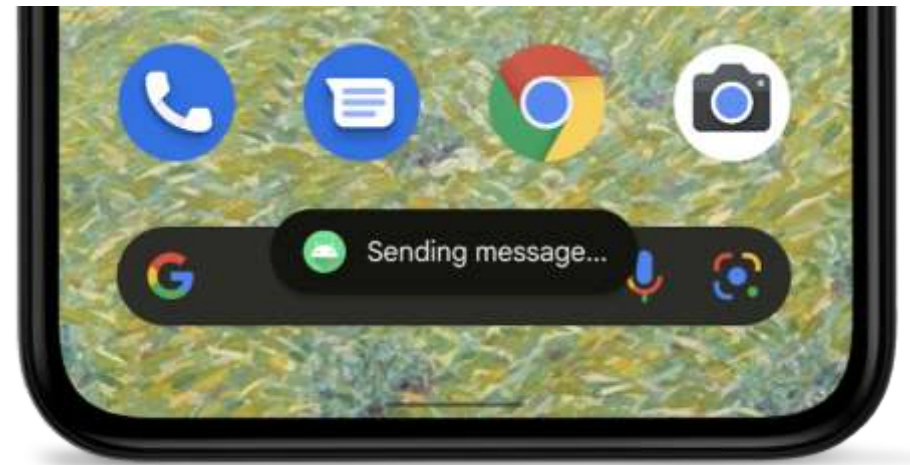


Toast widget

A toast provides simple feedback about an operation in a small popup. It only fills the amount of space required for the message and the current activity remains visible and interactive. Toasts automatically disappear after a timeout.

For example, clicking **Send** on an email triggers a "Sending message..." toast, as shown in the following screen capture:

If your app targets Android 12 (API level 31) or higher, its toast is limited to two lines of text and shows the application icon next to the text. Be aware that the line length of this text varies by screen size, so it's good to make the text as short as possible.



Toast widget

```
Toast.makeText ( context, message, duration ).show()
```

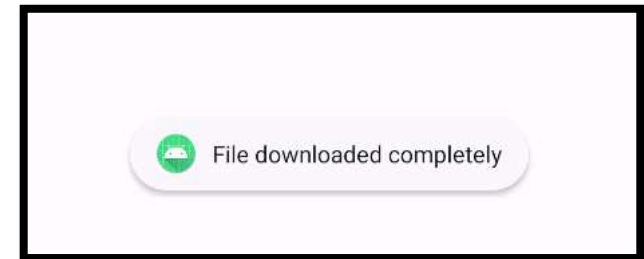
Context: A reference to the view's environment (where am I, what is around me...)

Message: The message you want to show

Duration: Toast.LENGTH_SHORT (0) about 2 sec
Toast.LENGTH_LONG (1) about 3.5 sec

Toast widget

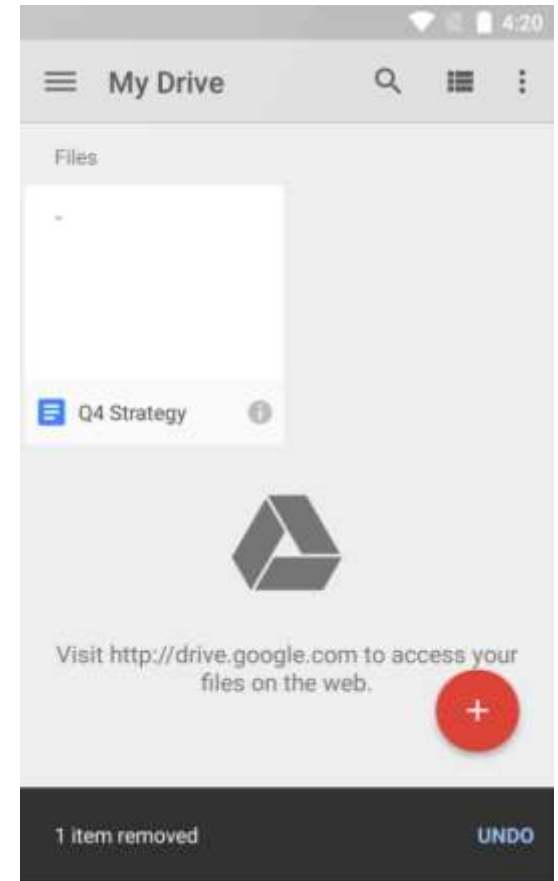
```
class MainActivity : AppCompatActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
  
        Toast.makeText(applicationContext, "File downloaded completely",  
            Toast.LENGTH_LONG).show()  
    }  
}
```



In this simple application, passing the **context** variable could be done using: **applicationContext**, or **this**

Snackbar widget

- Snackbars provide brief messages about app processes at the bottom of the screen.
- Snackbars inform users of a process that an app has performed or will perform. They appear temporarily, towards the bottom of the screen. They shouldn't interrupt the user experience, and they don't require user input to disappear.
- Only one snackbar may be displayed at a time.
- A snackbar can contain a single action. "Dismiss" or "cancel" actions are optional.



Using snackbar

The Snackbar class provides static make methods to produce a snackbar configured in the desired way.

These methods take a View, which will be used to find a suitable ancestor ViewGroup to display the snackbar in, a text string to display, and a duration to display the snackbar for.

Available duration presets are:

- LENGTH_INDEFINITE (Show the snackbar until it's either dismissed or another snackbar is shown)
- LENGTH_LONG (Show the snackbar for a long period of time)
- LENGTH_SHORT (Show the snackbar for a short period of time)



Showing a snackbar

Calling `make` creates the snackbar, but doesn't cause it to be visible on the screen. To show it, use the `show` method on the returned `Snackbar` instance. Note that only one snackbar will be shown at a time. Showing a new snackbar will dismiss any previous ones first.

To show a snackbar with a message and no action:

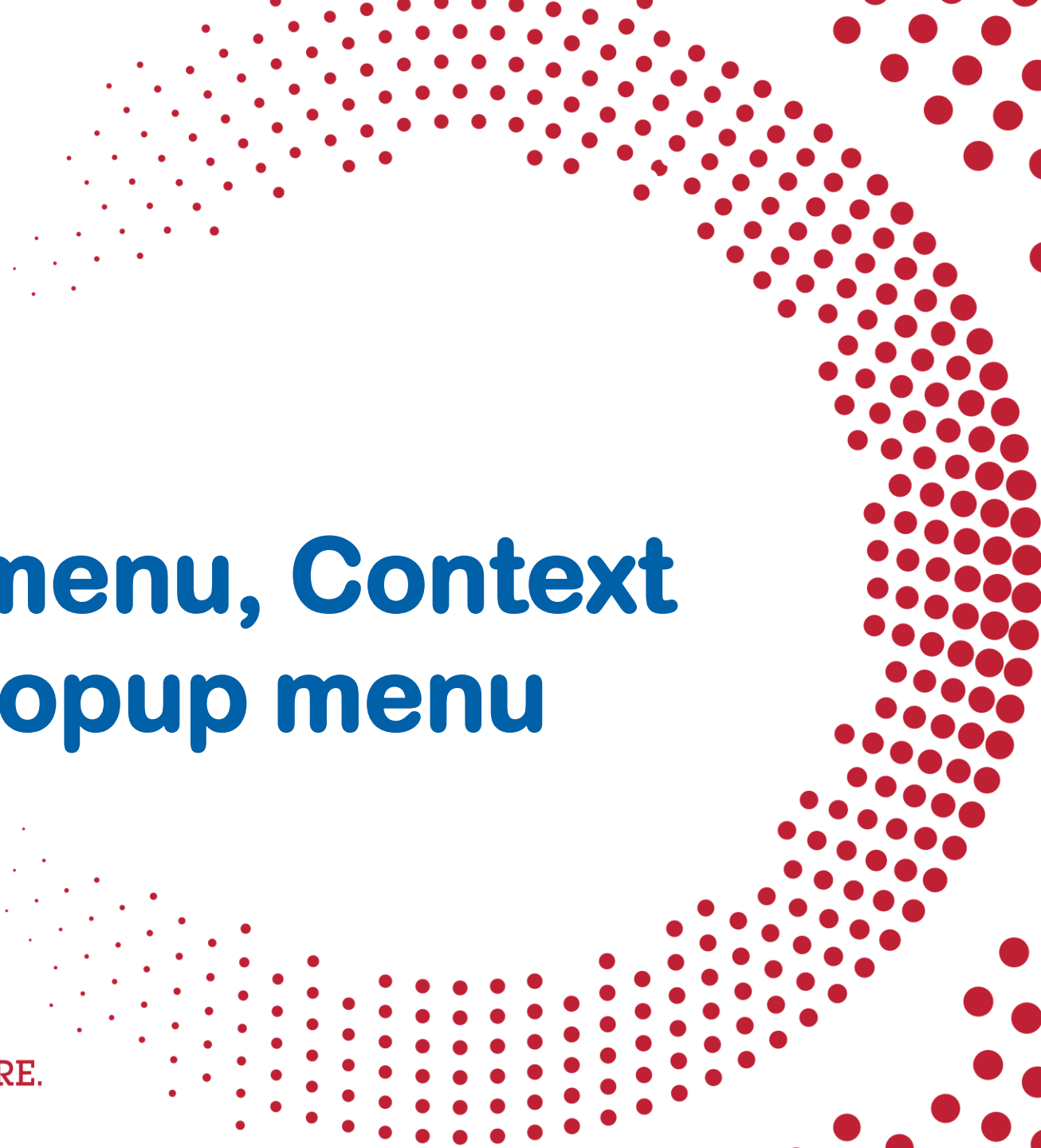
```
// The view used to make the snackbar.  
// This should be contained within the view hierarchy you want to display the  
// snackbar. Generally it can be the view that was interacted with to trigger  
// the snackbar, such as a button that was clicked, or a card that was swiped.  
val contextView = findViewById<View>(R.id.context_view)  
  
Snackbar.make(contextView, R.string.text_label, Snackbar.LENGTH_SHORT)  
    .show()
```

Adding an action

To add an action, use the `setAction` method on the object returned from `make`. Snackbars are automatically dismissed when the action is clicked.

To show a snackbar with a message and an action:

```
Snackbar.make(contextView, R.string.text_label, Snackbar.LENGTH_LONG)
    .setAction(R.string.action_text) {
        // Responds to click on the action
    }
    .show()
```

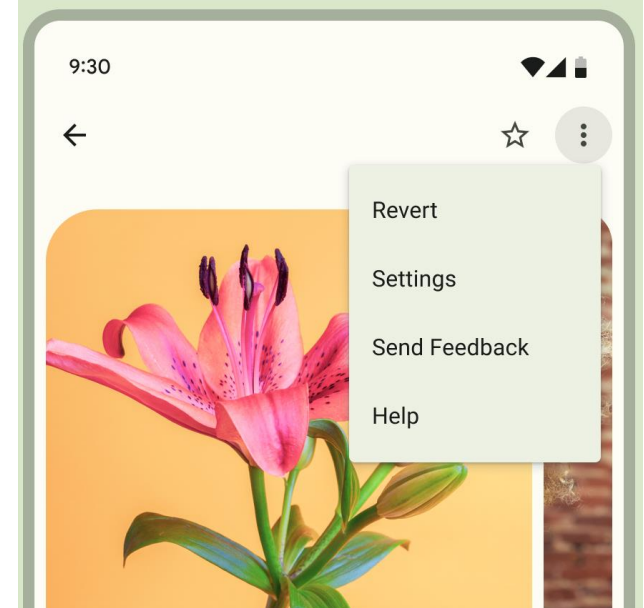


Menus: **Option menu, Context** **menu, Popup menu**

ONE LOVE. ONE FUTURE.

Menu in Android application

- Menus are a common user interface component in many types of apps.
- Three fundamental types:
 - Options menu and app bar
 - Context menu and contextual action mode
 - Popup menu



Define a menu in XML

Android provides a standard XML format to define menu items. You can then inflate the menu resource in your activity or fragment.

Using a menu resource is good practice for the following reasons:

- It's easier to visualize the menu structure in XML.
- It separates the content for the menu from your app's behavioral code.
- It lets you create alternative menu configurations for different platform versions, screen sizes, and other configurations.

Define a menu in XML

To define a menu, create an XML file inside your project's **res/menu/** directory and build the menu with the following elements:

<menu>

Defines a Menu, which is a container for menu items. A **<menu>** element must be the root node for the file, and it can hold one or more **<item>** and **<group>** elements.

<item>

Creates a MenuItem, which represents a single item in a menu. This element can contain a nested **<menu>** element to create a submenu.

<group>

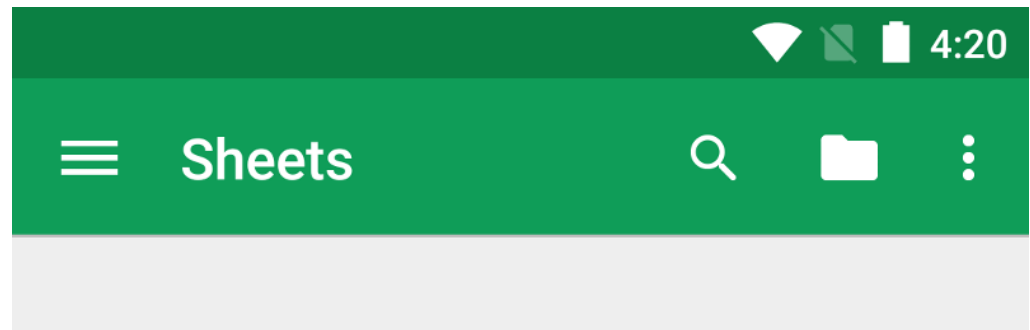
An optional, invisible container for **<item>** elements. It lets you categorize menu items so they share properties, such as active state and visibility. For more information, see the Create a menu group section.

Example menu XML

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">
    <item android:id="@+id/new_game"
        android:icon="@drawable/ic_new_game"
        android:title="@string/new_game"
        app:showAsAction="ifRoom"/>
    <item android:id="@+id/help"
        android:icon="@drawable/ic_help"
        android:title="@string/help" />
</menu>
```

Create an option menu

- The options menu includes actions and other options that are relevant to the current activity context, such as "Search," "Compose email," and "Settings."
- Can be declared from Activity subclass or a Fragment subclass. If both activity and fragments declare items for the options menu, the items are combined in the UI.
- To specify the options menu for an activity or fragments, override **onCreateOptionsMenu()**. In this method, menu resource, defined in XML, can be inflated into the Menu provided in the callback.
- When a menu item is selected, the system calls activity's **onOptionsItemSelected()** method.



Option menu example

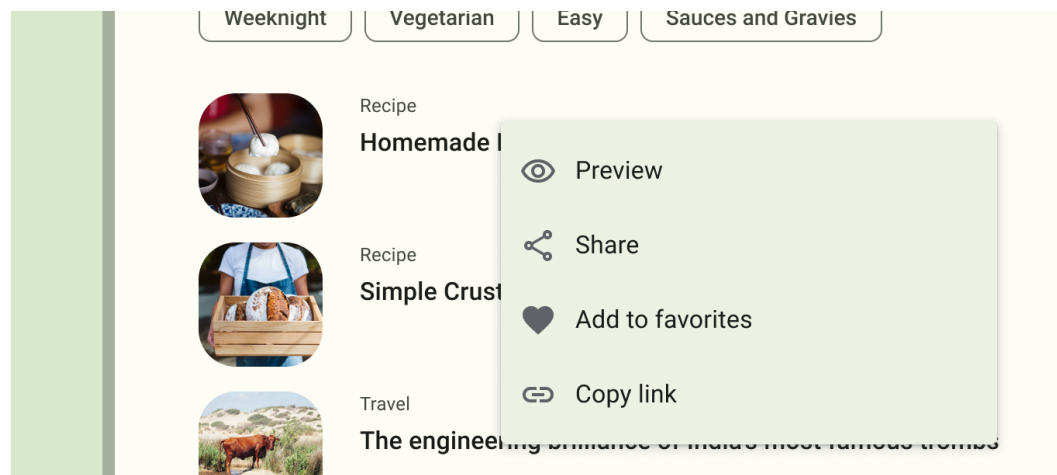
```
override fun onCreateOptionsMenu(menu: Menu): Boolean {
    val inflater: MenuInflater = menuInflater
    inflater.inflate(R.menu.game_menu, menu)
    return true
}

override fun onOptionsItemSelected(item: MenuItem): Boolean {
    // Handle item selection.
    return when (item.itemId) {
        R.id.new_game -> {
            newGame()
            true
        }
        R.id.help -> {
            showHelp()
            true
        }
        else -> super.onOptionsItemSelected(item)
    }
}
```

Create a contextual menu

A contextual menu offers actions that affect a specific item or context frame in the UI.

- Floating context menu: A menu appears as a floating list of menu items, similar to a dialog, when the user performs a touch & hold on a view that declares support for a context menu.
- Contextual action mode: This mode is a system implementation of ActionMode that displays a contextual action bar.



Create a floating context menu

- Register the View the context menu is associated with by calling **registerForContextMenu()** and passing it the View.
- Implement the **onCreateContextMenu()** method in your Activity or Fragment.
- Implement **onContextItemSelected()**. When the user selects a menu item, the system calls this method to perform the appropriate action.

Floating context menu example

```
override fun onCreateContextMenu(menu: ContextMenu, v: View,  
                                menuInfo: ContextMenu.ContextMenuInfo) {  
    super.onCreateContextMenu(menu, v, menuInfo)  
    val inflater: MenuInflater = menuInflater  
    inflater.inflate(R.menu.context_menu, menu)  
}  
  
override fun onContextItemSelected(item: MenuItem): Boolean {  
    val info = item.menuInfo as AdapterView.AdapterContextMenuInfo  
    return when (item.itemId) {  
        R.id.edit -> {  
            editNote(info.id)  
            true  
        }  
        R.id.delete -> {  
            deleteNote(info.id)  
            true  
        }  
        else -> super.onContextItemSelected(item)  
    }  
}
```

Use the contextual action mode

- The contextual action mode is a system implementation of `ActionMode` that focuses user interaction toward performing contextual actions.
- When a user enables this mode by selecting an item, a contextual action bar appears at the top of the screen to present actions the user can perform on the selected items.
- While this mode is enabled, the user can select multiple items, if your app supports that, and can deselect items and continue to navigate within the activity.
- The action mode is disabled and the contextual action bar disappears when the user deselects all items, taps the Back button, or taps the Done action on the left side of the bar.

Use the contextual action mode

- For views that provide contextual actions, you usually invoke the contextual action mode when one or both of these two events occurs:
 - The user performs a touch & hold on the view.
 - The user selects a checkbox or similar UI component within the view.
- How your app invokes the contextual action mode and defines the behavior for each action depends on your design. There are two designs:
 - For contextual actions on individual, arbitrary views.
 - For batch contextual actions on groups of items in a RecyclerView, letting the user select multiple items and perform an action on them all.

Contextual action mode example

- Implement the **ActionMode.Callback** interface, specify the actions for the contextual action bar, respond to click events on action items, and handle other lifecycle events for the action mode.

```
private val actionModeCallback = object : ActionMode.Callback {  
    override fun onCreateActionMode(mode: ActionMode, menu: Menu): Boolean {  
        val inflater: MenuInflater = mode.menuInflater  
        inflater.inflate(R.menu.test_menu, menu)  
        return true  
    }  
    override fun onPrepareActionMode(mode: ActionMode, menu: Menu): Boolean {  
        return false // Return false if nothing is done  
    }  
    override fun onActionItemClicked(mode: ActionMode, item: MenuItem): Boolean {  
        return when (item.itemId) {  
            R.id.action_share -> {  
                mode.finish() // Action picked, so close the CAB.  
                true  
            }  
            R.id.action_delete -> {  
                mode.finish() // Action picked, so close the CAB.  
                true  
            }  
            else -> false  
        }  
    }  
    override fun onDestroyActionMode(mode: ActionMode) {  
        actionMode = null  
    }  
}
```



Contextual action mode example

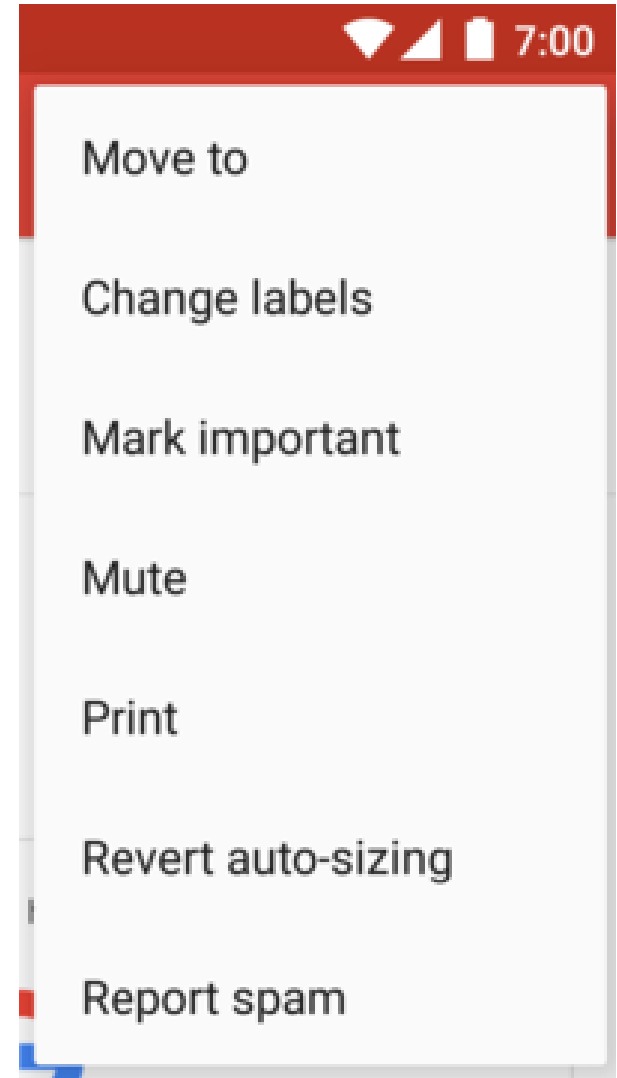
- Call **startActionMode()** to show the bar, such as when the user performs a touch & hold on the view.

```
findViewById<ImageView>(R.id.image_view).setOnLongClickListener {  
    when (actionMode) {  
        null -> {  
            actionMode = this.startSupportActionMode(actionModeCallback)  
            true  
        }  
        else -> false  
    }  
}
```

Create a popup menu

A PopupMenu is a modal menu anchored to a View. It appears below the anchor view if there is room, or above the view otherwise.

- Providing an overflow-style menu for actions that relate to specific content.
- Providing a second part of a command sentence, such as a button marked Add that produces a popup menu with different Add options.
- Providing a menu similar to a Spinner that doesn't retain a persistent selection.



Create a popup menu

If you define your menu in XML, here's how you can show the popup menu:

- Instantiate a **PopupMenu** with its constructor, which takes the current app **Context** and the **View** to which the menu is anchored.
- Use **MenuInflater** to inflate your menu resource into the **Menu** object returned by **PopupMenu getMenu()**.
- Call **PopupMenu.show()**.
- Implement the **PopupMenu.OnMenuItemClickListener** interface and register it with **PopupMenu** by calling **setOnMenuItemClickListener()**. When the user selects an item, the system calls the **onMenuItemClick()** callback.

Popup menu example

```
fun showMenu(v: View) {
    PopupMenu(this, v).apply {
        setOnMenuItemClickListener(this@SecondActivity)
        inflate(R.menu.test_menu)
        show()
    }
}

override fun onMenuItemClick(item: MenuItem): Boolean {
    return when (item.itemId) {
        R.id.action_share -> {
            share(item)
            true
        }
        R.id.action_delete -> {
            delete(item)
            true
        }
        else -> false
    }
}
```



Cơ bản về Kotlin Coroutines Áp dụng trong ứng dụng Android

ONE LOVE. ONE FUTURE.

Nội dung

- Giới thiệu chung
- Ví dụ đầu tiên
- Khái niệm hàm suspend
- Khởi tạo coroutine
- Chờ và hủy coroutine
- Async và await
- Coroutine context
- Coroutine scope

Giới thiệu về coroutines

- Coroutine là cơ chế cho phép thực hiện các đoạn mã nguồn đồng thời và xử lý kết quả bất đồng bộ.
- Cách sử dụng đơn giản và hiệu quả hơn so với các kỹ thuật truyền thống như Thread, AsyncTask, Callback.
- Sử dụng ít bộ nhớ hơn
- Cho phép chuyển đổi linh hoạt giữa các ngữ cảnh (chạy nền, cập nhật giao diện)
- Được sử dụng để thực hiện kết nối CSDL (ROOM), kết nối Internet.

Ví dụ đầu tiên

- Để tạo và chạy coroutine cần thêm các thư viện vào project.
- Đối với project Kotlin

```
implementation("org.jetbrains.kotlinx:kotlinx-coroutines-core-jvm:1.5.0")
```

- Đối với project Android

```
implementation 'org.jetbrains.kotlinx:kotlinx-coroutines-core:1.2.1'
```

```
implementation 'org.jetbrains.kotlinx:kotlinx-coroutines-android:1.1.1'
```

Ví dụ đầu tiên

Tạo file **test.kt** và chạy đoạn mã sau:

```
import kotlinx.coroutines.*

fun main() = runBlocking {
    launch {
        delay(5000L)
        println("World")
    }

    println("Hello")
}
```

Kết quả thực hiện:

```
Hello
World
```

=> Từ “World” xuất hiện sau từ “Hello” 5 giây.

Tạo file **test.kt** và chạy đoạn mã sau:

```
import kotlinx.coroutines.*

fun main() = runBlocking {
    launch {
        delay(5000L)
        println("World")
    }

    println("Hello")
}
```

Giải thích:

launch – tạo và chạy một coroutine mới đồng thời với phần còn lại của mã nguồn

delay – hàm suspend dừng coroutine trong một khoảng thời gian, việc tạm dừng coroutine không làm dừng luồng, cho phép các coroutine khác vẫn có thể hoạt động

runBlocking – được dùng để kết nối mã nguồn không dùng coroutine và mã nguồn sử dụng coroutine, tạo và chạy một coroutine, hàm trả về kết quả khi các coroutine bên trong kết thúc

Cập nhật file **test.kt** như sau:

```
import kotlinx.coroutines.*

fun main() = runBlocking {
    launch { doWorld() }
    println("Hello")
}

suspend fun doWorld() {
    delay(5000L)
    println("World")
}
```

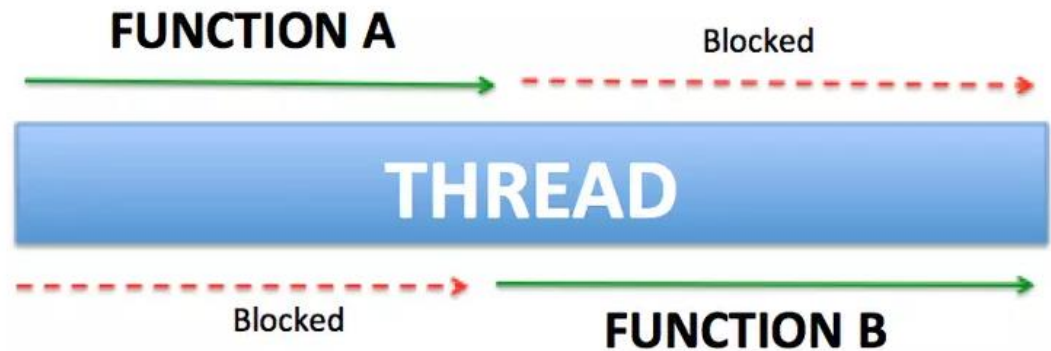
doWorld() là ví dụ về hàm suspend được gọi trong các coroutines, nội dung hàm có thể gọi các hàm suspend khác cho phép tạm dừng coroutine.

Khái niệm hàm suspend

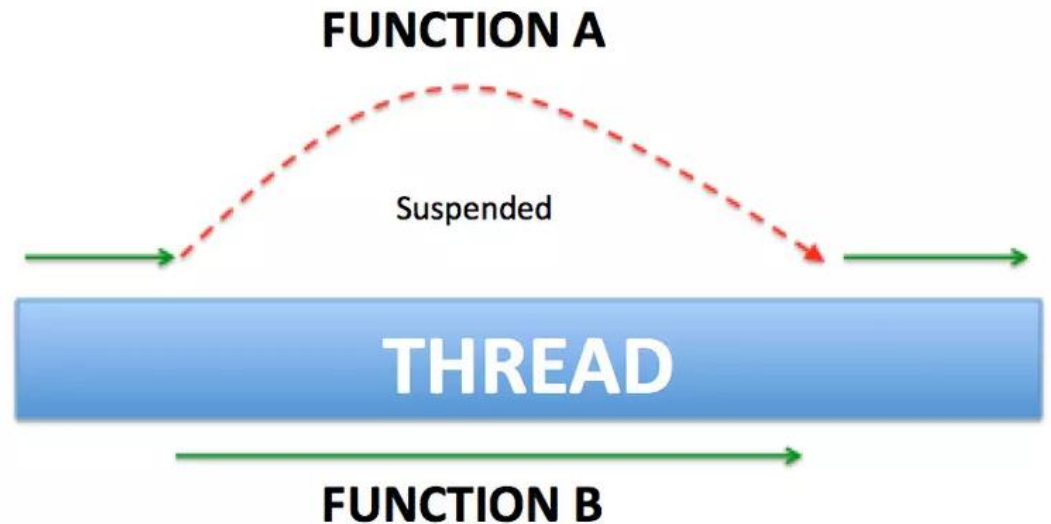
- Là hàm được sử dụng trong các coroutines hoặc trong các hàm suspend khác.
- Có khả năng tạm dừng việc thực thi và tiếp tục khi cần thiết.

Khái niệm hàm suspend

Hàm block: dừng luồng cho đến khi kết thúc hàm, các hàm khác phải chờ.



Hàm suspend: nếu chưa có kết quả thì sẽ tạm dừng nhưng không dừng luồng, cho phép các hàm khác không phải chờ.



Cách thức khởi tạo coroutine

- Để tạo và chạy các coroutine, có thể sử dụng các hàm
- **launch()** – khởi tạo coroutine thực hiện công việc mà không trả về kết quả
 - **runBlocking()** – khởi tạo coroutine và đợi cho đến khi hoàn tất
 - **async()** – khởi tạo coroutine thực hiện công việc và có trả về kết quả

Hàm launch()

```
fun CoroutineScope.launch(  
    context: CoroutineContext = EmptyCoroutineContext,  
    start: CoroutineStart = CoroutineStart.DEFAULT,  
    block: suspend CoroutineScope.() -> Unit  
): Job
```

- Tạo và chạy coroutine mới mà không block luồng hiện tại và trả về kết quả là đối tượng tham chiếu đến coroutine thuộc lớp **Job**. Quản lý coroutine thông qua đối tượng này.
- Ngữ cảnh thực thi được kế thừa từ **CoroutineScope**, có thể được bổ sung bằng tham số **context**.
- Mặc định coroutine được xếp lịch để thực hiện ngay lập tức, hoặc theo các lựa chọn khác bằng tham số **start**.

Ví dụ

```
launch {  
    delay(2000L)  
    println("Third message!")  
}  
launch {  
    delay(1000L)  
    println("Second message!")  
}  
println("First message!")
```

Kết quả khi chạy:

- “First message!” được hiển thị đầu tiên.
- “Second message!” hiển thị sau 1 giây.
- “Third message!” hiển thị sau 2 giây.

Ví dụ

```
val job1 = lifecycleScope.launch(Dispatchers.IO) {  
    // Thực hiện kết nối Internet  
}  
  
val job2 = lifecycleScope.launch(Dispatchers.Main) {  
    // Cập nhật giao diện  
}
```

- **job1** chạy ở luồng background, thực hiện kết nối CSDL, kết nối Internet
- **job2** chạy ở luồng giao diện, thực hiện cập nhật giao diện
- **lifecycleScope** là phạm vi gắn với activity, fragment

Hàm runBlocking()

```
expect fun <T> runBlocking(  
    context: CoroutineContext = EmptyCoroutineContext,  
    block: suspend CoroutineScope.() -> T  
) : T
```

- Tạo và chạy coroutine mới và dừng luồng cho đến khi hoàn thành.
- Hàm được thiết kế để chuyển đổi giữa kiểu mã nguồn blocking và mã nguồn suspending.
- Được sử dụng chủ yếu trong hàm **main()** với mục đích thử nghiệm.

Ví dụ

```
fun main() {  
    runBlocking {  
        delay(2000L)  
        println("First message!")  
    }  
    println("Second message!")  
}
```

Kết quả khi chạy:

- “First message!” hiển thị sau 2 giây.
- “Second message!” hiển thị sau cùng.

Chờ và hủy coroutine đang chạy

- Hàm **join()** – chờ coroutine hoàn thành
- Hàm **cancel()** – hủy thực hiện coroutine
- Hàm **withTimeout()** – chạy coroutine trong khoảng thời gian giới hạn

Chờ coroutine hoàn thành

- Sử dụng hàm **join()** của lớp **Job**

Ví dụ: **job1.join()** sẽ tạm dừng **job2** cho đến khi **job1** hoàn thành

```
runBlocking {  
    val job1 = launch {  
        repeat(10) {  
            delay(1000L)  
            println("Job1: ${it + 1}")  
        }  
    }  
    val job2 = launch {  
        println("Waiting for job1 complete...")  
        job1.join()  
        repeat(10) {  
            delay(1000L)  
            println("Job2: ${it + 1}")  
        }  
    }  
}
```

Hủy coroutine đang thực hiện

- Sử dụng hàm **cancel()** của lớp **Job**, **CoroutineScope** hoặc **CoroutineContext**

```
fun CoroutineScope.cancel(cause: CancellationException? = null)
fun CoroutineScope.cancel(message: String, cause: Throwable? = null)
fun CoroutineContext.cancel(cause: CancellationException? = null)
fun Job.cancel(message: String, cause: Throwable? = null)
```

Hủy các coroutine trong một phạm vi, ngữ cảnh hoặc một coroutine cụ thể. Các tham số kèm theo cho biết nguyên nhân việc hủy coroutine.

Chú ý: hàm **cancel()** không hủy được coroutine chạy với ngữ cảnh **NonCancellable**

Ví dụ

```
runBlocking {  
    val job1 = launch {  
        repeat(10) {  
            delay(1000L)  
            println("Job1: ${it + 1}")  
        }  
    }  
  
    val job2 = launch {  
        println("Waiting for 5 seconds then cancel Job1")  
        delay(5000L)  
        job1.cancel()  
    }  
}
```

job2 hủy **job1** sau 5 giây, **job1** đếm đến 4 thì bị dừng.

Hàm withTimeout()

```
suspend fun <T> withTimeout(timeMillis: Long, block: suspend  
CoroutineScope.() -> T): T
```

- Chạy đoạn chương trình (có thể bị suspend) trong một coroutine với thời gian giới hạn (tính theo ms).
- Nếu hết thời gian mà đoạn chương trình chưa hoàn thành thì tạo exception thuộc kiểu `TimeoutCancellationException`.

Ví dụ

```
runBlocking {  
    withTimeout(5000L) {  
        repeat(10) {  
            delay(800L)  
            println("Counting: ${it + 1}")  
        }  
    }  
}
```

Kết quả thực hiện

Counting: 1

Counting: 2

Counting: 3

Counting: 4

Counting: 5

Counting: 6

Exception in thread "main" kotlinx.coroutines.TimeoutCancellationException: Timed out waiting for 5000 ms

Hàm `async()`

```
fun <T> CoroutineScope.async(  
    context: CoroutineContext = EmptyCoroutineContext,  
    start: CoroutineStart = CoroutineStart.DEFAULT,  
    block: suspend CoroutineScope.() -> T  
) : Deferred<T>
```

- Được sử dụng để tạo và chạy một coroutine mới với các tham số tương tự hàm **launch()**
- Trả về kết quả trong tương lai của coroutine trong một đối tượng thuộc lớp **Deferred** (là lớp con của lớp **Job**)
- Kết quả của coroutine có được khi hoàn thành và được trả về bởi hàm **await()**

Hàm await()

- Đợi cho đến khi có kết quả của đối tượng **Deferred** (coroutine được tạo bởi `async` hoàn thành)

```
runBlocking {  
    val value1: Deferred<Int> = async {  
        println("Compute value 1")  
        delay(3000L)  
        println("Return value 1")  
        return@async Random.nextInt(10, 100)  
    }  
  
    val value2: Deferred<Int> = async {  
        println("Compute value 2")  
        delay(1000L)  
        println("Return value 2")  
        return@async Random.nextInt(10, 100)  
    }  
  
    println("Value 1: ${value1.await()}")  
    println("Value 2: ${value2.await()}")  
}
```

Kết quả khi chạy:

```
Compute value 1  
Compute value 2  
Return value 2  
Return value 1  
Value 1: 57  
Value 2: 84
```

CoroutineContext

- Các coroutine luôn hoạt động trong ngữ cảnh được mô tả bởi giá trị thuộc kiểu CoroutineContext.
- Các thành phần chính của một CoroutineContext
 - Job – nắm giữ thông tin về lifecycle của coroutine
 - Dispatcher – quyết định luồng (thread) nào mà coroutine hoạt động trên đó
- Các loại dispatcher
 - Dispatchers.Main – chạy trên luồng giao diện
 - Dispatchers.IO – chạy trên luồng background
 - Dispatchers.Defaults – chạy trên luồng background
 - newSingleThreadContext("thread_name") – chạy trên luồng mới với tên là **thread_name**

Ví dụ

```
runBlocking {  
    launch { // context of the parent, main runBlocking coroutine  
        println("main runBlocking      : I'm working in thread ${Thread.currentThread().name}")  
    }  
    launch(Dispatchers.Default) { // will get dispatched to DefaultDispatcher  
        println("Default              : I'm working in thread ${Thread.currentThread().name}")  
    }  
    launch(Dispatchers.IO) { // will get dispatched to DefaultDispatcher  
        println("IO                  : I'm working in thread ${Thread.currentThread().name}")  
    }  
    launch(newSingleThreadContext("MyOwnThread")) { // will get its own new thread  
        println("newSingleThreadContext : I'm working in thread ${Thread.currentThread().name}")  
    }  
}
```

Kết quả thực hiện:

Default : I'm working in thread DefaultDispatcher-worker-1
IO : I'm working in thread DefaultDispatcher-worker-2
newSingleThreadContext : I'm working in thread MyOwnThread
main runBlocking : I'm working in thread main

Chuyển đổi giữa các ngữ cảnh

Sử dụng hàm `withContext()`

```
runBlocking {  
    println("Started in ${Thread.currentThread().name}")  
    withContext(Dispatchers.IO) {  
        println("Changed to ${Thread.currentThread().name}")  
    }  
    println("Back to ${Thread.currentThread().name}")  
}
```

Kết quả thực hiện:

Started in main

Changed to DefaultDispatcher-worker-1

Back to main

- Là phạm vi mà các coroutine hoạt động, gắn liền với lifecycle của đối tượng như activity, fragment.
- Khi lifecycle của các đối tượng này kết thúc thì các coroutines đang chạy trong phạm vi tương ứng cũng bị kết thúc.
- Các kiểu phạm vi:
 - CoroutineScope – interface cơ bản, tạo scope với context tùy chọn
 - GlobalScope – được sử dụng để khởi tạo các coroutines bậc cao hoạt động suốt vòng đời của ứng dụng
 - MainScope – tạo scope với context là Main thread
 - lifecycleScope – scope gắn với Activity, Fragment
 - viewModelScope – scope gắn với ViewModel

Ví dụ

```
val scope = CoroutineScope(Dispatchers.Default)
runBlocking(Dispatchers.Default) {
    scope.launch {
        repeat(10) {
            delay(1000L)
            println("Job1: ${it + 1}")
        }
    }
    scope.launch {
        repeat(10) {
            delay(800L)
            println("Job2: ${it + 1}")
        }
    }

    delay(5000L)
    scope.cancel()
}
```



Persistence: Files & Preferences SQL Databases

ONE LOVE. ONE FUTURE.

Android Files

Persistence is a strategy that allows the reusing of volatile objects and other data items by storing them into a permanent storage system such as disk files and databases.

File IO management in Android includes –among others- the familiar IO Java classes: Streams, Scanner, PrintWriter, and so on.

Permanent files can be stored *internally* in the device's main memory (usually small, but not volatile) or *externally* in the much larger SD card.

Files stored in the device's memory, share space with other application's resources such as code, icons, pictures, music, etc.

Internal files are called: **Resource Files** or **Embedded Files**.

Exploring Android's File System

Use the emulator's **File Explorer** to see and manage your device's storage structure.

The diagram illustrates the Android File Explorer interface, showing the file system structure of an emulator. It highlights two main storage areas:

- Internal Main Memory:** This area is represented by the 'data' folder in the File Explorer. It contains subfolders like 'anr', 'app', 'app-asec', 'app-private', 'backup', 'dalvik-cache', and 'files'. The 'files' folder contains a 'notes.txt' file.
- External SD Card:** This area is represented by the 'mnt' folder in the File Explorer. It contains subfolders like 'asec', 'obb', and 'sdcard'. The 'sdcard' folder contains various files, including 'Amarcord.mp3', 'Android', 'Bailables.mp3', 'Bea-BW-Picture.jpg', 'Besame Mucho.mp3', 'Brazil_Bahia.mp3', 'Cancin India.m4a', 'Cinema Paradiso (Theme).mp3', 'DCIM', 'Download', 'IMG_20101209_154654.jpg', and 'IMG_20110110_183452.jpg'.

Arrows indicate the mapping between the File Explorer folders and the physical storage components. A red box highlights the 'data' folder in the left pane, and another red box highlights the 'mnt' folder. A SanDisk 2.0GB microSD card is shown next to the 'External SD Card' label.

Choosing a Persistent Environment

Your permanent data storage destination is usually determined by parameters such as:

- size (**small**/**large**),
- location (**internal**/**external**),
- accessibility (**private**/**public**).

Depending of your situation the following options are available:

1. Shared Preferences

Store private primitive data in *key-value* pairs.

2. Internal Storage

Store private data on the device's main memory.

3. External Storage

Store public data on the shared external storage.

4. SQLite Databases

Store structured data in a private/public database.

5. Network Connection

Store data on the web.

Shared Preferences

SharedPreferences files are good for handling a handful of Items. Data in this type of container is saved as **<Key, Value>** pairs where the *key* is a string and its associated *value* must be a primitive data type.

KEY	VALUE

This class is functionally similar to Java Maps, however; unlike Maps they are *permanent*.

Data is stored in the device's internal main memory.

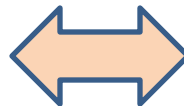
PREFERENCES are typically used to keep state information and shared data among several activities of an application.

Shared Preferences

- Using Preferences API calls
- Each of the Preference mutator methods carries a typed-value content that can be manipulated by an editor that allows putXxx... and getXxx... commands to place data in and out of the Preference container.

- Xxx = { Long, Int, Double, Boolean, String }

Preference Container	
Key	Value



E
D
I
T
O
R

.getXxx(key_n)
.getAll()
.getStringSet()
...

.putXxx(key_n, value_n)

.remove(key_n)
.clear()
.commit()
.apply()

Example: Shared Preferences

In this example the user selects a preferred 'color' and 'number'. Both values are stored in a SharedPreferences file.

```
fun usingPreferences() {  
    // Save data in a SharedPreferences container  
    // We need an Editor object to make preference changes.  
    ① → val myPrefs = getSharedPreferences("my_preferred_choices", MODE_PRIVATE)  
    val editor: SharedPreferences.Editor = myPrefs.edit()  
    ② → editor.putString("chosenColor", "RED")  
    editor.putInt("chosenNumber", 7)  
    editor.apply()  
  
    // retrieving data from SharedPreferences container (apply default if needed)  
    ③ → val favoriteColor = myPrefs.getString("chosenColor", "BLACK");  
    val favoriteNumber = myPrefs.getInt("chosenNumber", 11)  
}
```

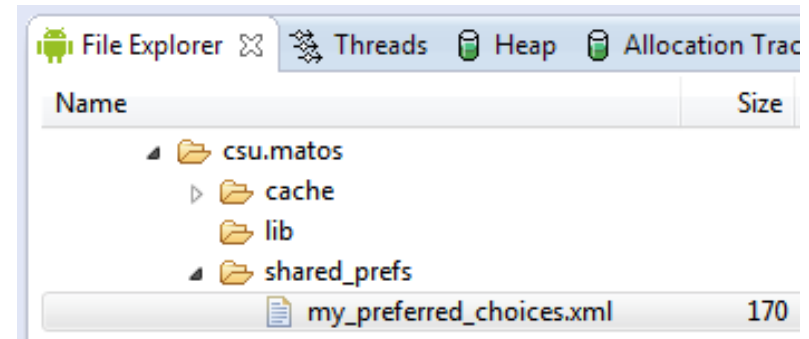
Shared Preferences. Example - Comments

1. The method `getSharedPreferences(...)` creates (or retrieves) a table called *my_preferred_choices* file, using the default **MODE_PRIVATE** access. Under this access mode only the calling application can operate on the file.
2. A `SharedPreferences` editor is needed to make any changes on the file. For instance `editor.putString("chosenColor", "RED")` creates (or updates) the key "chosenColor" and assigns to it the value "RED". All editing actions must be explicitly committed for the file to be updated.
3. The method **getXXX(...)** is used to extract a value for a given key. If no key exists for the supplied name, the method uses the designated default value. For instance `myPrefs.getString("chosenColor", "BLACK")` looks into the file *myPrefs* for the key "chosenColor" to returns its value, however if the key is not found it returns the default value "BLACK".

Shared Preferences. Example - Comments

SharedPreferences containers are saved as XML files in the application's internal memory space. The path to a preference files is
`/data/data/packageName/shared_prefs/filename.`

For instance in this example we have:



If you pull the file from the device, you will see the following

```
<?xml version="1.0" encoding="UTF-8" standalone="true"?>
- <map>
    <string name="favorite_color">#ff0000ff</string>
    <int name="favorite_number" value="101"/>
</map>
```

Files & Preferences

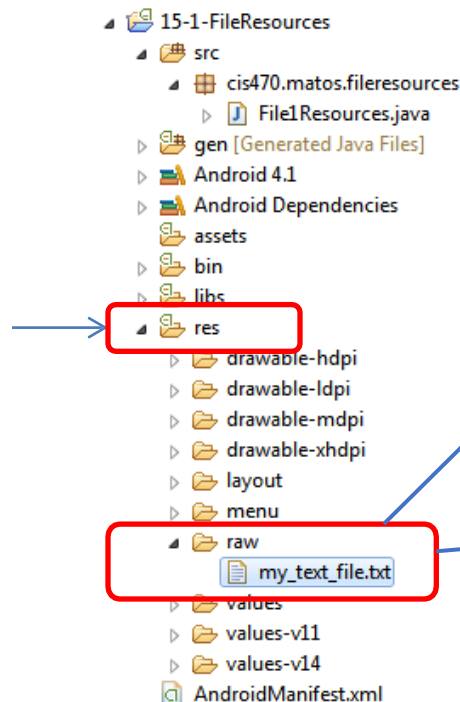
Internal Storage. Reading an Internal Resource File

An Android application may include resource elements such as those in:

res/drawable , **res/raw**, **res/menu**, **res/style**, etc.

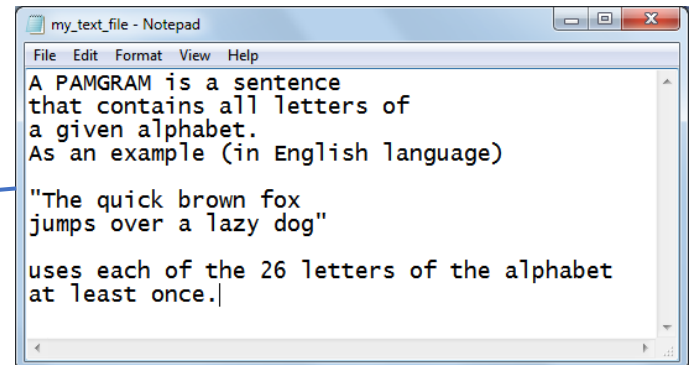
Resources could be accessed through the **.getResources(...)** method. The method's argument is the ID assigned by Android to the element in the R resource file. For example:

```
val inputStream: InputStream = resources.openRawResource(R.raw.my_text_file)
```



If needed create the **res/raw** folder.

Use drag/drop to place the file **my_text_file.txt** in **res** folder. It will be stored in the device's memory as part of the .apk



Example: Reading an Internal Resource File

This app stores a text file in its RESOURCE (**res/raw**) folder.
The embedded raw data is read and displayed in a text box.

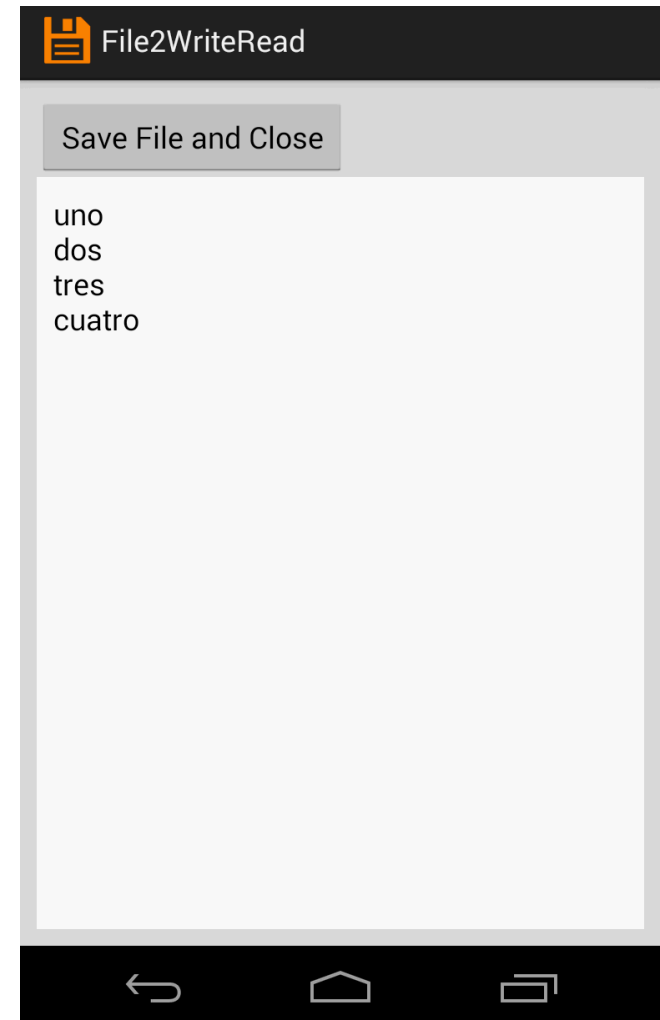
```
val inputStream: InputStream = resources.openRawResource(R.raw.my_text_file)
val reader: InputStreamReader = inputStream.reader()
val content = reader.readText()
reader.close()

binding.textRaw.text = content
```

Example: Reading / Writing an Internal File

In this example an application exposes a GUI on which the user enters a few lines of data. The app collects the input lines and **writes** them to a persistent **internal data file**.

Next time the application is executed, the *internal file* will be **read** and its data will be shown on the UI.



Writing to an internal file

```
val outputStream: OutputStream = openFileOutput("data.txt", MODE_PRIVATE)
// Output file will be saved to /data/data/vn.edu.hust.fileexamples/files/data.txt
val writer: OutputStreamWriter = outputStream.writer()
writer.write(binding.editText.text.toString())
writer.close()
```

Reading from an internal file

```
val inputStream: InputStream = openFileInput("data.txt")
val reader: InputStreamReader = inputStream.reader()
val content: String = reader.readText()
reader.close()

binding.textRaw.text = content
```

Delete an internal file

```
val file: File = File(filesDir.path + "/data.txt")
if (file.exists()) {
    if (file.delete())
        Toast.makeText(this, "File is deleted successfully.",
Toast.LENGTH_LONG).show()
    else
        Toast.makeText(this, "File is deleted failed.", Toast.LENGTH_LONG).show()
} else
    Toast.makeText(this, "File is not existed.", Toast.LENGTH_LONG).show()
```

Copy binary file from resource to internal storage

```
val inputStream = resources.openRawResource(R.raw.test)
val outputStream = openFileOutput("test.txt", MODE_PRIVATE)
val buffer = ByteArray(2048)
while (true) {
    val len = inputStream.read(buffer, 0, buffer.size)
    if (len <= 0)
        break
    outputStream.write(buffer, 0, len)
}
inputStream.close()
outputStream.close()
```

Reading / Writing External SD Files

SD cards offer the advantage of a *much larger capacity* as well as *portability*.

Many devices allow SD cards to be easily removed and reused in another device.

SD cards are ideal for keeping your collection of music, picture, ebooks, and video files.



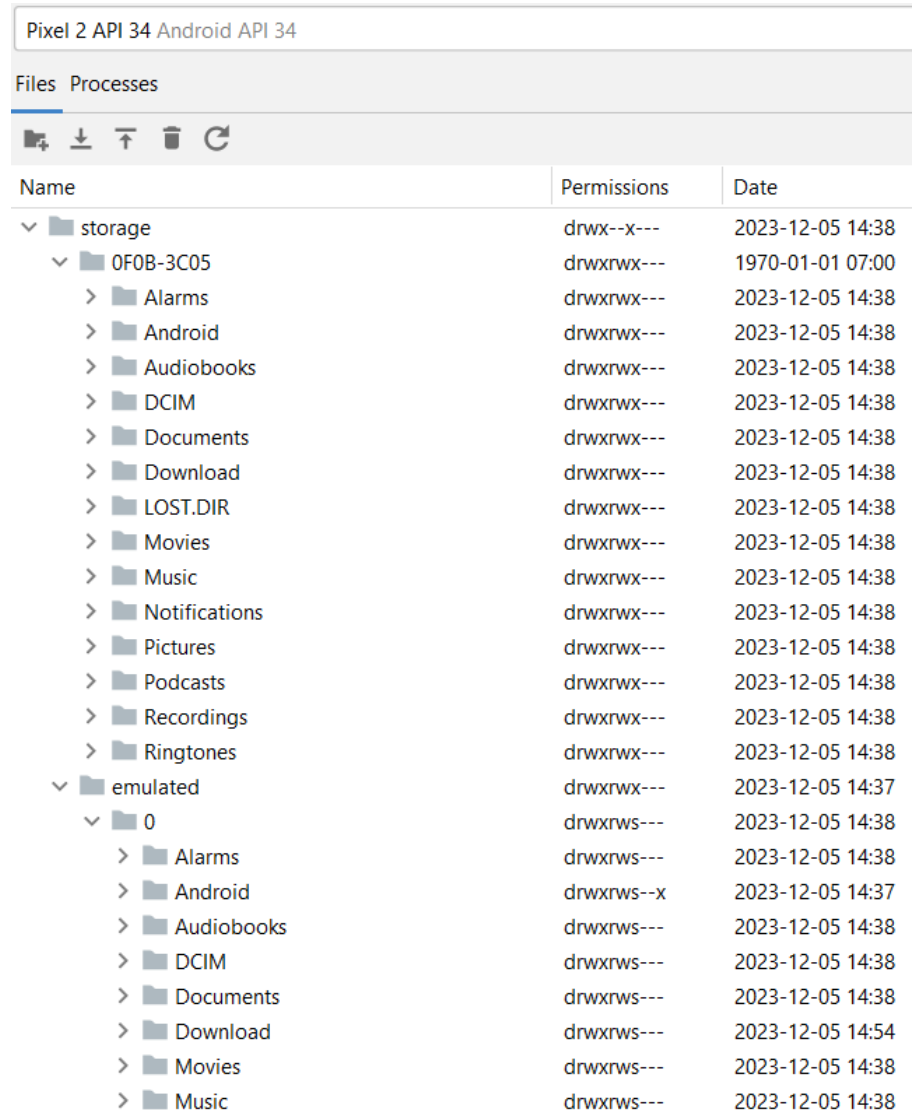
Reading / Writing External SD Files

Use the **Device Explorer** tool to locate files in your physical device (or emulator). External storage is managed differently depend on Android OS version.

=> **HARD CHALLENGE!!!**

2 types of sd card:

- **Emulated**: available for all type of smartphone models, mounted to /sdcard for compatibility
- **Physical**: almost available on old models



Name	Permissions	Date
storage	drwx--x---	2023-12-05 14:38
0F0B-3C05	drwxrwx---	1970-01-01 07:00
Alarms	drwxrwx---	2023-12-05 14:38
Android	drwxrwx---	2023-12-05 14:38
Audiobooks	drwxrwx---	2023-12-05 14:38
DCIM	drwxrwx---	2023-12-05 14:38
Documents	drwxrwx---	2023-12-05 14:38
Download	drwxrwx---	2023-12-05 14:38
LOST.DIR	drwxrwx---	2023-12-05 14:38
Movies	drwxrwx---	2023-12-05 14:38
Music	drwxrwx---	2023-12-05 14:38
Notifications	drwxrwx---	2023-12-05 14:38
Pictures	drwxrwx---	2023-12-05 14:38
Podcasts	drwxrwx---	2023-12-05 14:38
Recordings	drwxrwx---	2023-12-05 14:38
Ringtones	drwxrwx---	2023-12-05 14:38
emulated	drwxrwx---	2023-12-05 14:37
0	drwxrws---	2023-12-05 14:38
Alarms	drwxrws---	2023-12-05 14:38
Android	drwxrws--x	2023-12-05 14:37
Audiobooks	drwxrws---	2023-12-05 14:38
DCIM	drwxrws---	2023-12-05 14:38
Documents	drwxrws---	2023-12-05 14:38
Download	drwxrws---	2023-12-05 14:54
Movies	drwxrws---	2023-12-05 14:38
Music	drwxrws---	2023-12-05 14:38

Reading /Writing External SD Files

Although you may use the specific path to an SD file, such as:

`/sdcard/myfile.txt`

it is a better practice to determine the SD location as suggested below

```
val sdPath = Environment.getExternalStorageDirectory().path
```

WARNING

When you deal with external files you need to request permission to read and write. Add the following clauses to your AndroidManifest.xml (**apply for API level below 33**)

```
<uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
```

Reading / Writing External SD Files

From Android 6.0 (API Level 23), app needs to ask permission from user.

```
if (Build.VERSION.SDK_INT > Build.VERSION_CODES.M) {  
    if (checkSelfPermission(Manifest.permission.WRITE_EXTERNAL_STORAGE) !=  
        PackageManager.PERMISSION_GRANTED) {  
        requestPermissions(arrayOf(Manifest.permission.WRITE_EXTERNAL_STORAGE), 1234)  
    }  
}
```

```
override fun onRequestPermissionsResult(requestCode: Int, permissions: Array<out  
String>, grantResults: IntArray) {  
    super.onRequestPermissionsResult(requestCode, permissions, grantResults)  
    if (requestCode == 1234) {  
        if (grantResults[0] == PackageManager.PERMISSION_GRANTED) {  
            Log.v("TAG", "Permission granted!")  
        } else  
            Log.v("TAG", "Permission denied!")  
    }  
}
```

In Android 10 (API Level 29), use “requestLegacyExternalStorage” to read and write external storage.

Reading / Writing External SD Files

From Android 11 (API Level 30), an app can request All files access from the user by doing the following:

1. Declare the **MANAGE_EXTERNAL_STORAGE** permission in the manifest.
2. Use the **ACTION_MANAGE_ALL_FILES_ACCESS_PERMISSION** intent action to direct users to a system settings page where they can enable the following option for your app: Allow access to manage all files.

To determine whether your app has been granted the **MANAGE_EXTERNAL_STORAGE** permission, call `Environment.isExternalStorageManager()`.

<https://developer.android.com/training/data-storage/manage-all-files>

```
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.R) {  
    if (!Environment.isExternalStorageManager()) {  
        val intent = Intent(Settings.ACTION_MANAGE_ALL_FILES_ACCESS_PERMISSION)  
        startActivity(intent)  
    }  
}
```

Writing to an external file

```
val path = Environment.getExternalStorageDirectory().path
val file = File("${path}/data.txt")
val outputStream: OutputStream = file.outputStream()
val writer: OutputStreamWriter = outputStream.writer()
writer.write(binding.editText.text.toString())
writer.close()
```

Reading from an external file

```
val path = Environment.getExternalStorageDirectory().path
val file = File("${path}/data.txt")
val inputStream: InputStream = file.inputStream()
val reader: InputStreamReader = inputStream.reader()
val content: String = reader.readText()
reader.close()

binding.editText.setText(content)
```

Copy from internal storage to the shared storage

```
val fromPath = filesDir.path + "/data.txt"
val toPath = Environment.getExternalStorageDirectory().path + "/data.txt"
val fromFile = File(fromPath)
val toFile = File(toPath)
fromFile.copyTo(toFile)
```

List all file and folder in the root directory of the shared storage

```
val root = Environment.getExternalStorageDirectory()
for (entry in root.listFiles()) {
    if (entry.isDirectory)
        Log.v("TAG", "Directory")
    else if (entry.isFile)
        Log.v("TAG", "File")
    Log.v("TAG", entry.path)
}
```

Using SQL databases in Andorid

Included into the core Android architecture there is an standalone Database Management System (DBMS) called **SQLite** which can be used to:

Create a database,

Define

SQL tables,

indices,

queries,

views,

triggers

Insert rows,

Delete rows,

Change rows,

Run queries and

Administer a SQLite database file.



Characteristics of SQLite

- Transactional SQL database engine.
- Small footprint (less than 400KBytes)
- Serverless
- Zero-configuration
- The source code for SQLite is in the public domain.
- According to their website, SQLite is the *most widely deployed SQL database engine in the world* .

Reference:

<http://sqlite.org/index.html>

Creating a SQLite database - Method 1

```
SQLiteDatabase.openDatabase( myDbPath,  
                             null,  
                             SQLiteDatabase.CREATE_IF_NECESSARY);
```

If the database does not exist then create a new one. Otherwise, open the existing database according to the flags:

OPEN_READWRITE, OPEN_READONLY, CREATE_IF_NECESSARY .

Parameters

path to database file to open and/or create

factory an optional factory class that is called to instantiate a cursor when query is called, or *null* for default

flags to control database access mode

Returns the newly opened database

Throws *SQLException* if the database cannot be opened

Example1: Creating a SQLite database - Method 1

```
class MainActivity : AppCompatActivity() {  
    lateinit var db: SQLiteDatabase  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
  
        val binding = ActivityMainBinding.inflate(layoutInflater)  
        setContentView(binding.root)  
  
        db = SQLiteDatabase.openDatabase(filesDir.path + "/mydb", null,  
            SQLiteDatabase.CREATE_IF_NECESSARY)  
    }  
  
    override fun onStop() {  
        db.close()  
        super.onStop()  
    }  
}
```

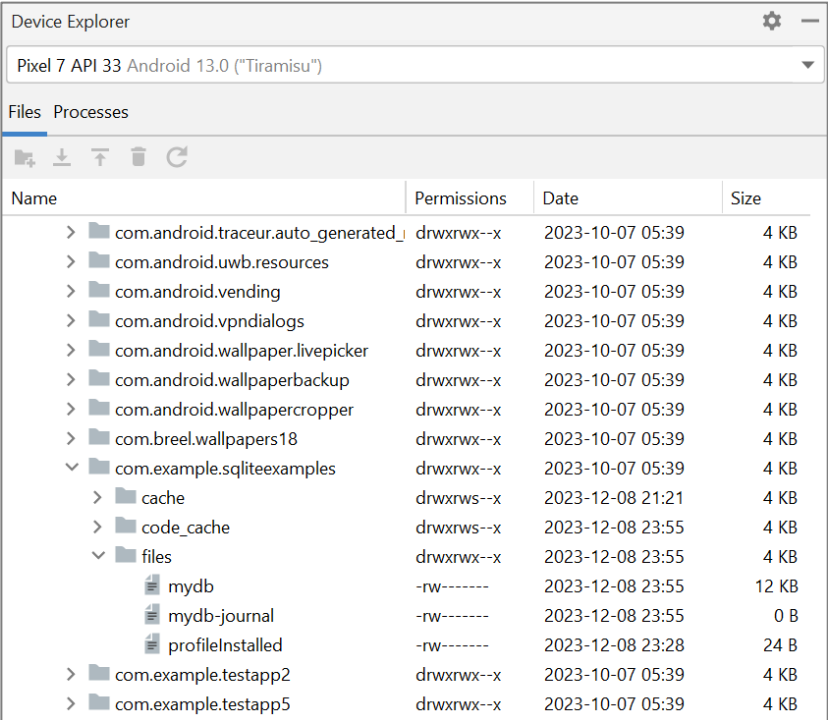
Example1: Creating a SQLite database - Using Memory

SQLite Database is stored using Internal Memory

Path:

`/data/data/com.example.sqliteexamples/files/`

Where: `com.example.sqliteexamples` is the package's name



Name	Permissions	Date	Size
> com.android.traceur.auto_generated	drwxrwx--x	2023-10-07 05:39	4 KB
> com.android.uwb.resources	drwxrwx--x	2023-10-07 05:39	4 KB
> com.android.vending	drwxrwx--x	2023-10-07 05:39	4 KB
> com.android.vpndialogs	drwxrwx--x	2023-10-07 05:39	4 KB
> com.android.wallpaper.livepicker	drwxrwx--x	2023-10-07 05:39	4 KB
> com.android.wallpaperbackup	drwxrwx--x	2023-10-07 05:39	4 KB
> com.android.wallpapercropper	drwxrwx--x	2023-10-07 05:39	4 KB
> com.breel.wallpapers18	drwxrwx--x	2023-10-07 05:39	4 KB
▼ com.example.sqliteexamples	drwxrwx--x	2023-10-07 05:39	4 KB
> cache	drwxrws--x	2023-12-08 21:21	4 KB
> code_cache	drwxrws--x	2023-12-08 23:55	4 KB
▼ files	drwxrwx--x	2023-12-08 23:55	4 KB
mydb	-rw-----	2023-12-08 23:55	12 KB
mydb-journal	-rw-----	2023-12-08 23:55	0 B
profileInstalled	-rw-----	2023-12-08 23:28	24 B
> com.example.testapp2	drwxrwx--x	2023-10-07 05:39	4 KB
> com.example.testapp5	drwxrwx--x	2023-10-07 05:39	4 KB

An Alternative Method: `openOrCreateDatabase`

An alternative way of opening/creating a SQLite database in your local Android's internal data space is given below

```
val db: SQLiteDatabase = openOrCreateDatabase(  
    "myfriendsDB",  
    MODE_PRIVATE,  
    null)
```

Assume app is made in a namespace called `com.example.sqliteexamples`, then the full name of the newly created database file will be:

`/data/data/com.example.sqliteexamples/database/myfriendsDB`

- The file can be accessed by all components of the same application.
- Other **MODE** values: `MODE_WORLD_READABLE`, and `MODE_WORLD_WRITEABLE` were deprecated on API Level 17.
- **null** refers to optional **factory** class parameter (skip for now)

Type of SQL Commands

Once created, the SQLite database is ready for normal operations such as: *creating, altering, dropping resources (tables, indices, triggers, views, queries etc.) or administrating database resources (containers, users, ...).*

Action queries and **Retrieval queries** represent the most common operations against the database.

- A **retrieval query** is typically a *SQL-Select* command in which a table holding a number of fields and rows is produced as an answer to a data request.
- An **action query** usually performs maintenance and administrative tasks such as manipulating tables, users, environment, etc.

Transaction Processing

Transactions are desirable because they help maintaining consistent data and prevent unwanted data losses due to abnormal termination of execution.

In general it is convenient to process **action queries** inside the protective frame of a **database transaction** in which the policy of “*complete success or total failure*” is transparently enforced.

*This notion is called: **atomicity** to reflect that all parts of a method are fused in an indivisible ‘statement’.*

Transaction Processing

The typical Android's way of running transactions on a SQLiteDatabase is illustrated by the following code fragment (Assume **db** is a SQLiteDatabase)

```
db.beginTransaction()  
try {  
    //perform your database operations here ...  
    db.setTransactionSuccessful() //commit your changes  
}  
catch (e: SQLException) {  
    //report problem  
}  
finally {  
    db.endTransaction()  
}
```

The transaction is defined between the methods: *beginTransaction* and *endTransaction*. You need to issue the *setTransactionSuccessful()* call to commit any changes. The absence of it provokes an implicit *rollback* operation; consequently *the database is reset to the state previous to the beginning of the transaction*.

Create and Populate a SQL Table

The **SQL** syntax used for creating and populating a table is illustrated in the following examples

```
create table tblAMIGO(  
    recID integer PRIMARY KEY autoincrement,  
    name text,  
    phone text );
```

```
insert into tblAMIGO(name, phone) values ('AAA', '555-1111' );
```

The *autoincrement* value for *recID* is NOT supplied in the insert statement as it is internally assigned by the DBMS.

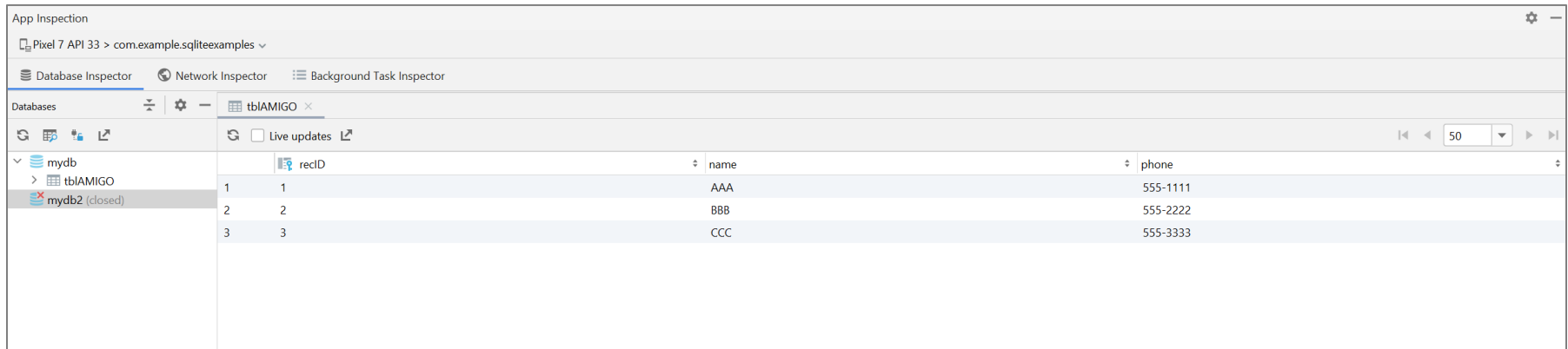
Example 2. Create and Populate a SQL Table

- Our Android app will use the **execSQL(...)** method to manipulate SQL *action queries*. The example below creates a new table called **tblAmigo**.
- The table has three fields: a numeric unique identifier called **recID**, and two string fields representing our friend's **name** and **phone**.
- If a table with such a name exists it is first dropped and then created again.
- Finally three rows are inserted in the table

```
db.execSQL("create table tblAMIGO(" +  
    "recID integer primary key autoincrement," +  
    "name text," +  
    "phone text);")  
db.execSQL("insert into tblAMIGO(name, phone) values('AAA', '555-1111');")  
db.execSQL("insert into tblAMIGO(name, phone) values('BBB', '555-2222');")  
db.execSQL("insert into tblAMIGO(name, phone) values('CCC', '555-3333');")
```

Example 2. Create and Populate a SQL Table

Use **Database Inspector** tool (in **App Inspection**) to monitor the database when app is running.



Example 2. Create and Populate a SQL Table

Comments

1. The field **recID** is defined as the table's **PRIMARY KEY**.
2. The “*autoincrement*” feature guarantees that each new record will be given a unique serial number (0,1,2,...).
3. On par with other SQL systems, SQLite offers the data types: ***text, varchar, integer, float, numeric, date, time, timestamp, blob, boolean.***
3. In general any well-formed DML SQL action command (insert, delete, update, create, drop, alter, etc.) could be framed inside an **execSQL(...)** method call.

Caution:

You should call the **execSQL** method inside of a ***try-catch-finally*** block. Be aware of potential **SQLiteException** conflicts thrown by the method.

Asking Questions - SQL Queries

1. **Retrieval queries** are known as *SQL-select* statements.
2. *Answers* produced by retrieval queries are always held in a *table*.
3. In order to process the resulting table rows, the user should provide a **cursor** device. Cursors allow a *row-at-the-time access* mechanism on SQL tables.

Android-SQLite offers two strategies for phrasing *select* statements: ***rawQueries*** and ***simple queries***. Both return a database *cursor*.

1. **Raw queries** take for input any (syntactically correct) SQL-select statement. The select query could be as complex as needed and involve any number of tables (only a few exceptions such as outer-joins)
2. **Simple queries** are compact *parametized* lookup functions that operate on a single table (for developers who prefer not to use SQL).

SQL Select Statement – Syntax <http://www.sqlite.org/lang.html>

```
select    field1, field2, ... , fieldn
from      table1, table2, ... , tablen
```

```
where     ( restriction-join-conditions )
order by  fieldn1, ..., fieldnm
group by  fieldm1, ... , fieldmk
having    (group-condition)
```

The first two lines are mandatory, the rest is optional.

1. The *select* clause indicates the fields to be included in the answer
2. The *from* clause lists the tables used in obtaining the answer
3. The *where* component states the conditions that records must satisfy in order to be included in the output.
4. *Order by* tells the sorted sequence on which output rows will be presented
5. *Group by* is used to partition the tables and create sub-groups
6. *Having* formulates a condition that sub-groups made by partitioning need to satisfy.

Two Examples of SQL-Select Statements

Example A.

```
SELECT    LastName, cellPhone
FROM      ClientTable
WHERE     state = 'Ohio'
ORDER BY  LastName
```

Example B.

```
SELECT    city, count(*) as TotalClients
FROM      ClientTable
GROUP BY  city
```

Example3. Using a Parameterless RawQuery (version 1)

Consider the following code fragment

```
val c1: Cursor = db.rawQuery("select * from tblAMIGO", null)
```

1. The previous *rawQuery* contains a select-statement that retrieves all the rows (and all the columns) stored in the table `tblAMIGO`. The resulting table is wrapped by a **Cursor** object `c1`.
2. The 'select *' clause instructs SQL to grab all-columns held in a row.
3. Cursor `c1` will be used to traverse the rows of the resulting table.
4. Fetching a row using cursor `c1` requires advancing to the next record in the answer set (cursors are explained a little later in this section).
5. Fields provided by SQL must be bound to local Java variables (soon we will see to that).

Example3. Using a Parametized RawQuery (version 2)

Passing arguments.

Assume we want to count how many friends are there whose name is 'BBB' and their recID > 1. We could use the following solution:

```
val sql = "select count(*) as Total " +  
          "from tblAMIGO " +  
          "where recID > ? and name = ?"  
val args = arrayOf("1", "BBB")  
val cs = db.rawQuery(sql, args)
```

The various symbols '?' in the SQL statement represent positional placeholders. When **.rawQuery()** is called, the system binds each empty placeholder '?' with the supplied **args**-value. Here the first '?' will be replaced by "1" and the second by "BBB".

Example3. Using a Stitched RawQuery (version 3)

As in the previous example, assume we want to count how many friends are there whose name is 'BBB' and their recID > 1. We could use the following solution:

```
val args = arrayOf("1", "BBB")
val sql = "select count(*) as Total " +
    "from tblAMIGO " +
    "where recID > ${args[0]} and name = ${args[1]}"
val cs = db.rawQuery(sql, null)
```

Instead of the symbols '?' acting as placeholder, we conveniently concatenate the necessary data fragments during the assembling of our SQL statement.

SQL Cursors

Cursors are used to gain sequential & random access to tables produced by SQL *select* statements.

Cursors support *one row-at-the-time* operations on a table. Although in some DBMS systems cursors can be used to update the underlying dataset, the SQLite version of cursors is **read-only**.



Cursors include several types of operators, among them:

1. **Positional awareness:** isFirst(), isLast(), isBeforeFirst(), isAfterLast().
2. **Record navigation:** moveToFirst(), moveToLast(), moveToNext(), moveToPrevious(), move(n).
3. **Field extraction:** getInt, getString, getFloat, getBlob, getDouble, etc.
4. **Schema inspection:** getColumnName(), getColumnNames(), getColumnIndex(), getColumnCount(), getCount().

Example 4A. Traversing a Cursor – Simple Case

```
val cs = db.rawQuery("select * from tblAMIGO", null)
cs.moveToFirst()
do {
    val recID = cs.getInt(0)
    val name = cs.getString(1)
    val phoneIndex = cs.getColumnIndex("phone")
    val phone = if (phoneIndex > -1) cs.getString(phoneIndex) else ""
    Log.v("TAG", "$recID - $name - $phone")
} while (cs.moveToNext())
cs.close()
```

1. Prepare a `rawQuery` passing a simple sql statement with no arguments, catch the resulting tuples in cursor `cs`.
2. Move the fetch marker to the absolute position prior to the first row in the file. The valid range of values is $-1 \leq \text{position} \leq \text{count}$.
3. Use **`moveToNext()`** to visit each row in the result set

SQLite Simple Queries - Template Based Queries

- Simple SQLite queries use a *template* oriented schema whose goal is to 'help' non-SQL developers in their process of querying a database.
- This *template* exposes all the components of a basic SQL-select statement.
- Simple queries can *only* retrieve data from a *single table*.
- The method's signature has a fixed sequence of seven arguments representing:
 1. the table name,
 2. the columns to be retrieved,
 3. the search condition (where-clause),
 4. arguments for the where-clause,
 5. the group-by clause,
 6. having-clause, and
 7. the order-by clause.

SQLite Simple Queries - Template Based Queries

The signature of the SQLite simple **.query** method is:

```
db.query (table: String!,  
          columns: Array<out String!>!,  
          selection: String!,  
          selectionArgs: Array<out String!>!,  
          groupBy: String!,  
          having: String!,  
          orderBy: String!)
```

Example6. SQLite Simple Queries

In this example we use the **tblAmigo** table. We are interested in selecting the columns: *recID*, *name*, and *phone*. The condition to be met is that RecID must be greater than 2, and names must begin with 'B' and have three or more letters.

```
val columns = arrayOf("recID", "name", "phone")
val cs = db.query("tblAMIGO", columns,
    "recID > 2 and length(name) >= 3 and name like 'B%",
    null, null, null,
    "recID")
val numRecords = cs.count
```

We enter **null** in each component not supplied to the method. For instance, in this example select-args, having, and group-by are not used.

Example7. SQLite Simple Queries

In this example we will construct a more complex SQL select statement.

We are interested in tallying how many groups of friends whose recID > 3 have the same name. In addition, we want to see 'name' groups having no more than four people each.

A possible SQL-select statement for this query would be something like:

```
select  name, count(*) as TotalSubGroup
  from  tblAMIGO
 where  recID > 3
 group  by name
having  count(*) <= 4;
```

Example7. SQLite Simple Queries

An equivalent Android-SQLite solution using a simple template query follows.

```
val selectColumns = arrayOf("name", "count(*) as TotalSubGroup")
val whereCondition = "recID > ?"
val whereConditionArgs = arrayOf("3")
val groupBy = "name"
val having = "count(*) < 4"
val orderBy = "name"

val cs = db.query("tblAMIGO", selectColumns, whereCondition, whereConditionArgs,
    groupBy, having, orderBy)
```

Example7. SQLite Simple Queries

Observations

1. The *selectColumns* string array contains the output fields. One of them (*name*) is already part of the table, while *TotalSubGroup* is an alias for the computed count of each name sub-group.
2. The symbol **?** in the *whereCondition* is a *place-marker* for a substitution. The value “3” taken from the *whereConditionArgs* is to be injected there.
3. The *groupBy* clause uses ‘*name*’ as a key to create sub-groups of rows with the same *name* value. The *having* clause makes sure we only choose subgroups no larger than four people.

SQL Action Queries

Action queries are the SQL way of performing maintenance operations on tables and database resources. Example of action-queries include: *insert*, *delete*, *update*, *create table*, *drop*, etc.

Examples:

```
insert into tblAmigos  
  values ( 'Macarena',  '555-1234' );
```

```
update tblAmigos  
  set name = 'Maria Macarena'  
  where phone = '555-1234';
```

```
delete from tblAmigos  
  where phone = '555-1234';
```

```
create table Temp ( column1 int, column2 text, column3 date );
```

```
drop table Temp;
```

SQLite Action Queries Using: ExecSQL

Perhaps the simplest Android way to phrase a SQL action query is to ‘stitch’ together the pieces of the SQL statement and give it to the easy to use –but rather limited- ***execSQL(...)*** method.

Unfortunately SQLite ***execSQL*** does **NOT** return any data. Therefore knowing how many records were affected by the action is not possible with this operator. Instead you should use the Android versions describe in the next section.

Android's INSERT, DELETE, UPDATE Operators

- Android provides a number of additional methods to perform *insert*, *delete*, *update* operations.
- They all return some feedback data such as the record ID of a recently inserted row, or number of records affected by the action. This format is recommended as a better alternative than execSQL.

```
public long insert(String table,  
                  String nullColumnHack,  
                  ContentValues values )
```

```
public int update( String table,  
                  ContentValues values,  
                  String whereClause,  
                  String[] whereArgs )
```

```
public int delete( String table,  
                  String whereClause,  
                  String[] whereArgs)
```

ContentValues Class

- This class is used to store a set of **[name, value]** pairs (functionally equivalent to Bundles).
- When used in combination with SQLite, a ContentValues object is just a convenient way of passing a variable number of parameters to the SQLite action functions.
- Like bundles, this class supports a group of put/get methods to move data in/out of the container.

```
val myArgs= ContentValues()  
  
myArgs.put("name", "ABC")  
myArgs.put("phone", "555-7777")
```

myArgs

Key	Value
name	ABC
phone	555-7777

Android's INSERT Operation

```
public long insert(String table, String nullColumnHack, ContentValues values)
```

The method tries to insert a row in a table. The row's column-values are supplied in the map called *values*. If successful, the method returns the **rowID** given to the new record, otherwise -1 is sent back.

Parameters

table	the table on which data is to be inserted
nullColumnHack	Empty and Null are different things. For instance, <i>values</i> could be defined but empty. If the row to be inserted <i>is empty</i> (as in our next example) this column will explicitly be assigned a NULL value (which is OK for the insertion to proceed).
values	Similar to a bundle (<i>name, value</i>) containing the column values for the row that is to be inserted.

Android's INSERT Operation

```
val rowValues = ContentValues()

rowValues.put("name", "ABC")
rowValues.put("phone", "555-1010")
var rowPosition = db.insert("tblAMIGO", null, rowValues)

rowValues.put("name", "DEF")
rowValues.put("phone", "555-2020")
rowPosition = db.insert("tblAMIGO", null, rowValues)

rowValues.clear()

rowPosition = db.insert("tblAMIGO", null, rowValues)

rowPosition = db.insert("tblAMIGO", "name", rowValues)
```

Android's INSERT Operation

Comments

1. A set of **<key, values>** called **rowValues** is created and supplied to the insert() method to be added to tblAmigo. Each *tblAmigo* row consists of the columns: *recID*, *name*, *phone*. Remember that *recID* is an *auto-incremented* field, its actual value is to be determined later by the database when the record is accepted.
2. The newly inserted record returns its rowID (4 in this example)
3. A second record is assembled and sent to the insert() method for insertion in tblAmigo. After it is collocated, it returns its rowID (5 in this example).
4. The rowValues map is reset, therefore rowValues which is not null becomes empty.
5. SQLite rejects attempts to insert an empty record returning rowID -1.
6. The second argument identifies a column in the database that allows NULL values (**NAME** in this case). Now SQL purposely inserts a NULL value on that column (as well as in other fields, except the key **RecId**) and the insertion successfully completes.

Android's UPDATE Operation

```
public int update ( String table, ContentValues values,  
                    String whereClause, String[] whereArgs )
```

The method tries to update row(s) in a table. The SQL **set column=newvalue** clause is supplied in the *values* map in the form of [key,value] pairs. The method returns the number of records affected by the action.

Parameters

table	the table on which data is to be updated
values	Similar to a bundle (<i>name, value</i>) containing the columnName and NewValue for the fields in a row that need to be updated.
whereClause	This is the condition identifying the rows to be updated. For instance "name = ? " where ? Is a placeholder. Passing null updates the entire table.
whereArgs	Data to replace ? placeholders defined in the whereClause.

Android's UPDATE Operation

Example

We want to use the `.update()` method to express the following SQL statement:

```
update tblAmigo set name = 'maria' where (recID > 2 and recID < 7)
```

Here are the steps to make the call using Android's equivalent Update Method

```
val whereArgs = arrayOf("2", "7")
val updateValues = ContentValues()
updateValues.put("name", "Maria")
val recAffected = db.update("tblAMIGO", updateValues,
    "recID > ? and recID < ?", whereArgs)
```

Android's UPDATE Operation

Comments

1. Our **whereArgs** is an array of arguments. Those actual values will replace the placeholders '?' set in the whereClause.
2. The map **updValues** is defined and populated. In our case, once a record is selected for modifications, its "name" field will be changed to the new value "maria".
3. The **db.update()** method attempts to update all records in the given table that satisfy the filtering condition set by the **whereClause**. After completion it returns the number of records affected by the update (0 if it fails).
4. The update **filter** verifies that "*recID > ? and recID < ?*". After the args substitutions are made the new filter becomes: "*recID > 2 and recID < 7*".

Android's DELETE Operation

```
public int delete ( String table, String whereClause, String[] whereArgs )
```

The method is called to delete rows in a table. A filtering condition and its arguments are supplied in the call. The condition identifies the rows to be deleted. The method returns the number of records affected by the action.

Parameters

table	the table on which data is to be deleted
whereClause	This is the condition identifying the records to be deleted. For instance "name = ? " where ? Is a placeholder. Passing null deletes all the rows in the table.
whereArgs	Data to replace '?' placeholders defined in the whereClause.

Android's DELETE Operation

Example

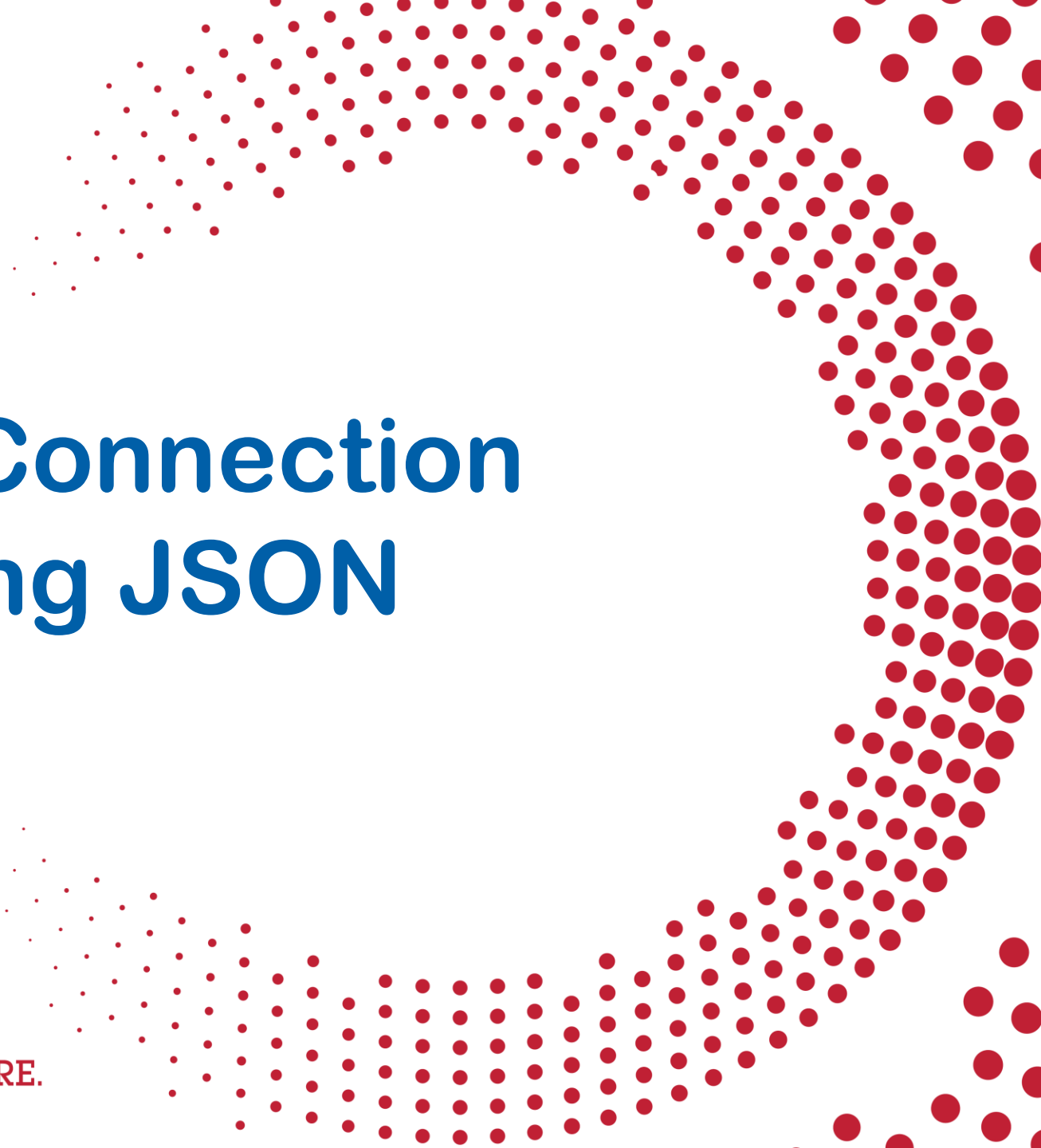
Consider the following SQL statement:

```
delete from tblAmigo where recID > 2 and recID < 7
```

An equivalent implementation using the Android's **delete method** follows:

```
val whereArgs = arrayOf("2", "7")  
val recAffected = db.delete("tblAMIGO",  
    "recID > ? and recID < ?", whereArgs)
```

A record should be deleted if its recID is in between the values 2, and 7. The actual values are taken from the *whereArgs* array. The method returns the number of rows removed after executing the command (or 0 if none).



Http Connection Parsing JSON

ONE LOVE. ONE FUTURE.



HTTP Connection

ONE LOVE. ONE FUTURE.

Steps to connect to the Internet

1. Add permissions to Android Manifest
2. Check Network Connection
3. Create Coroutines
4. Implement background task
 - a. Create URI
 - b. Make HTTP Connection
 - c. Connect and GET Data
5. Process results
 - a. Parse Results

Network Prerequisites

- The **<uses-permission>** element must be included in the AndroidManifest.xml resource so as to allow the application to connect to the network
- Permissions are used to ask the operating system to access any privileged resource
- The **<uses-permission>** tag causes the application to request to use an Android resource that must be authorized
 - The tag must be an immediate child of **<manifest>**

```
<uses-permission android:name="android.permission.INTERNET" />  
<uses-permission  
  android:name="android.permission.ACCESS_NETWORK_STATE" />
```

Getting network information for API $\geq$ 29

- ConnectivityManager
 - Answers queries about the state of network connectivity
 - Notifies applications when network connectivity changes
- NetworkCapabilities
 - Describes capability of a network interface of a given transport type
 - Cellular or Wi-Fi

Getting network information for API >= 29

```
val connectivityManager = getSystemService(Context.CONNECTIVITY_SERVICE) as ConnectivityManager
val networkCapabilities = connectivityManager.getNetworkCapabilities(connectivityManager.activeNetwork)
if (networkCapabilities == null) {
    Log.v("TAG", "No connection")
} else {
    if (networkCapabilities.hasTransport(NetworkCapabilities.TRANSPORT_WIFI)) {
        Log.v("TAG", "Has WIFI connection")
    }
    if (networkCapabilities.hasTransport(NetworkCapabilities.TRANSPORT_CELLULAR)) {
        Log.v("TAG", "Has cellular connection")
    }
}
```

Getting network information for API < 29

- ConnectivityManager
 - Answers queries about the state of network connectivity
 - Notifies applications when network connectivity changes
- NetworkInfo
 - Describes status of a network interface of a given type
 - Mobile or Wi-Fi

Check if network is available

```
ConnectivityManager cm = (ConnectivityManager)
getSystemService(Context.CONNECTIVITY_SERVICE);
NetworkInfo networkInfo = cm.getActiveNetworkInfo();
if (networkInfo != null && networkInfo.isConnected())
    Log.v("TAG", "Connected");
else
    Log.v("TAG", "No network connection");
```

Check for Wifi and mobile

```
NetworkInfo networkInfo =  
    connMgr.getNetworkInfo(ConnectivityManager.TYPE_WIFI);  
boolean isWifiConn = networkInfo.isConnected();  
  
networkInfo =  
    connMgr.getNetworkInfo(ConnectivityManager.TYPE_MOBILE);  
boolean isMobileConn = networkInfo.isConnected();
```

Protocols

- Android supports several different network protocols.
 - TCP / IP (through the **Socket** class)
 - SMTP (through the **GMailSender** class)
 - HTTP
 - And others
- In this lesson, you will work with HTTP

HTTP

- Hyper Text Transfer Protocol
- In our case the transfer protocol is HTTP
 - We connect the client device to a server and get data
 - We then process that data somehow
 - We might render a Web page
 - We might parse and process XML
 - Or any other message

HTTP

- Two HTTP clients
 - `HttpClient`
 - `HttpURLConnection`
- Both support HTTPS and IPV6
- Use `HttpURLConnection` for post Lollipop devices (version 5.x)

The URL class

- The `java.net.URL` class represents a url
 - Convert strings to URLs
 - Convert URLs to strings
- <http://developer.android.com/reference/java/net/URL.html>

The URL class

Protocol	http
Authority	username:password@host:8080
User Info	username:password
Host	host
Port	8080
File	/directory/file?query
Path	/directory/file
Query	query
Ref	ref

Opening a connection

- The `URL.openConnection()` method establishes a connection to a resource
- Over this connection, you make the request and get the response
- We will use HTTP here but other protocols are supported

Opening a connection

- The `setReadTimeout()` mutator defines the time to wait for data
- The `setConnectTimeout()` mutator the time to wait before establishing a connection
- The `setRequestMethod()` defines whether the request will be a GET or a POST

Opening a connection

- **getResponseCode()** gets the HTTP response code from the server
 - -1 if there is no response code.
 - Such as 404 not found?
- **getInputStream()** gets the input stream from which you can read data
 - Works just like an open file

Sending GET request

```
lifecycleScope.launch(Dispatchers.IO) {  
    val url = URL("https://httpbin.org/get")  
    val conn = url.openConnection() as HttpURLConnection  
    Log.v("TAG", "Response code: ${conn.responseCode}")  
  
    val reader = conn.inputStream.reader()  
    val content = reader.readText()  
    reader.close()  
  
    // TODO: Process result  
}
```

Download file

```
LifecycleScope.launch(Dispatchers.IO) {  
    val url = URL("https://lebavui.github.io/videos/ecard.mp4")  
    val conn = url.openConnection() as HttpURLConnection  
    Log.v("TAG", "Response code: ${conn.responseCode}")  
  
    val reader = conn.InputStream  
    val writer = openFileOutput("download.mp3", MODE_PRIVATE)  
  
    val buffer = ByteArray(2048)  
  
    while (true) {  
        val len = reader.read(buffer)  
        if (len <= 0)  
            break  
        writer.write(buffer, 0, len)  
    }  
  
    writer.close()  
    reader.close()  
}
```

Sending POST request

```
LifecycleScope.launch(Dispatchers.IO) {  
    val url = URL("https://httpbin.org/post")  
    val conn = url.openConnection() as HttpURLConnection  
  
    conn.requestMethod = "POST"  
    val params = "user=admin&password=123456"  
    conn.doOutput = true  
    val writer = conn.outputStream.writer()  
    writer.write(params)  
    writer.flush()  
    writer.close()  
  
    Log.v("TAG", "Response code: ${conn.responseCode}")  
  
    val reader = conn.inputStream.reader()  
    val content = reader.readText()  
    reader.close()  
  
    // TODO: Process result  
}
```

3rd party libraries

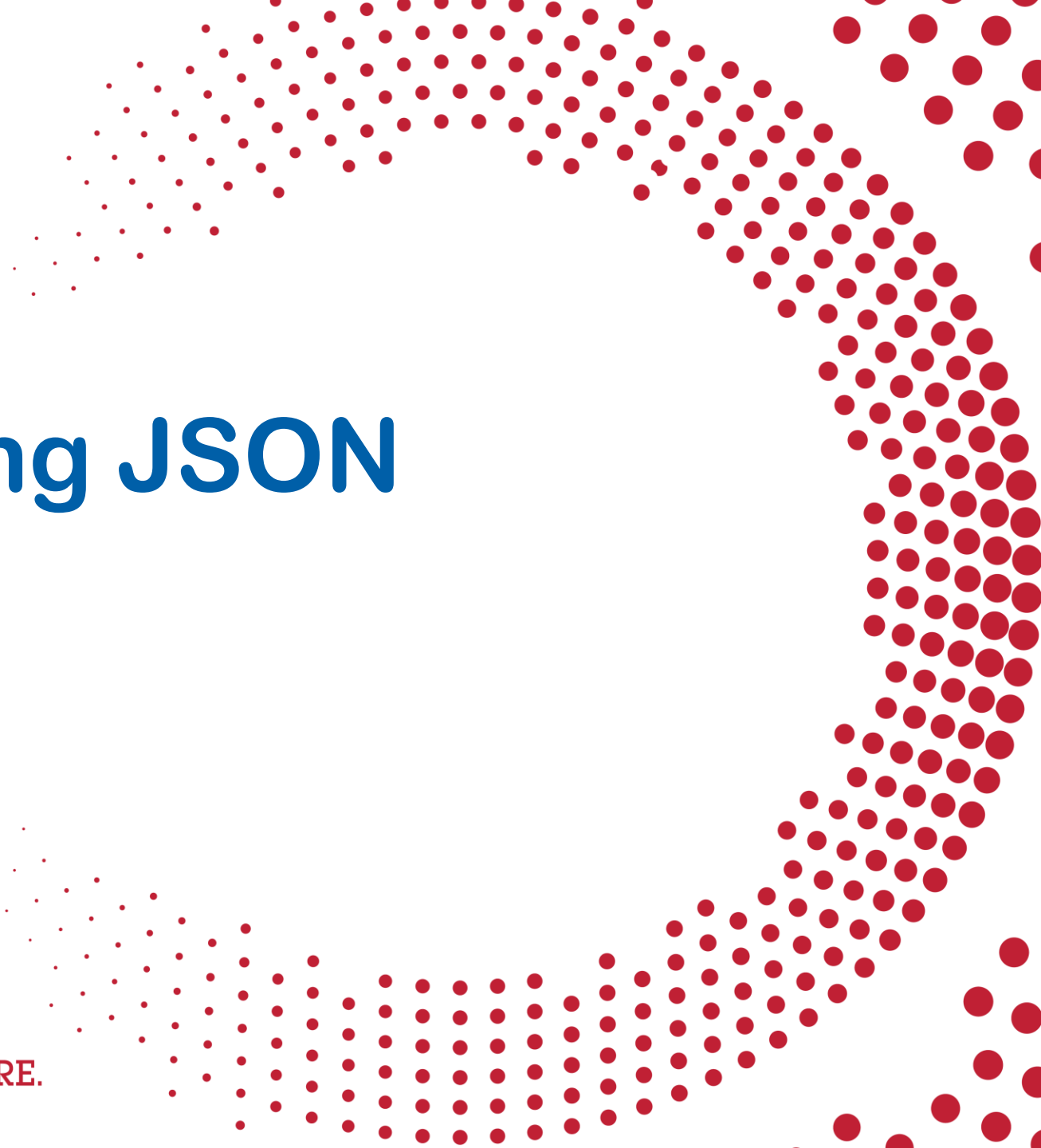
Http Client:

- Volley <https://github.com/google/volley>
- OKHttp <http://square.github.io/okhttp/>
- Retrofit <http://square.github.io/retrofit/>

Image Loader:

- Picasso <http://square.github.io/picasso/>
- Universal Image Loader

<https://github.com/nostra13/Android-Universal-Image-Loader>



Parsing JSON

ONE LOVE. ONE FUTURE.

What is JSON

- **JavaScript Object Notation**
- JSON is a syntax for storing and exchanging data
- JSON is text, written with JavaScript object notation
- Completely language independent
- Easy to understand, manipulate and generate

JSON syntax

- Data is in name/value pairs

```
"name": "John"
```

- Data is separated by commas

```
"name": "John", "age": 20, "gender": "male"
```

- Curly braces hold objects

```
{"name": "John", "age": 20, "gender": "male"}
```

- Square braces hold arrays

```
[{"name": "John", "age": 20, "gender": "male"}, {"name": "Peter", "age": 21, "gender": "male"}, {"name": "July", "age": 19, "gender": "female"}]
```

JSON values

- In JSON, values must be one of the following data types:

- A string

```
{ "name": "John" }
```

- A number

```
{ "age": 30 }
```

- An object

```
{ "employee": { "name": "John", "age": 30, "city": "New York" } }
```

- An array

```
{ "employees": [ "John", "Anna", "Peter" ] }
```

- A Boolean

```
{ "sale": true }
```

- Null

```
{ "middlename": null }
```

Parsing JSON string

- Basic types such as *strings*, *numbers*, and *Boolean values*, are represented in Java as their corresponding types (String, int/double, boolean, respectively).
- Compound types are represented using classes in the org.json package.
 - JSON arrays are represented by the class org.json.JSONArray;
 - JSON objects are represented by the class org.json.JSONObject.
- *Null values* are represented by the instance JSONObject.NULL.

Parsing JSON string

- To parse compound JSON data from a String, create a new Java object of the appropriate type, passing the String as the only argument to the constructor.

```
// Parsing JSON object  
val jsonData = JSONObject(jsonString)
```

```
// Parsing JSON array  
val jsonData = JSONArray(jsonString)
```

Parsing JSON string – Retrieving data

- Values for the keys in a JSONObject may be obtained using ***get*(String key)*** methods
 - ***getBoolean(key)*** will get a boolean value
 - ***getInt(key)*** will get an int value
 - ***getString(key)*** will get a String value
 - ***getJSONObject(key)*** will get a JSONObject value
 - ***isNull(String key)*** may be used to test if the value of a key is null. It will also return true if key does not exist in the JSONObject
- ***keys()*** will return an iterator Java object (java.util.Iterator) which you can use to iterate through the keys in a JSONObject.
- Values in a JSONArray may be obtained using ***get*(int index)***
- Both JSONObject and JSONArray provide the ***count()*** method to return the number of items in the *object/array*

Example: Parsing JSON data from URL

- Sample URL contains JSON data
<https://jsonplaceholder.typicode.com/users>
- Steps:
 - Use coroutine to get data in background
 - Implement GET request by using URLConnection
 - Parse JSON by using JSONArray and JSONObject classes
 - Show data using ListView

Serialize object to JSON

Convert JAVA object to JSON string

- Use GSON library <https://github.com/google/gson>
- Main functions:
 - toJson() serialize object to Json
 - fromJson() deserialize json to object
- Example:

```
val gson = Gson() // Or use new GsonBuilder().create();
val obj1 = MyType()
val json = gson.toJson(obj1) // serializes obj1 to Json
val obj2 = gson.fromJson(json, MyType.class) // deserializes json into obj2
```

A decorative background consisting of a large number of red dots of varying sizes. These dots are arranged in a circular pattern, with the density of the dots increasing towards the right side of the image, creating a sense of depth and movement.

Working with Socket class

ONE LOVE. ONE FUTURE.

Working with Socket class

Setup a client socket

```
lifecycleScope.launch(Dispatchers.IO) {  
    val serverAddress = InetAddress.getByName("192.168.1.26")  
    client = Socket()  
    client?.connect(InetSocketAddress(serverAddress, 8000))  
  
    withContext(Dispatchers.Main) {  
        if (client?.isConnected == true) {  
            Toast.makeText(applicationContext, "Connected", Toast.LENGTH_LONG).show()  
  
            // TODO: Read message from server  
  
        }  
        else  
            Toast.makeText(applicationContext, "Connect Failed", Toast.LENGTH_LONG).show()  
    }  
}
```

Receive data from server

```
withContext(Dispatchers.IO) {  
    val reader = client?.getInputStream()?.reader()  
    val bufferedReader = BufferedReader(reader)  
    while (client?.isConnected == true && client?.isClosed == false) {  
        val line = bufferedReader.readLine()  
        if (line.isNullOrEmpty())  
            break  
        Log.v("TAG", "Received: $line")  
        withContext(Dispatchers.Main) {  
            binding.textLog.append(line)  
        }  
    }  
    Log.v("TAG", "Disconnected")  
}
```

Working with Socket class

Send data to server

```
lifecycleScope.launch(Dispatchers.IO) {  
    if (client?.isConnected == true && client?.isClosed == false) {  
        val writer = client?.getOutputStream()?.writer()  
        writer?.write(message)  
        writer?.flush()  
  
        withContext(Dispatchers.Main) {  
            binding.editMessage.setText("")  
        }  
    }  
}
```